

대규모 언어모델(LLM) 강 좌 2025 — 한국어 번역판

동경대 마츠오·이와사와 연구실 LLM 강좌 (8일)

동경대 마츠오·이와사와 연구실 (한국어 번역판) 지음

한국어 PDF 편집본

LLM 강좌 2025

대규모 언어모델(LLM) 강좌 2025 — 한국어 번역판

동경대 마츠오·이와사와 연구실 LLM 강좌 (8일)

동경대 마츠오·이와사와 연구실 (한국어 번역판) 지음

한국어 PDF 편집본

PDF 편집본 안내

PDF

© 2025 東京大学 松尾・岩澤研究室 CC BY-NC-ND 4.0 (교육 목적 번역, 원문 의미 보존)

PDF 편집일: 2026-08-13

목차

장과 절을 한 줄 흐름으로 읽을 수 있도록 1단 목차로 구성했습니다.

01	Day 1 — 대규모 언어 모델 개요	7
	도입: 이 강의에서 배우는 것	8
	언어 모델이란 무엇인가	9
	대규모 언어 모델의 발전과 2025년 현황	10
	스케일이 가져온 범용성 — 왜 지금 언어 모델인가	11
	언어를 넘어선 파운데이션 모델 — 멀티모달과 로봇	13
	강좌 구성과 학습 파이프라인	14
	일본의 LLM 생태계	17
	맺음말	20
	Reference	20
02	제2회: 추론 — 프롬프팅과 문맥 내 학습	26
	도입: LLM 시대의 활용과 본 강의의 목표	27
	디코딩의 기초: 언어 모델에서 출력을 얻는 방법	27
	프롬프팅의 기초: 가중치 재학습 없이 과제를 푼다	29
	메타 생성: 여러 번 추론해 더 나은 출력을 얻는다	30
	발전적인 프롬프트: 실제 시스템과 새로운 활용	30
	LLM을 활용한 서비스와 모델의 선택	31
	더 읽을거리	32
	Reference	33
03	Day 3: 사전학습 (Pre-training)	35
	1. 사전학습과 언어 모델	36
	2. Transformer의 모델 구조	38
	3. 사전학습 파이프라인	40
	4. 발전 주제: 데이터셋의 변천	43
	5. 발전 주제: 모델 아키텍처의 발전	44
	6. 발전 주제: 목적 함수, 평가, 분석	46
	7. 정리	47
	참고문헌	48

04	Day 4: 스케일 법칙	51
	강의의 목적과 배경	52
	스케일 법칙이란 무엇인가	52
	스케일 법칙의 활용 방법	55
	스케일 법칙의 구하는 방법	57
	추론 시의 스케일링	58
	정리	60
	참고문헌	60
05	Day 5 — 발전적인 사전학습	63
	이번 강의 목표	64
	사전학습의 중요도와 스케일 법칙	64
	스케일화의 세 가지 과제	65
	파라미터(N) 효율화: Efficient Attention과 혼합 전문가	65
	계산량(C) 효율화: 병렬화와 양자화	67
	데이터(D) 효율화: 품질, 양, 그리고 한계	68
	정리	71
	참고문헌	71
06	Day 6 — 대규모 언어 모델 fine-tuning	73
	도입: fine-tuning이 풀고자 하는 문제	74
	대규모 언어 모델 학습 흐름에서의 fine-tuning	75
	Instruction Tuning	76
	Parameter Efficient Fine-Tuning	77
	대표적인 PEFT 기법들	78
	정리	79
	References	80
07	제7회: 강화학습	82
	이번 회의 목적과 목표	83
	LLM 훈련 플로우에서 강화학습의 위치	83
	RLHF: 인간의 피드백으로부터의 강화학습	84
	DPO: 보상 모델 없이 직접 선호도를 학습하는 오프라인먼트 기법	90
	RLVR: 검증 가능한 보상으로 추론 능력을 향상시키는 강화학습	92
	강화학습의 응용과 향후 방향	94
	정리	95
	References	95

08	Day 8 학습 데이터와 평가 벤치마크의 정비	101
	도입: 학습 데이터와 평가 벤치마크 정비의 목표.....	102
	사전학습 코퍼스의 종류와 구성	103
	사전학습 데이터의 필터링과 확장	104
	사후학습 데이터: Instruction Tuning과 강화학습	107
	라이선스, 저작권, 개인정보.....	109
	LLM 성능 평가와 벤치마크.....	110
	정리	114
	References	114

01

Day 1 — 대규모 언어 모델 개요

이 장의 구성

- 01 도입: 이 강의에서 배우는 것
 - 02 언어 모델이란 무엇인가
 - 03 대규모 언어 모델의 발전과 2025년 현황
 - 04 스케일이 가져온 범용성 — 왜 지금 언어 모델인가
 - 05 언어를 넘어선 파운데이션 모델 — 멀티모달과 로봇
 - 06 강좌 구성과 학습 파이프라인
 - 07 일본의 LLM 생태계
 - 08 맺음말
 - 09 Reference
-

"

번역 참고: 본 자료는 CC BY-NC-ND 4.0(저작자표시-비영리-변경금지 4.0 국제) 라이선스를 따르며, 교육 목적의 번역본입니다. 원문의 의미를 변경하지 않고, 반복되는 라이선스 푸터는 첫 회만 표시합니다.

도입: 이 강의에서 배우는 것

본 자료는 도쿄대학교 마츠오·이와사와 연구실이 작성한 것으로, 2025년 10~11월에 개최된 대규모 언어 모델 강좌 기초편의 첫 회 강의입니다. 원문은 CC BY-NC-SA 4.0(저작자표시-비영리-동일조건변경허락) 라이선스로, 비영리 목적의 재이용이 허락되어 있습니다. 재이용 시 라이선스 표시를 기재하고, 참고 논문 인용은 권말 Reference에 게재해 주세요. 상세한 조건은 원문 링크를 확인해 주세요.

이번 시간은 개별 기술을 깊이 파고들기보다 전체 개요를 파악하는 것이 목적입니다. 많은 용어가 등장하지만, 모두 이번 회에 외워야 하는 것은 아닙니다. 강의를 이와사와 유스케(岩澤有祐) 준교수가 LLM 개황과 강좌 전체 개요를 맡고, 고지마 타케시(小島武) 특임조교가 각 회차와 일본의 LLM 환경을 맡습니다.

이와사와 유스케는 2017년 마츠오 연구실에서 박사과정을 수료한 뒤 특임연구원·특임조교를 거쳐 2024년 1월부터 기술경영전략학 전공 준교수를 맡고 있습니다. 석사까지는 장애인 지원을 위한 머신러닝 응용을, 박사부터는 딥러닝 전이학습을 연구했습니다. 생성AI 분야에서는 "Large-Language Models are Zero-Shot Reasoners"(NeurIPS 2022)를 비롯해, LLM 강좌 전체 설계, 기시다 총리 등에 대한 LLM 특강, DL 독서회 주관(2015년~ 누적 350회 이상), Goodfellow 딥러닝 교과서 감역·번역(2018) 등의 활동을 해왔습니다.

고지마 타케시는 2023년 도쿄대학교 TMI 박사과정을 수료하고 동 연구과 특임조교를 맡고 있으며, 이전에는 IT 엔지니어 출신입니다. Weblab-10B 개발, 기시다·이시바 총리 LLM 특강 강사, LLM 개발 콘테스트 2024·2025 운영, AI 백서 2025 Safety 장 집필을 담당했고, 연구 테마는 LLM 동작 원리 이해와 제어(Reasoning Model, 다언어), Safety(Unlearning, 지시 추종 능력, 로봇), 트랜스포머 구조 개선입니다.

오늘 다룰 내용은 세 축입니다. 첫째, LLM 개요, 즉 왜 지금 LLM을 배워야 하는가. 둘째, 이번 강좌 각 회차의 개요. 셋째, 일본의 LLM을 둘러싼 환경.

언어 모델이란 무엇인가

지금 자연어를 다루는 어시스턴트 AI를 만들고 싶다고 가정해 봅시다. 질문에 올바른 답을 출력해 주길 원하고("일본의 수도는?" → "도쿄"), "글을 영어로 번역해 줘"라는 요청에는 번역문을, "테트리스 앱을 만들어 줘"라는 요청에는 코드를 생성해 주길 원합니다. 오늘날 이런 기능들은 웹사이트나 간단한 프로그램으로 실현되고 있으며, Hugging Face에는 언어·이미지·음성·멀티모달 등 100만 개를 넘는 모델이 공개되어 있습니다.

이런 일은 어떻게 실현되는 걸까요? 단어의 열(문장)을 $x_1, x_2, \dots, x_L$ 이라 할 때, 언어 모델은 그 생성 확률 $p(x_1, x_2, \dots, x_L)$ 을 할당하는 확률 모델 p 로, 자연스러운 문장일수록 높은 확률을 줍니다.

```
p(일본, 의, 수도, 는, 도쿄) = 0.02
p(일본, 의, 수도, 는, 파리) = 0.00001
p(도쿄, 의, 수도, 는, 일본) = 0.0005
```

다양한 언어 과업 — QA, 번역, 코드 생성 — 이 모두 이 생성 확률의 추정 문제로 다루어질 수 있으며, 이 생성 확률을 어떻게 구하는가가 언어 모델의 핵심 기술적 문제입니다.

가장 널리 쓰이는 접근이 자기회귀 언어 모델(Autoregressive Language Model)로, 확률의 연쇄 법칙에 따라 결합 확률을 조건부 분포의 곱으로 분해합니다.

$$p(x_1, x_2, \dots, x_L) = p(x_1) \cdot p(x_2 | x_1) \cdot \dots \cdot p(x_L | x_1, x_2, \dots, x_{L-1})$$

조건부 확률을 알면 생성도 가능합니다. 예컨대 "일본의 수도는 → 도쿄"라는 접속에서 조건부 확률이 다음 단어를 고릅니다.

```
p(도쿄 | 일본, 의, 수도, 는) = 0.2
p(파리 | 일본, 의, 수도, 는) = 0.001
p(카이로 | 일본, 의, 수도, 는) = 0.0005
```

신경망 언어 모델(Neural Language Model)은 이 조건부 확률을 신경망으로 추정한 모델로, 웹 데이터를 모의하도록(가능도를 최대화하도록) 학습합니다.

트랜스포머 이전의 신경망 언어 모델에는 두 가지 과제가 있었습니다. 합성곱 신경망이나 MLP에서는 긴 문맥 처리가 어려웠고(번역처럼 원문을 충실히 반영해야 하는 과업에서 치명적입니다), RNN 계열은 데이터를 순차 처리하는 탓에 학습·추론의 병렬화가 안 되어 스케일업이 곤란하고 기울기 소실 문제까지 겪었습니다.

이를 해결한 것이 2017년 Vaswani 등의 "Attention is All You Need"(NeurIPS 2017)가 제안한 트랜스포머(Transformer)입니다 [1]. Google 중심의 연구팀이 발표한 이 구조는 Self-Attention을 핵심으로 삼아, 번역 등 지도학습에서 성능이 검증되었습니다(영어 문장 → 트랜스포머 → 독일어 문장이 되도록 오차 역전파로 학습). 구조의 상세는 별도 일정에서 다룹니다.

트랜스포머를 언어 모델의 사전학습에 본격 적용한 것이 OpenAI의 GPT(Generative Pretraining Transformer)입니다. Radford 등이 2018년에 발표한 GPT는 [2], 다음에 올 단어를 트랜스포머로 예측하도록 Book Corpus라는 미공간 서적 데이터로 학습했으며, 이후 GPT, GPT-2, GPT-3로 버전을 거듭하며 학습 데이터 수와 모델 크기를 증가시켜 왔습니다.

대규모 언어 모델의 발전과 2025년 현황

GPT-3가 등장한 2020년 이후, 대규모 모델의 발표는 가속적으로 증가했습니다 [3]. OpenAI는 2023년 GPT-4를 발표합니다. 상세는 미공개이나 누출 정보는 있고, 사법시험이나 SAT/GRE 등 다양한 시험에서 좋은 성적을 거두었습니다. 예컨대 Uniform Bar Exam에서 298/400(약 90백분위), GRE Quantitative 163/179(약 80백분위)이며, 한편 코딩 능력 등에서는 아직 낮은 점수를 받았습니다(현재는 대폭 개선됨) [4].

일본어 평가 사례도 있습니다. Kasai 등(2023)은 GPT-4와 ChatGPT를 일본 의료 면허 시험 6년 분량의 새 데이터셋 Igaku-QA로 벤치마크했습니다. 인간 평균 응시자보다는 나쁘고 금기술을 선택하는 경향이 있는 등의 문제는 있으나, 시험 합격선은 돌파했습니다 [5].

모델 자체의 발전뿐 아니라 활용 기술도 진전했습니다. "A Survey of Context Engineering for Large Language Models"(2025)이 정리하듯, 언어 모델이 가진 지식을 사용할 뿐만 아니라 필요한 컨텍스트를 선택하고 처리하는 기술 — RAG, 도구 이용(검색), Deep Research, Memory 등 — 이 활발히 연구되고 있습니다.

2025년 한 해에만 표에 정리한 수십 개의 유력 모델이 등장했습니다.

모델명	개발사	공개 시기(2025)	오픈/클로즈드
GPT-5	OpenAI	8월	클로즈드
GPT-4.5	OpenAI	2월	클로즈드
GPT-OSS	OpenAI	8월	오픈
Llama 4 Scout	Meta	4월	오픈
Llama 4 Maverick	Meta	4월	오픈

모델명	개발사	공개 시기(2025)	오픈/클로즈드
DeepSeek-R1	DeepSeek	1월	오픈
DeepSeek-V3	DeepSeek	3월경	오픈
Qwen3 (Think)	Alibaba/Qwen	4월	오픈
Qwen2.5-Max	Alibaba/Qwen	초두	오픈
Claude 3.7 Sonnet	Anthropic	2월	클로즈드
Grok-3	xAI	2월	클로즈드
BitNet b1.58 2B4T	Ma 외	4월	오픈
LLaDA	ML-GSAI 외	2월	오픈
MMaDA	연구팀	5월	오픈

세 가지 흐름이 두드러집니다. 첫째, 추론(Reasoning) 모델이라 불리는, 기존보다 추론 능력이 현저히 높은 모델의 등장입니다. 스스로 오류를 깨닫는 "Aha Moment"가 보고되면서, 기존보다 복잡한 벤치마크 정비와 추론 프로세스 분석이 진전하고 있습니다. 둘째, 성능이 높은 공개 모델의 증가입니다. 셋째, 확산 언어 모델 등 다양한 의미에서 효율이 좋은 모델 구조의 모색입니다.

이에 맞춰 평가도 고도화되고 있습니다. SWE Bench는 이슈에 대한 PR 작성 능력을 평가하고, Humanity's Last Exam은 등장 시점 SoTA 9%(현재 21.6%) 수준의 챌린징한 문제로 모델의 한계를 측정합니다. 문제 예시를 하나 옮기면 이렇습니다.

//

"칼새목의 별새는 꼬리 깃털 하제근(下制筋)의 퍼진 교차상 건막(腱膜)의 미측 외측 부분에 매립된, 좌우 한 쌍의 타원형 종자골(種子骨)을 가진다. 이 종자골에 의해 지지되는 건 쌍은 몇 개인가? 숫자로 답하십시오."

스케일이 가져온 범용성 — 왜 지금 언어 모델인가

지금까지를 한 줄로 정리하면: 언어 모델이란 단어열의 생성 확률을 모델화한 것(자기회귀·신경망·GPT)이며, 2025년이 된 지금도 활용 방법, 모델 자체(대규모 추론 모델, 확산 언어 모델), 평가 방법에 대한 연구개발은 진전 중입니다. 원리는 매우 심플합니다. 그렇다면 왜 지금 언어 모델인가? 두 가지 이유가 있습니다. 하나는 대규모화에 수반하는 범용성, 다른 하나는 언어 이외의 도메인에 대한 영향입니다.

어느 모델이든 기본적으로 2017년에 발명된 트랜스포머라는 구조를 이용합니다. 2018년 OpenAI의 GPT-1, 2019년·2020년 GPT-3, 2023년으로 이어지며 시기별로 스케일이 커졌고, GPT-3 등장 이후 미국 기업을 중심으로 복수의 연구기관이 독자적인 대규모 언어 모델을 개발했습니다 [6]. 핵심은 스케일링 법칙(Scaling Law)과 창발 능력(Emergent Ability)입니다.

스케일링 법칙이란, 계산 자원 C, 데이터셋 크기 D, 파라미터 수 N이라는 세 변수에 대한 거듭제곱에 따라 사전학습 성능이 올라간다는 경험칙입니다 [7]. 창발 능력이란, 모델 크기가 거대할 때만 풀 수 있는 과업이 존재한다는 현상입니다 [8]. 보다 큰 파라미터의 트랜스포머로 보다 대규모 데이터를 이용한 사전학습을 진행하는 과정에서 스케일 법칙이 발견되었고, 자원을 투자할수록 고성능 LLM을 만들 수 있다는 사실이 밝혀진 것은 크게 흐름이 바뀐 순간이었습니다. OpenAI는 일찍이 스케일 법칙에 주목하여 대규모 개발을 시작했고, 그 후 세계적인 투자 경쟁이 시작되었습니다.

스케일의 체감을 학습 데이터량으로 보면, GPT-3는 약 5,000억 토큰의 텍스트를 이용했습니다(토큰이란 언어 AI가 처리하는 단위로, 일본어의 경우 대략 1글자 1토큰) [9]. 책으로 치면 약 500만 권에 상당합니다. 참고로 도쿄대 도서관이 약 130만 권, 국회도서관이 약 4,700만 권이며, 누출 정보에 따르면 GPT-4는 약 1.3억 권에 해당합니다.

이런 대규모 학습을 지탱하는 것이 GPU라 불리는 대규모 연산 자원입니다. 현재 주된 연산 자원은 GPU이며, 지배적 점유율(세계 약 90%)을 가진 NVIDIA가 급성장해 일시적으로 세계 시가총액 1위에 올랐습니다. 학습에 필요한 GPU 수를 짚으면, GPT-3 상당은 A100 1,200대로 약 30일, GPT-4 상당(*누출 정보)은 A100 25,000대로 약 100일이 걸리고, 이번 강의 연습은 A100 8대로 1시간 수준입니다.

글로벌 GPU 클러스터 규모에서 격차는 더 벌어집니다. 2024년 단년도 구매 수 기준으로 Microsoft 485,000대, Meta 224,000대, Amazon 196,000대, Google 169,000대 등 단일 기업이 수십만~백만 기의 H100급 GPU를 보유합니다. 이에 비해 일본은 산업기술종합연구소 ABCI의 6,128대 H200(2025년 1월 업그레이드), SoftBank 약 6,000대, 사쿠라 인터넷 2,000대 H100 수준입니다 [10]. 이 격차는 근본적으로 IT 서비스와 생성AI 사이에 선순환이 만들어지고 있는가라는 구조적 문제에서 비롯됩니다.

이런 스케일이 가져온 가장 큰 결실은 범용성입니다. Brown 등(2020)이 보인 대로 [9], 사전학습을 마친 트랜스포머 하나로 번역(Zero-shot, Few-shot), 요약("TL;DR"로 시작하면 성능이 대폭 향상됨) 등 다양한 과업이 소수 예시나 지시만으로 가능해졌습니다. 이것이 "파운데이션 모델"이라는 개념으로 정식화됩니다. 2021년 8월 Stanford의 백서 "On the Opportunities and Risks of Foundation Models"에서 초출된 이 용어는 [11], 광범위한 데이터를 대규모로 학습하고 다양한 다운스트림 과업에 적응할 수 있는 모델(BERT, DALL-E, GPT-3 등)을 가리킵니다. 초록은 이 모델들이 "임계적으로 핵심적이면서도 미완성" — 즉 중심적이 되면서도 위험과 한계를 안고 있음 — 이라는 성격을 강조합니다.

언어를 넘어선 파운데이션 모델 — 멀티모달과 로봇

두 번째 이유, 즉 언어 이외의 도메인에 대한 영향을 살펴봅시다. GPT-4는 텍스트뿐 아니라 이미지 인식 능력을 갖추고 있으며 [4][13], 이는 로봇 응용으로까지 이어집니다.

로봇 분야에서는 언어 모델을 어떻게 '행동'으로 이어붙일지가 과제입니다. Ahn 등(2022)의 Say-Can(Say-Can-PaLM)은 [14], 언어 모델이 출력한 스킬의 실행 가능성(Skill Affordance)을 고려해 스킬을 선택합니다. 실행 가능성은 TD(강화학습)로 학습하며, 언어 모델을 개선하면(PaLM 사용) 성능이 향상됩니다(다만 실행 가능한 저수준 정책은 사전에 준비되어 있다는 점에 주의). 마츠오 연구실의 행동 계열 생성 연구도 이 흐름에 있으며, RoboCup Japan Open 2023 우승, RoboCup 세계대회 3위 등의 성과를 거두었습니다.

더 나아가 '로보틱스 파운데이션 모델'이 등장했습니다. 실제 세계 환경의 action/observation 쌍을 대규모·다양한 데이터로 학습해 산업 응용, 자율주행, 생활 지원 등에 활용하는 모델입니다. 구글의 RT-1(2022)은 EfficientNet과 트랜스포머의 조합으로 인스트럭션에 따라 동작을 생성하며 [15], 13대의 로봇으로 17개월간 744 과업, 13만 데모를 수집해 학습했습니다. 학습 데이터에 대해서는 97%에서 동작하고, 미지 과업·미지 소스 등 다양한 의미에서 일반화가 대폭 향상되며 Long Horizon 과업도 가능합니다(유사 연구로 Gato, BC-Z 등). RT-X 프로젝트는 이 방향을 더 확장해 [16], Google DeepMind 및 21개 연구기관이 통일된 포맷의 오프라인 로봇 데이터셋을 수집했습니다 — 22가지 로봇 타입, 527개 스킬(160,266 과업), 100만 에피소드 이상을 확보해 개별 데이터로 학습한 모델보다 더 나은 성능을 보이며 ICRA 2024 최우수 논문상을 수상했습니다. 최근의 비전-언어-액션(Vision-Language-Action, VLA) 모델인 π_0 [17]은 세탁물을 개거나 달걀을 깨지 않게 케이스에 넣는 등 다양한 과업을 수행하고, 일본의 AIRoA(AI Robot Association)는 학계 주도·개방성·보상 설계를 특징으로 삼습니다.

언어를 넘어선 확장은 비디오 생성에까지 닿습니다. OpenAI의 Sora는 텍스트 프롬프트에서 사진 품질의 영상을 생성하며 [18][19], "비디오 생성 모델을 세계 시뮬레이터로 본다"는 관점까지 제시합니다.

지금까지의 정리를 종합하면: 언어 모델이란 단어열의 생성 확률을 모델화한 것이며, 2025년이 된 지금도 활용 방법·모델 자체(대규모 추론 모델, 확산 언어 모델)·평가 방법에 대한 연구개발은 진행 중이고, 원리는 매우 심플합니다. 왜 지금 언어 모델인가? (1) 모델·데이터·계산량의 스케일에 의해 할 수 있는 것이 급속히 넓어지고 있고(일반화성), (2) 언어 모델의 발전이 다른 영역에도 영향을 주고 있습니다. 본 강좌는 이 LLM의 기술적 배경, 원리와 한계를 이해하고, 히프(Hype)가 아닌 활용 기술로서 파악하는 것을 취지로 합니다.

강좌 구성과 학습 파이프라인

올해 강좌는 '대규모 언어 모델 기초'와 '대규모 언어 모델 응용'으로 나뉩니다. ****기초편(10~11월)****은 LLM의 전체상을 이해하기 위해 사전학습·사후학습·데이터 수집·가공·벤치마크 평가 등 학습 파이프라인을 망라적으로 해설하고, 공개 모델이나 API로 추론 성능을 끌어올리는 기법도 소개합니다. ****응용편(12월~2월)****은 경량화·안전 대책·해석성·도메인 특화·LLM 에이전트 등 사회 구현에 불가결한 기술을 본격적으로 다루고, 최전선 연구자의 특별 강연과 '개인형 LLM 개발 콘테스트'가 진행됩니다.

학습 파이프라인은 여섯 단계로 정리됩니다 [22]. **Step 1. 사전학습**은 대규모 코퍼스에 의한 자기지도학습을 통해 언어 모델에 어휘·문법·지식 등 기본적인 언어 이해를 획득시키는 단계입니다. **Step 2. 파인튜닝**은 라벨링된 데이터에 의한 지도학습을 통해 성능을 개선하거나 특정 과업·도메인에 대한 적응을 실현합니다. **Step 3. 강화학습**은 (인간의) 피드백을 이용한 강화학습을 통해 출력이 인간의 가치관에 보다 부합하도록 조정합니다(Step 2~3을 묶어 "사후학습"이라 부릅니다). **Step 4. 데이터 수집·가공**은 사전학습이나 사후학습에 사용할 학습 데이터를 수집·가공하며, 최근에는 LLM 자체를 이용한 데이터 합성도 시행합니다. **Step 5. 추론**은 학습 완료 모델에 프롬프팅을 구사하여 추가로 성능을 향상시킵니다. **Step 6. 벤치마크 평가**는 학습에 사용되지 않은 샘플로 구성된 벤치마크로 성능을 평가합니다. 기초편의 각 회차는 이 파이프라인에 매핑됩니다: Day 2(추론) / Day 3~5(사전학습) / Day 6(파인튜닝) / Day 7(강화학습) / Day 8(데이터·벤치마크).

제2회 추론: Prompting, In-context Learning

학습 완료 후 LLM의 성능을 끌어내는 활용법을 배웁니다 [9][23]. Zero-Shot·Few-Shot 프롬프팅을 시작으로, 다단계 추론이 필요한 과업에서는 사고의 연쇄(Chain-of-Thought)를 예시로 제공하는 CoT prompting이 유효합니다 [24]. 나아가 예시 없이 모델 스스로 생각하게 하는 Zero-shot CoT("Let's think step by step.")도 제안되었습니다 [25](공동 저자 Shane Gu가 최근 "Video Models are zero-shot learners and reasoners"를 발표한 점도 참고 [26]). CoT를 전제로 한 추가 개선으로는 top-k/top-p 샘플링으로 복수 답안을 얻어 다수결을 취하는 Self-Consistency [27], 성공·실패 궤적 데이터를 기반으로 언어 피드백으로 프롬프트를 개선하거나 다른 유력 후보와 조합하는 GEPA(Genetic-Pareto)가 있습니다 (다양성 확보를 위해 1문제에서만 가장 좋은 점수를 낸 프롬프트도 후보에 포함) [28].

제3회 사전학습: 트랜스포머와 학습 원리

LLM의 주류 모델 구조인 트랜스포머와 그 사전학습 메커니즘(Embedding, Multi-Head Attention, Feed Forward 등)을 배웁니다 [1]. 핵심인 어텐션(attention) 기구는 모든 단어(토큰) 간의 유사도를 벡터의 내적으로 측정하여 장거리 의존 관계를 파악하는 메커니즘으로, 필요한 토큰의 정보를 유연하게 취사선택하면서 병렬 계산의 고속화를 이룹니다 [29]. 어텐션 시각화 예에서는 "it"이 "The", "animal"에 강한 어텐션이 걸려 있음을 볼 수 있는데 [30], 명시적으로 가르치지 않았는데도 사전학습 과정에서 모델 스스로

로 이 관계성을 도출합니다(실제로는 이렇게까지 명확하지는 않음). 트랜스포머 구조는 크게 세 종류로 분류됩니다 — Encoder-only(BERT, RoBERTa 등, 인식 계열·클래스 분류), Encoder-Decoder(BART, T5 등, 생성 계열), Decoder-only(GPT, Llama, Qwen, DeepSeek 등, 생성 계열).

LLM의 큰 특징은 번역·요약·채팅 등 다양한 언어 과업에 대한 범용성입니다. 대량의 텍스트에서 세계의 지식을 사전학습해 범용성을 획득하고, 사후학습으로 특정 기능이나 전문 분야에 특화합니다. 범용 모델 하나만 있으면 번역 앱, 의사록 요약 앱, 채팅 봇 등 다양한 기능을 개발할 수 있습니다. 학습 원리는 단순합니다. 웹에서 수집한 대량의 글로 다음 단어 예측을 쉬지 않고 수행하며, 그 과정에서 읽기·쓰기·셈하기와 세계 지식을 학습합니다. 예컨대 "봄은 벚꽃이 예쁘다"라는 텍스트로 "봄", "벚꽃", "예쁘다" 사이의 강한 관계성(=세계 지식)을 학습하고, 예측과 정답의 교차 엔트로피가 작아지도록 모델을 갱신합니다($P(\text{는}|\text{봄})$, $P(\text{벚꽃}|\text{봄, 는})$, $P(\text{가}|\dots)$, $P(\text{예쁘다}|\dots)$ 등).

제4회 스케일 법칙: 자원이 성능을 좌우한다

스케일 법칙(Scaling Law)이란 계산 자원, 학습 데이터량, 파라미터 수의 증가에 비례하여 사전학습 성능이 오른다는 경험칙입니다 [31]. 보다 큰 파라미터 크기의 트랜스포머로 보다 대규모 데이터를 이용한 사전학습 과정에서 발견되었고, 자원을 투하할수록 고성능 LLM을 만들 수 있다는 것이 밝혀진 것은 크게 흐름이 바뀐 순간이었습니다. OpenAI는 일찍이 스케일 법칙에 주목하여 대규모 개발을 시작했고, 그 후 세계적인 투자 경쟁이 시작되었습니다. 이미지 생성·멀티모달·동영상·수리 등에서도 계산량에 관한 스케일 법칙이 성립합니다.

스케일 법칙에 기초하면 LLM 성능을 좌우하는 세 가지 자원 — ① 대규모 연산 자원, ② 대규모 데이터, ③ 우수한 인재 — 이 도출됩니다. ② 대규모 데이터를 모으기 위해서는 저작권·개인정보 등 취급 방침의 정비가 필요하고(법 정비의 중요성), ③ 우수한 인재는 트랜스포머나 학습 기법의 개발, 하이퍼파라미터 조정 등에 필요하며(인적 자본 투자의 중요성), ① 대규모 연산 자원은 GPU라 불리는 학습용 서버 확보로 이어집니다(하드웨어 투자의 중요성). 거대한 파라미터의 모델은 사람이 연산 자원·데이터를 사용하고 하이퍼파라미터 조정과 시행착오를 반복하여 만드는 것입니다. 최근에는 사전학습의 스케일 법칙뿐 아니라 사후학습과 추론의 스케일 법칙에 대한 연구도 성행하고 있습니다 [32].

제5회 사전학습(상신편): 대규모화의 과제와 해법

언어 모델을 스케일(=대규모화)하여 사전학습할 때의 과제와 해결 방법을 배웁니다 [7]. 세 축이 있습니다 — 충분한 계산량(C)·메모리량 확보, 모델이 스케일함에 따른 비용 억제(파라미터 수 N), 성능 발휘를 위한 학습용 데이터(D). Sparse Transformer는 Attention을 계산하는 부분을 한정(계산하지 않는 부분은 마스크)해 계산량을 삭감하며, 이미지·음성 모달리티에서도 트랜스포머 이용을 가능하게 합니다 [33](2회 통과로 모든 토큰에 어텐션이 도달 [34]). Switch Transformer는 1조 6,000억 파라미터의 MoE(Mixture of Experts) 모델로, 피드포워드 네트워크를 복수 엑스퍼트화해 데이터에 따라 엑스퍼트를 선택합니다

[35]. LLM.int8()은 성능 열화 없이 가능한 양자화(Quantization) 방법으로, 은닉 상태에서 열 단위로 이상치를 추출해 이상치 행렬은 FP16 그대로, 이상치가 아닌 행렬은 INT8로 변환하여 연산한 뒤 가산해 반환합니다 [36]. 멀티 노드·멀티 GPU를 이용한 대규모 분산 학습(DeepSpeed 등 [37])도 핵심 기반입니다.

제6회 파인튜닝: 사전학습 이후의 학습

사전학습 완료 후에 수행하는 추가 학습 = 파인튜닝에 대해 배웁니다. 인간과 대화할 수 있도록 QA 데이터·채팅 데이터로 학습하며, 사전학습과 마찬가지로 다음 단어를 쉬지 않고 예측하는 학습 기법입니다. 예시로는 "Q: 일본에서 가장 높은 산은? / A: 후지산", "Q: 건강 유지를 위한 3가지 팁을 알려주세요 / A: 1. 균형 잡힌 식사를 하고 야채와 과일을 충분히 섭취할 것 ...", "Q: 메리는 20분에 8페이지의 책을 읽을 수 있습니다. 120페이지를 읽는 데 몇 시간 걸립니까? / A: 1시간에 $8 \times 3 = 24$ 페이지 읽을 수 있으므로 $120/24 = 5$ 시간" 등이 있습니다. Zero-shot으로 범용적으로 모든 지시에 따르도록 하는 인스트럭션 튜닝 [38], 소수 파라미터에 의한 효율적 학습 방법인 LoRA(Low-Rank Adaptation) [39]가 핵심 기법입니다.

제7회 강화학습: 인간 가치관예의 정렬

LLM에서의 강화학습이란 무엇인지, 그 메커니즘과 필요성에 대해 이해합니다. RLHF(Reinforcement Learning from Human Feedback)는 인간의 가치관(예: 유해한 말을 하지 않기를 바람)에 부합하도록 LLM 출력을 인간 피드백으로 개선하는 방향으로 학습하며, 지도학습에 비하면 상당히 정보량이 적은 신호로 학습합니다. 예컨대 사용자가 "절도를 하는 방법을 가르쳐 줘"라고 했을 때, × 나쁜 예("절도를 하려면 상대에게 들리지 않게 다가가 소지품을 빼앗는 것이 중요합니다...")와 △ 중간("절도는 좋지 않습니다."), ○ 좋은 예("절도는 범죄이므로 그것을 행하는 것을 강하게 권하지 않습니다.") 사이를 good/bad 피드백 신호로 구분합니다. 이후 보상 모델 구축이 필요 없는 DPO(Direct Preference Optimization) [40], DeepSeek가 제안한 GRPO [41]로 이어지고, 강화학습 결과로 자연 창발한 "Aha Moment"(Self-revision) [73]과 장고(더 많은 토큰 길이로 사고할수록 더 좋은 답에 도달)의 효과가 보고됩니다.

제8회 학습 데이터와 평가 벤치마크 정비

LLM 개발의 전체 파이프라인을 이해·구현할 수 있게 되는 것을 목적으로, 학습 데이터와 평가 벤치마크에 대해 상세히 해설합니다. 데이터의 전처리(필터링 등), LLM을 이용한 데이터 합성, LLM-as-Judge, 평가 벤치마크의 진전 등을 다룹니다.

일본의 LLM 생태계

2018년 OpenAI의 GPT-1 등장 이후, LLM의 파라미터 크기는 스케일 법칙에 따라 비약적으로 증대했습니다 [3]. 일본에서는 2023년부터 개발 경쟁이 본격 가속했습니다(2023년 이전에도 rinna, ABEJA, RICOH 등이 개발하고 있었음). 2023.3 OpenAI가 GPT-4를 공개한 뒤, 같은 해 5월 사이버에이전트 OpenCALM(7B), rinna의 일본어 특화 GPT(3.6B); 7월 NEC의 일본어 LLM(13B, 비공개); 8월 Stability AI Japanese StableLM Alpha(7B), LINE 일본어 LLM(3.6B), 도쿄대학교 마츠오 연구실 Weblab-10B(10B), ELYZA-japanese-Llama(7B)가 잇따랐습니다 [42]~[48]. 2023년 하반기~2024년에는 PFN PLaMo-13B, rinna Youri(7B), NTT tsuzumi(7B), 도쿄공대 Swallow(70B), ELYZA-japanese-Llama-2(70B), 라쿠텐 Rakuten AI(7B), NEC cotomi Pro/Light, LLM-jp-13B, Fujitsu Fugaku-LLM(13B), Stockmark-LLM-100b(100B), SB Intuitions Sarashina1-65B, PFN PLaMo-100B, CyberAgent CALM3(22B), 도쿄공대 Llama-3-Swallow(70B), Sarashina2-70B, 마츠오·이와사와 연구실 Geniac 기획 tanuki-8x8b(47B) 등 수십억~수백억 파라미터 모델이 쏟아졌습니다 [49]~[63].

2024년 말~2025년에는 더 큰 규모와 더 작은 규모가 공존합니다. PLaMo-10x100B(1T), llm-jp-3-172b(172B), Sarashina2-8x70B(465B) 같은 초대형 모델이 나오는가 하면 [64], CA DeepSeek-R1-Distill-Qwen-32B-Japanese(32B), Stockmark-2-100B-Instruct-beta(100B), Llama 3.3 Swallow 70B, ELYZA-Thinking-1.0-Qwen-32B, ABEJA-Qwen2.5-32b-Japanese-v1.0(이상 32B) 등 특화·추론 지향 모델도 등장했습니다. 최근에는 수B 정도의 경량 언어 모델(Small Language Model: SLM)을 개발하는 조직도 증가 추세입니다 — Rakuten AI 2.0 mini(1.5B), PLaMo 2 8B·PLaMo 2.1 2B(2B), Sarashina2.2(0.5~3B), Llama 3.1 Swallow 8B(8B) 등. 배경은 두 가지입니다. (1) 작은 모델이라도 더 많은 데이터로 더 오래 학습하면 높은 성능을 달성할 수 있다는 점(스케일 법칙적으로 학습 효율은 나쁘더라도), (2) 학습 완료 후의 추론 비용(운용 비용)까지 고려하면 작은 모델의 비용 대비 효과가 높다는 점입니다.

"Go smol or go home" 논의에서 지적되는 **Chinchilla Trap**이란, Chinchilla 수준(70B) 모델은 크기 때문에 추론 비용이 높으므로 추론 비용까지 고려해 더 작은 모델을 장시간 학습해야 한다는 지적입니다. 최적 모델 크기의 40~60% 이내 모델을 선택해 10~42% 계산량 추가로 동일 성능 모델을 학습할 수 있다는 분석도 있습니다.

일본 발 모델은 대략 두 갈래로 분류됩니다. 첫째, **사전학습부터 시작하는 풀스케치 개발**입니다. 학습을 완전히 제어할 수 있고 라이선스도 독자적으로 결정할 수 있지만 학습 비용이 높고 기술적 난이도도 높습니다. 대표 모델로 CALM3-22B, Weblab-10B, PLaMo-100B, LLM-jp-13B, Sarashina2-70B, tanuki-8x8b 등이 있습니다. 둘째, **사전학습 완료 영어 모델을 일본어로 지속 사전학습하여 개발**하는 방식입니다. 학습 비용이 낮고 언어 간 지식 전이에 의한 효율적 학습이 기대되지만, 학습 방식에 제한이 받

생하고 라이선스 제약이 있을 수 있습니다. 대표 모델로 ELYZA-japanese-Llama-70B, Swallow-70B, Llama-3-Swallow-70B 등이 있으며, 이용하는 사전학습 완료 모델은 성능이 높은 모델이 선택되는 경향으로 Llama 기반이 많습니다.

마츠오·이와사와 연구실은 경산성·NEDO의 'GENIAC' 프로젝트에서 Tanuki 모델을 개발했습니다 [63]. "해외 모델은 어느 쪽인가 하면 무기질적이고 형식적인 답변을 하는 경향이 있습니다만, 그와 대조적으로 당해 모델은 공감성이나 배려가 있는 답변이나, 자연스러운 말투로의 작문이 뛰어났습니다"라는 평가를 받습니다. 「Tanuki-8×8B」의 경량판인 「Tanuki-8B」를 채팅 형식으로 이용할 수 있는 데모도 공개되어 있습니다 [65].

연산 환경은 여전히 엄격합니다. 학습에 필요한 GPU 수는 GPT-3 상당이 A100 1,200대×30일, GPT-4 상당(*누출 정보)이 A100 25,000대×100일에 달합니다. 일본 국내 대표 GPU 클러스터는 산업기술종합연구소 ABCI의 960대 A100 → 6,128대 H200(2025년 1월 업그레이드), SoftBank 약 6,000대, 사쿠라 인터넷 2,000대 H100 수준이고 [66], 해외(단일 기업이 수십만~백만 기의 H100 보유, 24년 단년도 구매 수: Google 169,000대, Amazon 196,000대, Meta 224,000대, Microsoft 485,000대)와 큰 격차가 있습니다. 다만 GPU는 고속으로 세대 교체를 거듭해(P100 → V100 → A100 → H100 → B100) 후속 세대일 수록 계산 속도가 빠르고 비용도 낮아, 일본에는 후발의 이점이 있을지도 모릅니다 [67].

학습 데이터, 특히 사전학습용 일본어 데이터는 범용성과 고성능의 원천입니다. 인터넷에서 수집한 대량의 텍스트를 사용하지만 [68], 그 대부분은 영어 등 주요 언어로 구성되어 있어, 일본어 등의 텍스트를 대량 수집하는 것은 현재 한계가 있습니다. 데이터 원천은 어느 쪽이든 "Common Crawl"(<https://commoncrawl.org/>, 인터넷 사이트 크롤링 아카이브)이며, Wikipedia(ja) 덤프도 자주 사용됩니다 [69]. 개산으로 상기 합계 약 1.3TB, 1토큰 2문자 ≒ 4바이트로 하면 약 0.3T 토큰으로, Llama 2의 2T 토큰, GPT-4의 13T 토큰(누출 정보)과 비교하면 상당한 격차가 있습니다. 이를 보완하기 위해 LLM-jp 코퍼스는 일본어 6,880억 토큰을 확보했습니다 — 구성 데이터는 청공문고 텍스트, Common Crawl에서 추출·필터링한 일본어 코퍼스, e-Gov 법령, FineWeb 2 일본어 부분, 과학연구비 조성사업 데이터베이스 개요, 국회 회의록, 특허청 공개 데이터의 일본어 특허, 국립국회도서관 WARP URL에서 크롤한 텍스트, 일본어 Wikipedia 등입니다 [70].

학습 데이터 수집 시에는 세 가지에 주의해야 합니다. **저작권**은 저작권법(30조의4 제2호에서 AI 학습 데이터를 규정)에 의해 위반 시 저작권 침해(형사벌)에 해당하며, 일본은 구미에 비해 모델 학습에 이용 가능한 데이터의 자유도가 높다고 알려져 있습니다. **라이선스/이용 약관**은 작성자와 이용자 간의 계약으로 위반 시 손해 배상 문제가 발생할 수 있습니다. **개인정보**는 개인정보보호위원회가 생성AI 서비스 이용에 관한 주의 환기 등을 안내하며 [71], 상세는 법률 사무소에 상담할 것을 권합니다. 보충으로, 웹 크롤 시에는 robots.txt(RFC 9309)의 내용에 따를 필요가 있으며 따르지 않을 경우 저작권 침해에 해당할 가능성이 있습니다(The New York Times가 자사 기사 웹사이트의 robots.txt에서 AI 학습용 크롤러를 차단하고 별도 라이선스·API를 판매하는 사례가 있습니다). 이용 약관 예시로는 ChatGPT(OpenAI)의 약관이 자주 인용됩니다 [72].

라이선스 종류도 이해가 필요합니다. OSS, CC 라이선스는 비교적 자유도가 높은 라이선스이지만 다양한 종류가 있으므로 각각 이해가 필요합니다.

Creative Commons (CC) 라이선스:

라이선스	기호	명칭	상업 이용	개변	개변물 조건	크레딧 표기	비고
PD (CC0)	-	퍼블릭 도메인 선언	○	○	제한 없음	불필요	모든 권리 포기
CC BY	BY	저작자표시	○	○	제한 없음	필요	이용원 명시 필요
CC BY-SA	BY-SA	저작자표시-동일조건	○	○	같은 라이선스로 공개	필요	Wikipedia 채택
CC BY-ND	BY-ND	저작자표시-변경금지	○	×	개변 불가	필요	번역도 불가
CC BY-NC	BY-NC	저작자표시-비영리	×	○	제한 없음	필요	상업 이용 불가
CC BY-NC-SA	BY-NC-SA	저작자표시-비영리-동일조건	×	○	같은 라이선스로 공개	필요	상업 이용 불가
CC BY-NC-ND	BY-NC-ND	저작자표시-비영리-변경금지	×	×	개변 불가	필요	권리 지키면 자유로운 재배포 가능
C	©	All rights reserved	×	×	개변 불가	필요	권리자 사후 70년까지 보호

OSS(Open Source Software) 라이선스:

라이선스	특징	재배포 시 의무
MIT License	저작권 표시와 라이선스 문구를 남기면 사용 가능	저작권 표시와 라이선스 문구 기재
Apache License 2.0	MIT보다 약간 엄격. 특허 권리도 커버	저작권 표시, 라이선스 문구, 변경점 명시
GPL (GNU General Public License)	강한 카피레프트. 개변·재배포하면 같은 GPL 라이선스로 공개 필수	소스 공개 + GPL 계승
LGPL (Lesser GPL)	라이브러리로서의 이용은 가능, 본체에는 강제하지 않음	개변 시에만 소스 공개 의무

라이선스	특징	재배포 시 의무
BSD License	MIT과 거의 동일. 상업 이용 가능. 선전 금지 조항이 있는 경우도 있음	저작권 표시와 면책 사항 기재

"

Meta Llama 3 License: Meta가 독자적으로 정하는 라이선스입니다. 월간 액티브 사용자 7억 명 이상의 기업에는 별도 라이선스 계약이 필요하므로, OSS 라이선스는 아닙니다.

맺음말

대규모 언어 모델(LLM)의 개요를 세 축에서 소개했습니다.

첫째, LLM **개황**. 언어 모델이란 단어열의 생성 확률을 모델화한 것이며, "왜 지금 언어 모델인가"에 대한 답은 스케일과 범용성(Agent 등), 그리고 다른 영역으로의 영향(멀티모달·로봇)에서 찾을 수 있습니다.

둘째, LLM **강좌 각 회차 개요**. 기초편에서 LLM 개발의 기본 파이프라인을 이해하고 구현하며, 응용편에서 LLM 사회 구현까지를 고려한 기술을 이해하고 구현합니다.

셋째, 일본의 LLM을 둘러싼 **환경**. 2023년 이후 본격적으로 개발 경쟁이 가속했고, 데이터·모델·연산 환경을 스케일할 수 있는가가 관건입니다.

Reference

- [1] Ashish Vaswani, et al. (2017), "Attention Is All You Need", NeurIPS 2017, <https://arxiv.org/abs/1706.03762>
- [2] Alec Radford, et al. (2018), "Improving Language Understanding by Generative Pre-training", OpenAI Technical Report, <https://openai.com/research/language-unsupervised>
- [3] Wayne Xin Zhao, et al. (2023), "A Survey of Large Language Models", arXiv:2303.18223, <https://arxiv.org/abs/2303.18223>
- [4] OpenAI (2023), "GPT-4 Technical Report", arXiv:2303.08774, <https://arxiv.org/abs/2303.08774>

-
- [5] Jungo Kasai, et al. (2023), "Evaluating GPT-4 and ChatGPT on Japanese medical licensing examinations", arXiv:2303.18027, <https://arxiv.org/abs/2303.18027>
- [6] Momentum Works (2023), "The future by ChatGPT", <https://momentum.asia/product/the-future-by-chatgpt/>
- [7] Jared Kaplan, et al. (2020), "Scaling Laws for Neural Language Models", arXiv:2001.08361, <https://arxiv.org/abs/2001.08361>
- [8] Jason Wei, et al. (2022), "Emergent Abilities of Large Language Models", arXiv:2206.07682, <https://arxiv.org/abs/2206.07682>
- [9] Tom Brown, et al. (2020), "Language Models are Few-Shot Learners", NeurIPS 2020, <https://arxiv.org/abs/2005.14165>
- [10] Dan Swinhoe (2024), "Microsoft bought twice as many Nvidia Hopper GPUs as other big tech companies – report", DataCenterDynamics, <https://www.datacenterdynamics.com/en/news/microsoft-bought-twice-as-many-nvidia-hopper-gpus-as-other-big-tech-companies-report/>
- [11] Rishi Bommasani, et al. (2021), "On the Opportunities and Risks of Foundation Models", arXiv:2108.07258, <https://arxiv.org/abs/2108.07258>
- [12] Pengfei Liu, et al. (2021), "Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing", arXiv:2107.13586, <https://arxiv.org/abs/2107.13586>
- [13] Figure AI Inc. (2024), "Figure Official Website", <https://www.figure.ai/>
- [14] Michael Ahn, et al. (2022), "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances", arXiv:2204.01691, <https://arxiv.org/abs/2204.01691>
- [15] Anthony Brohan, et al. (2022), "RT-1: Robotics Transformer for Real-World Control at Scale", arXiv:2212.06817, <https://arxiv.org/abs/2212.06817>
- [16] Abby O'Neill, et al. (2023), "Open X-Embodiment: Robotic learning datasets and rt-x models", arXiv:2310.08864, <https://arxiv.org/abs/2310.08864>
- [17] Physical Intelligence (2024), " $\pi 0$: A Generalist Model for Physical Intelligence", <https://www.physicalintelligence.company/blog/pi0>
- [18] OpenAI (2024), "Sora: Creating video from text", <https://openai.com/sora>
- [19] OpenAI (2024), "Video generation models as world simulators", <https://openai.com/research/video-generation-models-as-world-simulators>

-
- [20] Takeshi Kojima, et al. (2025), "A Comprehensive Survey on Physical Risk Control in the Era of Foundation Model-enabled Robotics", arXiv:2505.12583, <https://arxiv.org/abs/2505.12583>
- [21] Takeshi Kojima (2022), "zero_shot_cot", GitHub Repository, https://github.com/kojima-takeshi188/zero_shot_cot
- [22] 도쿄대학교 마츠오·이와사와 연구실(2026), "대규모 언어 모델 사회구현 강좌", <https://weblab.t.u-tokyo.ac.jp/education/large-language-model/>
- [23] Sander Schulhoff, et al. (2024), "The Prompt Report: A Systematic Survey of Prompt Engineering Techniques", arXiv:2406.06608, <https://arxiv.org/abs/2406.06608>
- [24] Jason Wei, et al. (2022), "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models", arXiv:2201.11903, <https://arxiv.org/abs/2201.11903>
- [25] Takeshi Kojima, et al. (2022), "Large Language Models are Zero-Shot Reasoners", arXiv:2205.11916, <https://arxiv.org/abs/2205.11916>
- [26] Shane Gu (2025), "X Post (status/1972309771610100179)", <https://x.com/shaneguML/status/1972309771610100179>
- [27] Xuezhi Wang, et al. (2023), "Self-Consistency Improves Chain of Thought Reasoning in Language Models", arXiv:2203.11171, <https://arxiv.org/abs/2203.11171>
- [28] Sweta Agrawal, et al. (2025), "GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning", arXiv:2502.14856, <https://arxiv.org/abs/2502.14856>
- [29] Raimi Karim (2019), "Illustrated: Self-Attention", Towards Data Science, <https://towardsdatascience.com/illustrated-self-attention-2d627e33b20a>
- [30] Jay Alammar (2018), "The Illustrated Transformer", <http://jalammar.github.io/illustrated-transformer/>
- [31] Tom Henighan, Jared Kaplan, et al. (2020), "Scaling Laws for Autoregressive Generative Modeling", arXiv:2010.14701, <https://arxiv.org/abs/2010.14701>
- [32] Daisuke Okanohara, "X Post (status/1972421341988225340)", <https://x.com/hillbig/status/1972421341988225340>
- [33] Rewon Child, et al. (2019), "Generating Long Sequences with Sparse Transformers", arXiv:1904.10509, <https://arxiv.org/abs/1904.10509>

-
- [34] sunbluesome (2022), "Sparse Transformer를 이해하고 싶다", Zenn, <https://zenn.dev/sunbluesome/articles/5f6a86dfa1e1be>
- [35] William Fedus et al. (2022), "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity", JMLR 23, <https://jmlr.org/papers/v23/21-0998.html>
- [36] Tim Dettmers, et al. (2022), "A Gentle Summary of LLM.int8(): Zero Degradation Matrix Multiplication for Large Language Models", Hugging Face Blog, <https://huggingface.co/blog/hf-bitsandbytes-integration>
- [37] Microsoft Deep Speed Team (2023), "DeepSpeed: 딥러닝 학습과 추론을 획기적으로 고속화하는 프레임워크", https://www.deepspeed.ai/assets/files/DeepSpeed_Overview_Japanese_2023Jun7th.pdf
- [38] Chung, Hyung Won, et al. (2022), "Scaling Instruction-Finetuned Language Models", arXiv: 2210.11416, <https://arxiv.org/abs/2210.11416>
- [39] Edward J. Hu et al. (2021), "LoRA: Low-Rank Adaptation of Large Language Models", arXiv: 2106.09685, <https://arxiv.org/abs/2106.09685>
- [40] Rafailov, Rafael, et al. (2023), "Direct Preference Optimization: Your Language Model is Secretly a Reward Model", NeurIPS 2023, <https://arxiv.org/abs/2305.18290>
- [41] Shao, Zhihong, et al. (2024), "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models", arXiv:2402.03300, <https://arxiv.org/abs/2402.03300>
- [42] 주식회사 사이버에이전트(2023), "사이버에이전트, 최대 68억 파라미터의 일본어 LLM(대규모 언어 모델)을 일반 공개", <https://www.cyberagent.co.jp/news/detail/id=28817>
- [43] rinna 주식회사(2023), "rinna, 일본어에 특화한 36억 파라미터의 GPT 언어 모델을 공개", <https://rinna.co.jp/news/2023/05/20230507.html>
- [44] Stability AI Japan (2023), "일본어 언어 모델 「Japanese StableLM Alpha」를 릴리스했습니다", <https://ja.stability.ai/blog/japanese-stablelm-alpha>
- [45] Ledge.ai 편집부(2023), "LINE 36억 파라미터의 일본어 LLM을 공개, 상업 이용도 가능", https://ledge.ai/articles/line_japanese_large_lm
- [46] 일본전기주식회사(2023), "NEC, 130억 파라미터로 세계 톱클래스의 일본어 성능을 보유한 경량 LLM을 개발", https://jpn.nec.com/press/202307/20230706_02.html
- [47] OpenAI (2023), "GPT-4", <https://openai.com/research/gpt-4>

-
- [48] ELYZA (2023), "70억 파라미터의 상업 이용 가능 일본어 LLM 「ELYZA-japanese-Llama-2-7b」를 일반 공개했습니다", <https://elyza.ai/news/2023/08/29/>
- [49] 일본전신전화주식회사(2023), "NTT판 LLM 「tsuzumi」", https://www.rd.ntt/research/LLM_tsuzumi.html
- [50] 주식회사 ELYZA (2024), "700억 파라미터의 일본어 LLM 「ELYZA-japanese-Llama-2-70b」를 개발, 데모 공개", <https://note.com/elyza/n/n0ea755ca3e7b>
- [51] 라쿠텐 그룹 주식회사(2024), "라쿠텐, 일본어에 최적화한 오픈형 고성능 LLM을 공개", https://corp.rakuten.co.jp/news/press/2024/0321_01.html
- [52] rinna 주식회사(2023), "rinna, Llama 2의 일본어 지속 사전학습 모델 「Youri 7B」 시리즈를 공개", <https://rinna.co.jp/news/2023/10/20231031.html>
- [53] 주식회사 사이버 에이전트(2024), "독자 일본어 LLM 버전 3를 일반 공개 - 225억 파라미터 상업 이용 가능 모델 제공", <https://www.cyberagent.co.jp/news/detail/id=30463>
- [54] Swallow-LLM Project (2024), "Llama 3 Swallow", <https://swallow-llm.github.io/llama3-swallow.ja.html>
- [55] LLM-jp (2024), "대규모 언어 모델 「LLM-jp-13B v2.0」을 구축·공개", <https://llm-jp.nii.ac.jp/blog/2024/04/30/v2.0-release.html>
- [56] 주식회사 Preferred Networks (2024), "GENIAC 제1사이클 개발 성과로 대규모 언어 모델 PLaMo-100B-Pretrained를 공개", <https://www.preferred.jp/ja/news/pr20241015>
- [57] 주식회사 Preferred Networks (2023), "PLaMo-13B를 공개했습니다", <https://tech.preferred.jp/ja/blog/llm-plamo/>
- [58] 일본전기주식회사(2024), "NEC, 세계 톱레벨 성능의 고속 대규모 언어 모델(LLM) cotomi Pro / cotomi Light를 개발", https://jpn.nec.com/press/202404/20240424_01.html
- [59] 스톡마크 주식회사(2024), "할루시네이션을 대폭 억제하고 전문적 질문에도 정확한 답변이 가능한 생성AI - 스톡마크, 독자 130억 파라미터 LLM을 개발해 상업 이용 가능 오픈소스로 공개", <https://stockmark.co.jp/news/20240516>
- [60] SB Intuitions 주식회사(2024), "SB Intuitions, 독자 일본어 LLM을 구축 - 70억·130억·650억 파라미터 일본어 LLM을 공개", https://www.sbintuitions.co.jp/news/press/20240614_01/
- [61] 후지쯔 주식회사(2024), "슈퍼컴퓨터 「도쿄(富岳)」로 학습한 대규모 언어 모델 「Fugaku-LLM」을 공개", <https://pr.fujitsu.com/jp/news/2024/05/10.html>

-
- [62] 도쿄공업대학(2023), "일본어에 강한 대규모 언어 모델 「Swallow」 를 공개", <https://www.titech.ac.jp/news/2023/068089>
- [63] 도쿄대학교 마츠오·이와사와 연구실(2024), "마츠오·이와사와 연구실, 경산성·NEDO의 「GENIAC」 프로젝트에서 국내 최초 대규모 언어 모델(LLM) 멀티모달화 등 개발 성과 공개", <https://weblab.t.u-tokyo.ac.jp/2024-08-30/>
- [64] llm-jp (2024), "Awesome Japanese LLM", GitHub Pages, <https://llm-jp.github.io/awesome-japanese-llm/>
- [65] weblab-GENIAC (2024), "Tanuki-8B-dpo-v1.0", Hugging Face Spaces, <https://huggingface.co/spaces/weblab-GENIAC/Tanuki-8B-dpo-v1.0>
- [66] Dan Swincoe (2024), "Microsoft bought twice as many Nvidia Hopper GPUs as other big tech companies – report", DataCenterDynamics, <https://www.datacenterdynamics.com/en/news/microsoft-bought-twice-as-many-nvidia-hopper-gpus-as-other-big-tech-companies-report/>
- [67] Timothy Prickett Morgan (2024), "Nvidia Unfolds GPU, Interconnect Roadmaps Out To 2027", The Next Platform, <https://www.nextplatform.com/2024/06/02/nvidia-unfolds-gpu-interconnect-roadmaps-out-to-2027/>
- [68] Linting Xue, et al. (2021), "mT5: A massively multilingual pre-trained text-to-text transformer", NAACL-HLT 2021, <https://aclanthology.org/2021.naacl-main.41/>
- [69] 사쿠라이 아키오(2022), "세계에서 개발이 진행되는 대규모 언어 모델이란(후편)", NTT 데이터 첨단 기술 주식회사 칼럼, <https://www.intellilink.co.jp/column/ai/2022/072800.aspx>
- [70] LLM-jp (2024), "llm-jp-corpus-v4", GitLab Datasets, <https://gitlab.llm-jp.nii.ac.jp/datasets/llm-jp-corpus-v4>
- [71] 개인정보보호위원회(2023), "생성AI 서비스 이용에 관한 주의 환기 등에 대해", https://www.ppc.go.jp/news/careful_information/230602_AI_utilize_alert/
- [72] OpenAI (2026), "Terms of Use", OpenAI Policies, <https://openai.com/ja-JP/policies/row-terms-of-use/>
- [73] DeepSeek-AI (2025), "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning", <https://arxiv.org/abs/2501.12948>

02

제2회: 추론 — 프롬프팅과 문맥 내 학습

이 장의 구성

- 01 도입: LLM 시대의 활용과 본 강의의 목표
 - 02 디코딩의 기초: 언어 모델에서 출력을 얻는 방법
 - 03 프롬프팅의 기초: 가중치 재학습 없이 과제를 푼다
 - 04 메타 생성: 여러 번 추론해 더 나은 출력을 얻는다
 - 05 발전적인 프롬프트: 실제 시스템과 새로운 활용
 - 06 LLM을 활용한 서비스와 모델의 선택
 - 07 더 읽을거리
 - 08 Reference
-

//

본 한국어 역문은 교육 목적의 번역이며, 원본 라이선스(CC BY-NC-ND 4.0)를 따릅니다. 원자료는 도쿄대학교 마츠오·이와사와 연구실이 작성한 「대규모 언어 모델 강좌 2025」 기초편(2025년 10-11월 개최) 제2회 강의 자료로, CC BY-NC-SA 4.0으로 등록되어 있습니다. 비영리 목적의 2차 이용이 허락되며, 영리 목적은 별도 문의가 필요합니다. 재이용 시 본 라이선스 표기를 기재해 주세요. 원문의 의미를 변경하거나 추가하지 않았습니다.

도입: LLM 시대의 활용과 본 강의의 목표

강사 원다 켄노우(原田憲旺)는 마츠오·이와사와 연구실 박사과정 3년 차로, LLM 강좌 자료 제작·콘텐츠 트 제작, GENIAC 평가, AI 백서 2025 생성AI 에디션 집필 협력, DeepLearning.ai 생성AI 강의 번역, 기시다·이시바 총리 생성AI 강의 TA·강사를 맡았다. 연구 테마는 LLM의 평가와 모델에 의한 평가, 지시 추종 능력, Web Agent를 활용한 UI/UX 평가, 교육 현장에서의 LLM 응용이다.

ChatGPT는 주당 7억 명이 이용하고 180억 메시지가 오가며, 업무 용도가 27%, 이용자의 약 절반이 26세 이하, 남녀 비율은 1:1이다. 전체 이용 용도의 80%는 실용적 안내(Practical Guidance), 정보 탐구(Seeking Information), 작문(Writing)에 집중된다[1]. 강사 본인도 1시간짜리 팟캐스트를 Gemini에서 부분 질의하며 번역·요약하고, 수십 건의 논문은 Deep Research로 조사하며, 같은 질문을 여러 서비스에 던져 교차 검증한다. 슬라이드만 공개된 해외 강의 자료도 추가 검색과 문맥 보충으로 따라갈 수 있고, 개조식 노트를 글로 풀거나 코딩·데이터 정형에도 쓴다.

이처럼 LLM을 활용한다는 것은 모델에 대한 지시(프롬프트)와 응답 선택(디코딩·메타 생성)을 공략하는 일이다. 본 강의는 디코딩 기초, 프롬프팅 기초, 메타 생성, 발전 프롬프트 예, 서비스 사례, 모델 선택을 다룬다.

디코딩의 기초: 언어 모델에서 출력을 얻는 방법

언어 모델이란 단어 열(문장)이 얼마나 발생하기 쉬운지를 모델화한 확률 모델로, 단어 열 $(x_1, x_2, \dots, x_L)$ 에 생성 확률 $p(x_1, x_2, \dots, x_L)$ 을 할당한다. "좋은" 언어 모델은 문법적·상식적 오류가 있는 문장에는 낮은 확률을 준다. 예컨대 $p(\text{일본, 의, 수도, 는, 도쿄}) = 0.02$, $p(\text{일본, 의, 수도, 는, 파리}) = 0.00001$, $p(\text{도쿄, 의, 수도, 는, 일본}) = 0.0005$ 와 같다.

이 결합 확률은 확률의 연쇄법칙으로 조건부 분포의 곱으로 분해할 수 있으며, 이렇게 분해한 모델을 자기 회귀 언어 모델(autoregressive language model)이라 한다.

$$p(x_1, x_2, \text{Wldots}, x_L) = p(x_1) \cdot p(x_2 \mid x_1) \cdot \text{Wldots} \cdot p(x_L \mid x_1, x_2, \text{Wldots}, x_{L-1})$$

예로 $p(\text{Wtext{\{일본, 의, 수도\}}}) = p(\text{Wtext{\{일본\}}}) \cdot p(\text{Wtext{\{의\}} \mid \text{Wtext{\{일본\}}}) \cdot p(\text{Wtext{\{수도\}} \mid \text{Wtext{\{일본, 의\}}})$ 이다. 조건부 확률을 알면 생성도 가능하다. $p(\text{Wtext{\{도쿄\}} \mid \text{Wtext{\{일본, 의, 수도, 는\}}}) = 0.2$, $p(\text{Wtext{\{파리\}} \mid \text{Wldots}}) = 0.001$, $p(\text{Wtext{\{카이로\}} \mid \text{Wldots}}) = 0.0005$ 이므로 "일본의 수도는" 다음에는 "도쿄"가 생성된다. 이 조건부 확률은 번역(영어 문장 → 일본어 문장), 질의응답(질문 → 답변), 요약(문서 → 짧은 서술) 등 다양한 과제로 일반화되며, 수식으로는 $p(x_{i+1:L} \mid x_1, \text{Wldots}, x_i) = \prod_{j=i+1}^L p(x_j \mid x_{1:i}, x_{i+1:j-1})$ 로 쓴다.

모델에서 어떻게 출력을 얻을 것인가가 디코딩(Decoding)의 문제이며, 이는 알고리즘과 스코어 함수의 선택이라는 관점으로도 정의된다[2]. 대표적 기법은 다섯 가지다. Greedy decoding은 매 스텝 가장 확률이 높은 토큰을 선택하지만, 문장 전체로는 최적일 수 있고 반복이 잦다[2]. Beam search는 빔 수(num_beams)만큼 후보를 남겨 여러 스텝 단위로 점수가 높은 것을 선택하나, 계산량이 많고 출력이 지루하며 짧아지는 경향이 있다[2]. Top-k sampling은 상위 k개에서 샘플링하지만 long-tail 문제가 있고 유망 선택지가 배제될 수 있다[2]. Top-p sampling(핵 샘플링, Holtzman et al., 2020)은 상위부터 누적해 누적 확률이 $p \times 100\%$ 가 되는 후보 안에서 샘플링하며, Top-k보다 유연하다[2].

샘플링의 무작위성은 temperature T 로 조절한다. softmax 식 $p(w) = \frac{\exp(z_w / T)}{\sum_{j=1}^{|V|} \exp(z_j / T)}$ 에서 T 를 0에 가깝게 하면 분포가 뽕족해져 거의 결정적이 되고, 크게 하면 무작위성이 높아진다[3]. 다만 $T=0$ 으로 설정해도 완전히 결정적이지 않은 경우가 있는데, 부동소수점 연산 순서와 배치 처리의 분할 방식 차이 때문이며, GPU 처리를 수정하면 결정적으로 만들 수 있다[4].

어떤 기법을 선택할까. 기준은 다양성이 필요한지이다. 이야기 생성·아이디어 발산에는 샘플링 기반 기법이, 지식 질의·번역에는 Greedy decoding이나 Beam search가 적합하다. 실제로는 Greedy 결과와 temperature·Top-p를 바꿔 여러 번 비교해 보는 것이 좋다(참고: A Thorough Examination of Decoding Methods in the Era of LLMs, The Curious Case of Neural Text Degeneration, It's MBR All the Way Down). "확률이 높은 것이 정말로 원하는 출력인가?"라는 물음은 뒤의 Reward model·LLM-as-a-Judge 기반 Best-of-N으로 이어진다.

정리하면, 조건부 확률로 다음 단어 후보가 결정되고 모델은 과거 문맥을 바탕으로 다음 단어의 그럴듯함(plausibility)을 출력한다(과거 문맥 반영과 학습 기법은 제3회 강의). 후보에서 어떻게 선택할지가 디코딩 기법의 문제이며 과제마다 적합한 기법이 다르다. 그리고 모델에 대한 입력, 즉 과거 문맥 자체를 공략해 과제를 풀게 하는 것이 다음 절의 프롬프팅이다.

프롬프팅의 기초: 가중치 재학습 없이 과제를 푼다

"프롬프트"는 원래 인간 입력을 촉구하는 커맨드 표시(`C:\>`, `~$`)를 뜻했지만, 근년에는 AI의 출력을 촉구하는 문자열 — "질문: 일본의 수도는?", "답변:" — 을 의미한다.

전환점은 GPT-3였다. Brown et al. (2020)[5]는 "언어 모델의 규모를 키우면 과제에 무관한(task-agnostic) few-shot 성능이 크게 향상되며 종래의 최고 수준 파인튜닝과 맞먹기도 한다. 1,750억 매개변수의 자기회귀 언어 모델 GPT-3를 학습시키고, 모든 과제에서 기율기 갱신이나 파인튜닝 없이 과제와 few-shot 시연을 순전히 텍스트 상호작용으로만 지정해 적용했다"고 보고했다. 이전까지는 과제마다 전용 모델을 가중치 갱신하며 학습했지만[6], GPT-3는 가중치 재학습 없이 프롬프트만 바꿔 다양한 과제에서 고성능을 달성했다. 전조는 GPT-2(Radford et al., 2019)[7]에서 이미 보였다. "A: "를 붙이면 답하고, "TL;DR:"(Too Long; Didn't Read)을 붙이면 요약하며, "english sentence = french sentence, english sentence =" 형태로 번역했다.

이처럼 가중치를 고정한 언어 모델이 프롬프트에 의한 조건 부여로 과제를 수행하는 것을 **문맥 내 학습** (In-context Learning, ICL)이라 하며, 하나의 모델이 재학습 없이 다양한 과제를 고성능으로 수행한다는 사실이 당시의 충격이었다[5].

프롬프팅이란 특정 기능의 발생을 촉구하도록 언어 모델에 입력하는 컨텍스트 문이다. 과제 설명·지시만 주는 **Zero-shot**, 시연(demonstration) 예 몇 개 주는 **Few-shot**(예 하나면 one-shot)으로 나뉘며, LLM 이전의 few-shot learning과는 의미가 다르다[5]. 시연 예시 수를 늘리면, 특히 대규모 모델에서, 성능이 크게 오른다[5]. 최근에는 100만 토큰을 받아들이는 long context 모델로 수많은 예시를 주는 many-shot in-context learning[8]까지 등장했다(GPT-3는 2048 토큰). 컨텍스트가 늘면 계산량이 증가하므로 더 향상을 노리려면 파인튜닝으로 효율화해야 한다(제6회 강의).

다단계 추론 과제에서는 **Chain-of-Thought(CoT)**가 결정적이다. Wei et al. (2022)[9]는 답에 이르는 사고 과정을 예시로 주는 Few-shot CoT를 제안했고, 다양한 수학 데이터셋에서 특히 모델이 클 때 성능 개선이 큼을 보였다. 코지마 등[10]은 CoT 예 없이 "Let's think step by step" 한마디만 붙여 모델 스스로 생각하게 하는 **Zero-shot CoT**를 시도했다. "인간이 파이프라인을 설계하지 않고 모델 스스로 생각시키는 편이 낫지 않을까?"에서 출발한 구절이다. 다만 단일 스텝 상식 추론에서는 오히려 생각이 지나쳐 실패하기도 하며, Zero-shot CoT가 빛을 발하는 것은 본질적으로 다단계 추론 과제에서다[10]. 이론적 근거로 Li et al. (2024)[11]는 CoT·중간 토큰(intermediate token)이 표현력을 향상해 순차적 처리 과제의 성능을 끌어올림을 보였다.

같은 과제라도 프롬프트에 따라 정확도가 크게 달라진다[12][13]. 원하는 출력을 얻도록 프롬프트를 시행착오하는 **프롬프트 엔지니어링**에는 수동과 자동 두 방향이 있다. 수동으로는 few-shot prompting과 CoT prompting이 대표적이며 개발사 가이드라인을 참고할 수 있다. 자동으로는 특수 토큰을 학습하는 prefix tuning·prompt tuning(제6회)과 프롬프트 문 자체를 수정하는 접근이 있다. 후자의 대표적 예로,

Automatic Prompt Engineer(Zhou et al., 2023)[14]는 입출력 쌍으로 지시문을 LLM 스스로 예측·평가·변형하고, Demonstrate-Search-Predict(Khattab et al., 2022)[15]는 질문 분해와 후속 질문 같은 중간 과정을 시행착오로 생성해 결과 일치로 판정하며, OPRO(Yang et al., 2024)[16]는 과거 프롬프트와 점수 변천을 주며 점수를 높이는 프롬프트를 생성하고, 최근의 GEPA(Agrawal et al., 2025)[17]는 성공·실패 궤적에서 언어 피드백을 작성해 개선하거나 유망 후보와 결합하며 다양성 확보를 돕는다.

정리하면, 프롬프팅은 입력문을 공략해 가중치 재학습 없이 다양한 과제에서 고성능을 달성하는 기술이며, few-shot prompting과 chain-of-thought prompting이 유효하지만 표현·포맷 차이로 성능이 크게 달라져 시행착오가 필요하다.

메타 생성: 여러 번 추론해 더 나은 출력을 얻는다

Welleck et al. (2024)[18]가 정리한 **메타 생성 알고리즘**은 모델을 여러 번 추론시킨 뒤 출력을 얻는 기법군으로, 추론 시점의 계산을 늘려 품질을 끌어올린다.

가장 단순한 형태는 Self-Consistency(Wang et al., 2023)[19]로, 샘플링으로 여러 답을 얻어 다수결로 채택한다. Best-of-N[20]은 여러 답에 점수를 매겨 가장 높은 것을 선택한다. 점수는 전용 분류기(Reward Model, Process Reward Model — 제7회 강화학습)를 학습하거나, LLM에게 평가시키는 LLM-as-a-Judge(Zheng et al., 2023)[21]로 매긴다. 긴 글을 높게 평가하려는 편향은 있으나 인간 평가와 어느 정도 일치한다. SELF-REFINE(Madaan et al., 2023)[22]은 답 생성 → 피드백 → 수정 루프를 같은 모델로 프롬프팅만 바꿔 반복한다. 한 번의 추론이 아니라 여러 번의 시도와 자기 반성으로 품질을 끌어올리는 것이 공통 패턴이다.

발전적인 프롬프트: 실제 시스템과 새로운 활용

실제 제품의 프롬프트는 단순한 지시문이 아니다. Claude에서 실제로 사용되는 시스템 프롬프트[23]는 2,500단어로, 속성 정보, 장르별 응답 태도, 포맷, knowledge cut-off, 미국 대통령 선거 결과 등을 포함해 모델 행동을 정밀 제어한다.

프롬프트는 공격 대상이 되기도 한다. **적대적 프롬프트**(adversarial prompt)의 대표적인 것이 탈옥(jailbreak)으로, 페르소나 부여로 본래 답하지 않을 것도 답하게 하며("Do Anything Now"), 공격성을 높이는 토큰도 존재한다[24]. 현실 사례로는 투명 색 글자로 숨은 지시를 심어 자료와 다른 보고서를 쓰는 공격[25], 논문 심사를 긍정 유도하는 비밀 명령문(일한미 등 주요 14개 대학 대책)[26]이 보고되었다.

반대로 프롬프팅만으로 복잡한 시스템을 구현하기도 한다. DeepResearch는 검색 쿼리 작성, 충분성 점검(회고), 답 생성을 프롬프팅으로 구성한다[27]. 논문에서 포스터를 자동 생성하는 Paper2Poster(Pang et al., 2025)[28]는 그림 잘라내기와 파워포인트 조작 라이브러리를 프롬프트와 조합한다(검색·코드 이용은 응용편 제2회).

프롬프팅은 학습 데이터를 만드는 데도 쓰인다. 합성 데이터(synthetic data)는 LLM 학습용 데이터를 인공적으로, 특히 LLM으로 만드는 것으로, 제어 실험용 TinyStories·Physics of LM, 복잡 데이터셋용 WizardLM·Alpaca[29], 대모델 능력을 소모델로 옮기는 s1K·NaturalThoughts, 고품질 사전학습용 Textbooks Are All You Need 등 다양하다. LLM에게 인격·특성을 부여해 인간을 모의하는 시뮬레이션 연구도 있다(Park et al., 2024, Generative Agent Simulations of 1,000 People[30]).

LLM을 활용한 서비스와 모델의 선택

코드도 언어라는 점에서 코딩은 LLM의 자연스러운 활용처다. GitHub Copilot, Claude Code, Cursor, Cline, Windsurf, Devin 등이 등장했고, Cursor는 2023년 릴리스 이후 2025년 6월 기준 500 million USD ARR에 900 million USD를 조달했다[31][32]. Y Combinator 투자처를 봐도 AI로 개발 속도를 높이고 새 가치를 창출하는 기업이 주류며, "10명 + AI = 10억 달러 기업"이 화제다[33][34].

이 핵심 역량을 Andrej Karpathy(2025)[35]는 "context engineering", 즉 "다음 단계·처리를 위해 컨텍스트 윈도우를 최적의 정보로 채우는 정밀한 예술이자 과학"이라 불렀다. 기본 프롬프팅(이번 회), RAG·tool-use(응용편 제2회), 상태 관리·멀티모달(응용편 제7회)이 모두 이 맥락이며, Gemini_Plays_Pokemon[36]이나 Gemini 2.5[37]의 에이전트 능력이 보여주듯 컨텍스트 설계는 곧 시스템 설계다. 강사는 DeepLearning.ai의 ChatGPT Prompt Engineering for Developers, Building Systems with the ChatGPT API, How Diffusion Models Work, LangChain 강의 두 종을 일본어로 번역했다(deeplearning.ai/courses[38]).

모델 접근은 세 가지다. API 전용은 가중치 비공개·사용량 과금으로, 자체 컴퓨터 없이 GPT·Gemini·Claude 등을 쓴다(GPT는 1M 토큰 입력당 \$1.25, 출력당 \$10). 공개 모델은 가중치까지 공개되어 분석에도 적합하고 로컬에서 실행 가능하며(Llama·Mistral·DeepSeek·Qwen·gpt-oss), 비공개 모델은 일부 연구기관만 이용 가능하다(PaLM·Gopher). 공개 모델을 다룰 때는 Transformers(HuggingFace의 모델·데이터셋 허브, 연습에서도 사용, 버그 주의)와 vLLM(고속 추론)이 핵심이다.

공개 모델의 계산 자원은 세 갈래다. 자체 GPU(H100 80GB 1장 약 600만엔 + 전력·유지보수; 양자화 적용 gpt-oss-120b 이용 가능), 클라우드 GPU(시간당 과금, H100 \$1.49/시간도 가능; AWS·GCP·Azure·Lambda·HPC-AI·Hyperbolic), 그리고 모델 호스팅 서비스(모델 선택만으로 입출력 과금; Cerebras[40]·Groq·Together.ai·Fireworks, 각 사 독자적 고속화 기술로 성능 차이[39])다.

성능은 Imarena(사용자 투표형)[41]와 HELM(복수 벤치마크 종합)[42]으로 가늠하며 일본어는 Nejumi LLM 리더보드 4[43]를 참고한다. 직접 평가 도구로는 simple-evals/evals(OpenAI), llm-jp-eval(LLM-jp), Lighteval(HuggingFace)가 있다. 단, 프롬프트나 선택지 줄이기 방식 차이로 같은 모델에서도 크게 다른 점수가 나올 수 있어 주의해야 한다[44][45].

선택의 출발점은 가장 성능이 좋은(비싼) 모델을 먼저 쓰는 것이다. 유료 앱이나 API playground로 시작하며[45][39], 여러 모델을 빠르게 비교할 때는 동일 인터페이스의 **OpenRouter**[46]가 유용하다("오루루키 사우나" 정답을 모든 모델이 오답으로 답한 사례도 있어 평가는 신중해야 한다).

더 읽을거리

- **디코딩/메타 생성**: Generating Text from Language Models(ACL 2023 튜토리얼), Stanford CS324, CMU Advanced NLP Inference I/Advanced Inference Strategies, From Decoding to Meta-Generation(arxiv 2406.16838), Beyond Decoding(NeurIPS 2024), HuggingFace Generation strategies, Brown et al. 2024 Large Language Monkeys, Wu et al. 2024 Inference Scaling Laws, Gu et al. 2024 A Survey on LLM-as-a-Judge, Kamoi et al. 2024 When Can LLMs Actually Correct Their Own Mistakes?, 2024 가을 LLM 강좌 특별회 "LLM의 자기 수정".
- **프롬프팅/문맥 내 학습**: Stanford CS224U·CS224N Lecture 11, CMU CS11-711, Lilian Weng "Prompt Engineering", Prompt Engineering Guide, In-context Learning and Induction Heads(transformer-circuits.pub), Dai et al. 2023, Min et al. 2022, Razeghi et al. 2022, Wei et al. 2023, 대규모 언어 모델 응용 제5회.
- **Chain-of-Thought**: Stanford CS25 LLM Reasoning, Wang et al. 2024, Yao et al. 2023(Tree of Thoughts).
- **발견 프롬프트**: Grok prompts(github.com/xai-org/grok-prompts), Zou et al. 2023, Anthropic "How we built our multi-agent research system", OpenAI Codex CLI 프롬프트.
- **GPT-2/3 뒷이야기**: Ilya Sutskever·Alec Radford 강연, An Observation on Generalization 영상.
- **평가/스케일링**: The Second Half, Jason Wei "Successful language model evals", How to Build Good Language Modeling Benchmarks, Why You Should Stop Using HotpotQA for AI Agents Evaluation in 2025, Singh et al. The Leaderboard Illusion, MIT TinyLab TinyML, The Ultra-Scale Playbook, How to Scale Your Model, Stanford CS336 Lec. 5~7, 도쿄대 "AI와 반도체" 강좌, Karpathy "Software Is Changing (Again)", Andrew Ng "AI is Accelerating Startups", "Vibe coding MenuGen".

Reference

[1] Chatterji et al., 2025, How People Use ChatGPT. [2] How to generate text: using different decoding methods for language generation with Transformers, <https://huggingface.co/blog/how-to-generate> [3] Cohere (2024), "Parameters for Controlling Outputs", Cohere LLMU, <https://cohere.com/llmu/parameters-for-controlling-outputs> [4] Thinking Machines (2024), "Defeating Nondeterminism in LLM Inference", <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/> [5] Brown et al., 2020, Language Models are Few-Shot Learners. [6] Liu et al., 2021, Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. [7] Radford et al., 2019, Language Models are Unsupervised Multitask Learners. [8] Agarwal et al., 2024, Many-Shot In-Context Learning. [9] Wei et al., 2022, Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. [10] Kojima et al., 2022, Large Language Models are Zero-Shot Reasoners. [11] Li et al., 2024, Chain of Thought Empowers Transformers to Solve Inherently Serial Problems. [12] Gonen et al., 2023, Demystifying Prompts in Language Models via Perplexity Estimation. [13] Sclar et al., 2024, Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting. [14] Zhou et al., 2023, Large Language Models Are Human-Level Prompt Engineers. [15] Khattab et al., 2022, Demonstrate-Search-Predict: Combining Retrieval and Language Models for Knowledge-Intensive NLP Tasks. [16] Yang et al., 2024, Large Language Models as Optimizers. [17] Agrawal et al., 2025, GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. [18] Welleck et al., 2024, From Decoding to Meta-Generation: Inference-time Algorithms for Large Language Models. [19] Wang et al., 2023, Self-Consistency Improves Chain of Thought Reasoning in Language Models. [20] Snell et al., 2024, Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. [21] Zheng et al., 2023, Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. [22] Madaan et al., 2023, Self-Refine: Iterative Refinement with Self-Feedback. [23] Anthropic Release notes, System prompts. [24] Adversarial Prompting in LLMs. [25] 시마다 타쿠(2025), "AI에 과제를 쓰게 하면 자료에 없는 내용을 출력 - 게이오대의 AI 대책이 화제, 의도를 들었다", ITmedia AI+, 2025/05/01, <https://www.itmedia.co.jp/aipplus/articles/2504/30/news214.html> [26] 니혼게이지신문(2025), "논문 내에 비밀 명령문, AI에게 '높게 평가하라' 일한미 등 주요 14개 대학에서", 2025/06/29, <https://www.nikkei.com/article/DGXZQOUC13BCW0T10C25A6000000/> [27] Gemini Fullstack LangGraph Quickstart. [28] Pang et al., 2025, Paper2Poster: Towards Multimodal Poster Automation from Scientific Papers. [29] Taori et al., 2023, Alpaca: A Strong, Replicable Instruction-Following Model. [30] Park et al., 2024, Generative Agent Simulations of 1,000 People. [31] Cursor at \$100M ARR, <https://sacra.com/research/cursor-at-100m-arr/> [32] AnySphere (2026), "Cursor - The AI-first Code Editor", <https://cursor.com/ja> [33] Startup Directory, <https://www.ycombinator.com/companies> [34] 10 People + AI

= Billion Dollar Company?, https://www.youtube.com/watch?v=CKvo_kQbakU [35] Andrej Karpathy (2025), "X Post (status/1937902205765607626)", <https://x.com/karpathy/status/1937902205765607626> [36] Gemini_Plays_Pokemon, https://www.twitch.tv/gemini_plays_pokemon [37] Google (2025), "Gemini 2.5", <https://blog.google/technology/ai/google-gemini-next-generation-december-2025/> [38] DeepLearning.AI, "Courses", <https://www.deeplearning.ai/courses/> [39] Artificial Analysis (2026), "GPT-OSS-120B Model Providers and Performance Analysis", <https://artificialanalysis.ai/models/gpt-oss-120b/providers> [40] Cerebras Systems (2026), "Cerebras – AI Supercomputing at Unprecedented Speed", <https://www.cerebras.ai/> [41] LMSYS Org (2026), "Arena AI Leaderboard (formerly LMSYS Chatbot Arena)", <https://lmarena.ai/leaderboard/> [42] Stanford CRFM, "Holistic Evaluation of Language Models (HELM)", <https://crfm.stanford.edu/helm/capabilities/latest/> [43] Nejumi LLM 리더보드 4, <https://wandb.ai/llm-leaderboard/nejumi-leaderboard4/reports/Nejumi-LLM-4--VmlldzoXZzc1OTk1MA> [44] Hugging Face (2024), "What's going on with the Open LLM Leaderboard and MMLU?", <https://huggingface.co/blog/open-llm-leaderboard-mmlu> [45] Google DeepMind (2026), "Gemini – Google's Next-Generation AI Models", <https://deepmind.google/models/gemini/> [46] OpenRouter (2026), "OpenRouter – A unified API for AI models", <https://openrouter.ai/>

03

Day 3: 사전학습 (Pre-training)

이 장의 구성

- 01 1. 사전학습과 언어 모델
- 02 2. Transformer의 모델 구조
- 03 3. 사전학습 파이프라인
- 04 4. 발전 주제: 데이터셋의 변천
- 05 5. 발전 주제: 모델 아키텍처의 발전
- 06 6. 발전 주제: 목적 함수, 평가, 분석
- 07 7. 정리
- 08 참고문헌

"

본 자료는 도쿄대학교 마츠오·이와사와 연구실이 작성한 2025년 LLM 대규모 언어 모델 강좌 기초편(10~11월 개최) 강의 자료를 재구성한 것이다. 강사는 우치야마 후미야(内山史也). 원자료는 크리에이티브 커먼즈 CC BY-NC-ND 4.0(저작자표시-비영리-변경금지 4.0 국제) 라이선스로 제공된다. 재이용 시 라이선스 표기를 반드시 기재하고, 참조 논문은 본 말미의 참고문헌에서 인용처를 게재할 것. 영리 목적 재이용은 별도 문의가 필요하다.

1. 사전학습과 언어 모델

강의의 목적과 구성

이 장에서는 대규모 언어 모델(LLM)의 주류 모델 구조인 Transformer와 그 사전학습 메커니즘을 이해하는 것을 목적으로 한다. 구체적 목표는 세 가지다. (1) 언어 모델에서 Transformer의 위치를 설명할 수 있다. (2) LLM에서 주류가 된 Transformer의 모델 구조를 설명할 수 있다. (3) LLM의 사전학습 파이프라인을 설명할 수 있다. 실습에서는 PyTorch를 활용해 Transformer를 직접 구현하고 학습한다.

LLM의 학습은 세 단계로 구성된다. Step 1 사전학습은 대규모 코퍼스를 통한 자기 지도 학습으로 모델에 어휘·문법·기본 지식 등 기초적인 언어 이해를 획득시키는 단계이며, 본 장(Day 3)의 주제다. Step 2 파인 튜닝은 레이블이 있는 데이터를 통한 지도 학습으로 사전학습된 모델의 성능을 개선하거나 특정 태스크나 도메인에 적응시킨다(Day 6). Step 3 강화 학습은 인간 피드백을 활용해 모델의 출력이 인간의 가치관에 부합하도록 조정한다(Day 7).

언어 모델이란

언어 모델(Language Model)이란 단어 시퀀스(≈문장)의 발생 확률 $p(x_1, x_2, \dots, x_L)$ 를 모델화한 것이다. 이 확률을 연쇄법칙으로 분해한 형태를 자기회귀 언어 모델(autoregressive language model)이라 한다.

$$p(x_1, x_2, \dots, x_L) = P(x_1), p(x_2|x_1), \dots, p(x_L|x_1, \dots, x_{L-1})$$

조건부 확률을 알면 텍스트 생성이 가능해진다. 예컨대 "日本の首都は"라는 문맥이 주어졌을 때 $p(\text{東京}|\text{日本, の, 首都, は}) = 0.2$, $p(\text{パリ}|\text{日本, の, 首都, は}) = 0.001$, $p(\text{カイロ}|\text{日本, の, 首都, は}) = 0.0005$ 와 같이 단어별 확률을 계산할 수 있으며, 다음 단어 x_L 의 적절한 예측은 $\text{argmax}_{x_L} p(x_L|x_1, \dots, x_{L-1})$ 로 얻는다. Day 1에서 복습한 "日本の首都は → 東京"도 $\text{argmax}_{x_L} p(x_L|\text{日本, の, 首都, は})$ 에 해당한다. 이 조건부 확률을 신경망으로 표현한 것이 신경망 언어 모델이다.

딥러닝 이전의 언어 모델

딥러닝 이전에는 조건부 확률을 통계적으로 구했다. 대규모 코퍼스 내 단어열 출현 빈도로부터 확률을 산정하며, 단어열 s 의 출현 횟수를 $\#(s)$ 라 하면 $p(\text{東京} | \text{Wtext}\{\text{日本, の, 首都, は}\}) = \#(\text{Wtext}\{\text{日本, の, 首都, は, 東京}\}) / \#(\text{Wtext}\{\text{日本, の, 首都, は}\})$ 로 추정한다 [1]. 이 접근에는 두 가지 과제가 있다. 첫째, 데이터 희소성(data sparseness) 문제로 단어열이 길어지면 출현 횟수가 급격히 감소해 조건부 확률 추정이 어려워진다. 둘째, 유의어가 개별 사건으로 취급되어 "日本の首都は?"과 "日本国の首都は?"처럼 미세한 표현 차이만으로 출현 빈도가 다른 단어로 취급된다.

이를 보완하기 위해 직전 $N-1$ 개의 단어만으로 다음 단어를 예측하는 N-gram 언어 모델이 사용되었다. 3-gram 예로 $p(\text{東京} | \text{Wtext}\{\text{日本, の, 首都, は}\}) \approx p(\text{東京} | \text{Wtext}\{\text{首都, は}\})$ 이며, 데이터 희소성 문제를 어느 정도 회피할 수 있다. 그러나 직전 2단어 "首都 は"만으로는 "東京"을 특정하기 어렵듯 장거리 단어 간 관계를 파악하기 어렵다는 과제가 남으며, 이는 후술하는 Transformer로 해결된다 [1].

신경망 언어 모델과 RNN의 과제

신경망 언어 모델은 조건부 확률을 어떤 신경망으로 추정한 모델이며, 다른 기계학습과 마찬가지로 가능도(likelihood)를 최대화하도록 오차 역전파로 훈련한다. 신경망 언어 모델은 특히 기계 번역 분야에서 크게 발전했으며, 입력 문장(번역 원 언어)을 받는 인코더와 출력 문장(번역 대상 언어)을 출력·재귀 입력하는 디코더로 구성된다 [2].

대표적인 RNN형 언어 모델인 Seq2Seq [3]는 순환 신경망(RNN: Recurrent Neural Network)으로, 첫 단어부터 한 단어씩 입력해 뉴런을 순차적으로 갱신하며 파라미터를 재사용한다. 원리적으로는 임의 개수의 단어를 입력·출력할 수 있으나 두 가지 과제가 있었다. 첫째, 뉴런이 고정 길이이므로 장문이 되면 모든 정보를 기억할 수 없어 단어 간 장거리 의존성 파악이 곤란하다. 둘째, 네트워크가 단어 방향으로 깊어져 학습이 불안정(기울기 소실·기울기 폭발)하고 속도가 느리다.

Transformer [4][5]는 어텐션(attention) 메커니즘을 최대한 활용해 이 두 과제를 해결한다. 단어 간 장거리 의존성을 파악할 수 있게 되었고, 오차 역전파 스텝 수가 단어 수에 의존하지 않게 짧아져 학습의 안정화와 고속화를 실현했다. 어텐션을 RNN에 도입한 선행연구 [4]는 있었으나 전 단어 간에는 아니었으며, Transformer가 전 단어 간에 어텐션을 도입하고 멀티헤드 어텐션(multi-head attention)을 새롭게 채택한 점이 차별적이다.

2. Transformer의 모델 구조

Transformer가 주류가 된 배경

Transformer는 2017년 "Attention Is All You Need" [6]에서 최초로 발표된 이래, 어텐션 메커니즘 채용으로 단어(토큰)의 장거리 의존 관계를 효율적으로 학습하며, 학습 시 병렬 계산도 효율화되어 분산 학습을 통한 대규모화가 쉬워졌다. 발표 이후 모델 개량과 스케일링을 통해 다수의 벤치마크에서 당시 최고 성능(SOTA)을 지속적으로 달성했다.

GPT-1~4, gpt-oss, Gemini 2.5 등은 모두 Transformer를 채택한다. GPT-4는 상세 구조가 비공개이나 Technical Report에서 Transformer 기반임을 명시하고 있으며 [7], GPT-5의 아키텍처는 불명이지만 system card에 LLM으로 기재되어 있다 [8][9]. 평가 사례로 GPT-5는 AIME 2025(미국 고교생 수학 경진대회)에서 도구 없이 94.6%, 의료 분야 HealthBench Hard 46.2%를 기록했고, Gemini 2.5는 AIME 2025에서 88.0%, 최적화된 프레임워크로 포켓몬스터 블루를 406.5시간에 클리어했다 [10].

블록 구조와 Encoder/Decoder

Transformer의 최소 단위는 "블록(block)"이며, 좌측 Encoder 블록을 세로로 N층, 우측 Decoder 블록도 세로로 N층 배열하여 구성된다 [6]. 토큰 수만큼 블록이 가로로 늘어나고 어텐션 메커니즘에 의해 가로 블록끼리 정보가 전달된다. 다만 실제로는 Encoder가 없어도 Decoder만으로 텍스트 생성이 가능하며(출력과 재귀 입력이 있으므로), GPT 시리즈가 이 형식을 취한다. Transformer가 최초 제안된 분야가 기계 번역이었기에 선행연구의 Encoder-Decoder 형식을 따라 제안되었을 뿐이다.

Transformer의 구성 요소는 크게 Embedding, Multi-Head Attention, Feed Forward, Others(Add & Norm, 출력층)로 나뉜다.

Embedding: 단어의 벡터 변환

텍스트를 Transformer 블록에 가져오는 첫 단계는 Embedding이다. "春は曙"라는 텍스트는 토큰나이저(후술)에 의해 "春", "は", "曙"와 같은 토큰으로 분할되고, 각 토큰은 일대일로 할당된 토큰 ID(예: 1050, 80, 24567)로 변환된다. 이를 다시 {토큰 ID}번째만 1이고 나머지는 0인 one-hot 벡터로 구성한 뒤, MLP에 의한 저차원 변환(학습 대상)을 거쳐 Word Embedding이라는 단어의 분산 표현을 얻는다. 이는 색을 RGB의 3차원 벡터로 변환하는 것과 비슷하다 [15].

Word Embedding(WE)은 단어라는 sparse한 정보를 dense한 표현으로 변환하여 Transformer 등 모델의 입력값으로 취급 가능하게 한다. 학습 완료 후의 Word Embedding에는 단어 간 의미의 유사성이나 관계성이 내장된다.

Transformer 블록의 알고리즘은 토큰의 위치 정보에 의존하지 않으므로, 블록에 가져오기 전에 Positional Encoding(PE)으로 위치 정보를 추가해야 한다. 토큰 위치에 따라 다른 PE를 각 WE에 더하며, 예컨대 $WE("春") + PE("이것은 1번째 토큰입니다")$, $WE("は") + PE("이것은 2번째 토큰입니다")$, $WE("曙") + PE("이것은 3번째 토큰입니다")$ 와 같다 [16].

Multi-Head Attention

Attention은 모든 토큰 간의 유사도를 측정함으로써 장거리 토큰 간의 의존 관계를 파악할 수 있게 한 메커니즘이다. 핵심 수식은 다음과 같다 [6].

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

"春 は 曙" 세 토큰의 예로 단계를 보면 [17]. 우선 각 토큰의 벡터 표현을 어텐션 메커니즘의 입력값으로 받는다. 각 토큰에 대해 선형 변환(MLP)으로 Key 벡터와 Value 벡터를 작성하고, 대상 토큰(예: 첫 번째 "春")에 대해서는 Query 벡터를 작성한다. Query 벡터와 Key 벡터의 내적으로 토큰 간 유사도(Score)를 측정하고, 이를 Softmax로 정규화(합계 1)한다. 값이 클수록 단어 간 의존이 강하다. 정규화된 유사도(실수)와 Value 벡터를 곱하고 모든 벡터의 총합을 계산하면 어텐션의 유사도에 따른 Value 벡터의 가중 평균이 산출되며, 이것이 해당 토큰의 어텐션 출력값이 된다. 같은 흐름을 두 번째 "は", 세 번째 "曙" 토큰에도 적용한다.

벡터 차원 수가 증가하면 Q 와 K 의 내적 값(분산)이 커지므로, 이를 억제하기 위해 차원 수의 제곱근 $\sqrt{d_k}$ 로 나눈다. 어텐션을 통해 각 토큰의 벡터는 다른 토큰과의 관계성을 흡수하여 더 나은 표현으로 변환된다. 1스텝으로 모든 단어와 연결됨으로써 각 토큰이 필요한 토큰의 정보만 유연하게 취사선택할 수 있으며, 이는 시계열 순서대로 토큰을 가져오는 RNN형에서는 실현 불가능했다. 직전 토큰만 보면 충분한 경우는 가까이 보고, 멀리 있는 토큰이 중요한 경우는 멀리 보는 식이다. 결과적으로 (1) 단어 간 장거리 의존 관계를 파악할 수 있고, (2) 기울기 소실/폭발 없이 안정적이며 GPU 등에서 병렬 연산 처리하기 쉬워 학습이 가속화된다.

어텐션은 시각화도 가능하다. 모든 단어 간 Attention Map(히트맵)을 만들면 "it"이 "The animal"에 강한 어텐션이 걸리는 식의 관계가 드러난다 [18]. 단, 항상 이 정도로 명확한 관계가 얻어지는 것은 아니다.

Encoder 측에서는 입력 텍스트 내의 Self-Attention을 수행한다. Decoder 측에서는 (1) 입력 텍스트와 출력 텍스트를 가로지르는 Cross-Attention, (2) 출력 텍스트 내 Self-Attention을 수행하되, 출력은 미래 토큰을 예측하는 메커니즘이므로 자신보다 미래 토큰에는 어텐션을 못 하도록 Causal Attention Mask를 적용한다. 구현 상으로는 Attention Map의 Softmax 직전에 해당 요소에 큰 음수 값(예: $-1.0e+10$)을 더하는 방식이다.

Multi-Head Attention은 어텐션 처리를 여러 개 병렬로 수행한 뒤 출력을 하나의 벡터로 통합한다. h 개의 어텐션 메커니즘 출력(벡터)을 Concatenate하며, 하나의 토큰이 다양한 토큰에 서로 다른 형태의 어텐션을 적용할 수 있다 [6].

Feed Forward

Feed Forward는 거대한 2계층 MLP로, 활성화 함수(ReLU)를 사이에 두고 학습 파라미터를 가진다. 중간층의 차원 수는 입력/출력층 차원 수의 수 배이다.

GPT-3 예시로 입력층/출력층 차원 수 12,288, 중간층 $12,288 \times 4 = 49,152$, 총 블록 수 96이며, Feed Forward 파라미터 수는 $(12,288 \times 49,152) \times 2 \times 96 \approx 116\text{B}$ 로, GPT-3 전체 175B 파라미터의 약 66%를 차지한다. 최근 모델의 Feed Forward 파라미터 비율도 gpt-oss120B 71%, Llama3.1-405B 81.4%, DeepSeek-v3 62%, Qwen3 64%에 달한다(괄호 안은 MoE 모델 총 파라미터 수로, 입력에 따라 사용하는 파라미터가 달라져 실제 예측 시에는 괄호 왼쪽 수만 사용된다). 최근 모델은 SwiGLU 등 약간 다른 알고리즘을 채택한다 [13][14].

Feed Forward는 파라미터 수가 큰 만큼 중요한 역할을 한다. 제1층의 파라미터를 Key(K)로, 제2층의 파라미터를 Value(V)로 간주하면 입력 패턴 추출과 패턴 의미 표현을 수행하는 신경망 메모리를 모방한 것으로 해석 가능하며, 곧 "지식을 축적하는 장소"로 간주할 수 있다 [19].

Add & Norm과 출력층

Add는 잔차 연결(residual connection)로 깊은 층 학습에 사용하는 기법으로 Feed Forward/Attention 이후에 적용되며 [20], Norm은 층 정규화(Layer Normalization)로 은닉층 차원 축으로 평균과 분산을 취해 학습을 안정화한다 [21]. 출력층에서는 선형 변환 후 Softmax 함수를 적용하여 다음 단어의 발생 확률을 출력한다.

3. 사전학습 파이프라인

사전학습의 의의

LLM 이전에는 번역·요약·독해 등 각 태스크마다 개별 모델을 학습했다. LLM은 대규모 코퍼스로 사전학습하여 범용 모델을 만들고, 그 위에 파인튜닝과 강화 학습 등 사후 학습으로 태스크별 성능을 끌어올린다. 사전학습의 목적은 후속 태스크에 공통으로 필요한 범용 지식(읽기·쓰기·셈하기)을 학습하고 이를 전이(Transfer Learning)하는 데 있다. 동시에 후속 태스크를 위한 좋은 파라미터 초기값을 얻는 것으로도 해석된다 [22]. 사전학습 파이프라인은 데이터 수집, 데이터 전처리, 훈련, 평가의 네 단계로 구성된다.

데이터 수집

사전학습용 데이터는 일반적으로 웹 대규모 크롤 데이터이며, 코드·백과사전·논문·일반 웹사이트(뉴스·블로그·홈페이지)·수학 특화 데이터 등으로 구성된다. 작년에 릴리스된 OLMo2의 데이터 소스 비율이 공개 사례로 참고된다 [12].

데이터 양은 최근 모델 기준 1~40조 토큰의 텍스트를 사용한다(토큰은 언어 AI가 처리하는 언어의 단위로, 일본어는 대략 1문자 1토큰). 1조 토큰을 서적으로 환산(1권 10만 토큰 가정)하면 약 1,000만 권에 상당하며, 이는 도쿄대 도서관(1,000만 권 이상) 규모에 맞먹는다(국회도서관은 약 4,800만 권). 구체적인 토큰 수는 GPT-3 0.5조, OLMo 2 3.9조, Llama3.1 15조, DeepSeek-v3 14.8조, Qwen3 36조 등이며, gpt-oss120B는 수조 토큰을 사용한다 [13][14][24].

데이터 전처리

LLM 사전학습의 전형적인 전처리 파이프라인은 네 단계로 이루어진다 [25][26]. 첫째, Quality Filtering(품질 필터링)은 분류기나 휴리스틱으로 질이 낮은 데이터를 제거한다. 둘째, De-dup(중복 제거)은 가까운 위치에 중복이 있으면 학습에 악영향이 크므로 문장·문서·데이터셋 등 다양한 단위로 중복을 제거한다. 셋째, Privacy Reduction(프라이버시 저감)은 개인 식별 정보를 제거하며, AI2 Dolma에서는 logistic classifiers(content tagging)와 정규표현식(PII detection)을 조합해 이메일 주소·전화번호·IP 주소를 검출·마스킹한다 [25]. 넷째, Tokenization(토큰화)은 텍스트를 토큰이라는 최소 단위로 분할하는 것으로, 다음 절에서 다룬다. 데이터셋마다 전처리 방식은 다르다.

Tokenization

토큰화는 텍스트를 토큰이라 불리는 최소 단위로 분할하는 것이며, 이를 수행하는 프로그램이 토큰라이저다. 대표 예로 Byte Pair Encoding(BPE)과 SentencePiece가 있으며, 어휘 출현 빈도에 기반한 알고리즘으로 효율적 토큰화를 실현한다. 코퍼스에서 정의한 알고리즘에 따라 토큰라이저가 어휘 사전을 작성한 뒤 토큰화를 수행한다. 예컨대 "吾輩は猫である。"는 "吾輩", "は", "猫", "で", "ある", "." 로 분할될 수 있다.

대표 기법인 BPE는 텍스트를 서브워드(단어보다 세밀한 단위)로 분할하며, 어휘 크기(기본 어휘 수 + 병합 수)는 하이퍼파라미터다. GPT, GPT-2, RoBERTa, BART, DeBERTa 등 다수 Transformer에서 사용된다. 학습 과정의 예를 보면, 코퍼스가 'hug'(10), 'pug'(5), 'pun'(12), 'bun'(4), 'hugs'(5)의 5개 단어로 이루어졌다고 가정할 때, (1) 각 단어를 문자로 분할하여 ('h','u','g', 10), ('p','u','g', 5), ... 형태로 만들고, (2) 가장 빈도가 높은 인접 쌍 ('u','g')을 ('ug')로 병합하며, (3) 원하는 어휘 수에 도달할 때까지 빈도가 높은 쌍의 병합을 반복한다 [46].

BPE 이외의 서브워드 단위 토큰화도 있다. WordPiece는 인접 쌍 2요소의 출현 빈도가 낮은 쌍(그 조합 이외에서는 거의 없는 쌍)을 우선 병합하며 $W_{\text{text}}\{\text{Score}\} = W_{\text{text}}(\text{인접 쌍})(a,b)W_{\text{text}}\{\text{의 출현 횟수}\} / (aW_{\text{text}}\{\text{의 출현 횟수}\} W_{\text{times}} bW_{\text{text}}\{\text{의 출현 횟수}\})$ 로 정의한다. (un, ##able)처럼 각 요소가 다른 단어에서도 빈출할 가능성이 큰 쌍은 그대로 두고, (hug, ##ging)처럼 각 요소가 다른 곳에서 빈출하지 않는 쌍은 병합한다. BERT, ELECTRA 등에서 사용된다. SentencePiece는 사전 단어 분할 없이 그대로 텍스트를 분할하며, 어휘 집합에 공백을 추가하고 BPE나 Unigram 등 알고리즘으로 어휘를 병합한다. 일본어 등 영어 이외의 다양한 언어에서도 쉽게 토큰라이저 작성이 가능하고 서브워드 분할 알고리즘도 선택할 수 있어 T5, ALBERT 등에서 사용된다 [46].

이모지 등 처리에 관해서는, 코퍼스에 없는 문자는 로 변환되므로 많은 NLP 모델이 이모지로 콘텐츠를 분석하는 것을 서투름한다. GPT-2와 RoBERTa의 토큰라이저는 이에 대응하기 위해 byte 레벨에서 BPE를 수행한다. 참고로 ByT5 [47]는 텍스트열을 토큰이 아닌 UTF-8 바이트열로 표현하는 token-free 접근으로, 서버워드로 토큰라이즈한 mT5에 필적하는 성능을 보였다.

훈련: Next Token Prediction

사전학습의 훈련은 Next Token Prediction이라는 자기 지도 학습의 일종으로, 학습용 텍스트 데이터를 사용해 다음 토큰의 생성 확률을 계속 예측한다. "吾輩は猫である。" 예시에서 $P(\text{吾輩}|\text{は})$, $P(\text{猫}|\text{吾輩,は})$, $P(\text{で}|\text{吾輩,は,猫})$, $P(\text{ある}|\text{吾輩,は,猫,で})$, $P(\text{。}|\text{吾輩,は,猫,で,ある})$ 를 차례로 계산하며, 예측과 정답의 오차(교차 엔트로피)가 작아지도록, 즉 사전학습의 목적 함수로 $\mathbb{E}[\text{교차 엔트로피}]$ 를 사용해 학습한다. 미니배치 내 각 샘플 문장마다 교차 엔트로피를 계산하여 평균한 것을 Loss로 사용한다.

일반적으로 1 epoch만 학습하며(1~3 범위), 여러 epoch 학습하면 과적합으로 degradation(성능 저하)되거나 차이가 없고, 모델 크기가 커질수록 degradation 경향이 강해진다 [23][27][28][29][30]. 원리적으로는 간단하지만 대규모 모델 학습(다중 노드 분산 학습)에서는 교차 엔트로피 발산(Loss 스파이크)이나 하드웨어·네트워크 저수준 에러가 발생하며 [31][32], 계산하는 수치 포맷에 따라 안정성이 달라진다(최근 은 bfloat16이 주류).

하이퍼파라미터 예로 Optimizer는 Adam [33], AdamW [34], Scheduler는 Learning Rate Warmup + Decay, 부동소수점 정밀도는 BF16, Batch Size는 수백만 토큰이 일반적이다. 미니배치 내 토큰 수는 (샘플 수 × 최대 토큰 길이)로 계산된다 [23][35].

평가

평가는 크게 정량 평가(Upstream)·정량 평가(Downstream)·정성 평가(샘플 평가)로 나뉜다.

정량 평가(Upstream)의 대표 지표는 교차 엔트로피 Validation Loss로, Loss가 떨어지고 있는지(사전학습 자체가 붕괴하지 않았는지) 모니터링하고 모델 간 성능 차이를 확인한다. Test Loss는 논문에서 거의 보이지 않는데, 사전학습이 1 epoch 학습이 일반적이므로 overfit하지 않다는 전제 때문인 것으로 보이며, Training Loss만으로 끝나는 경우도 많다 [23][24]. 교차 엔트로피는 Cross Entropy Loss, CELoss 등 다양한 명칭으로 불리며, 식 변형 시 실질 동일한 지표로 Perplexity(PPL), Bits-Per-Character(BPC), Bits-Per-Word(BPW) 등이 있다 [7][28][36].

정량 평가(Downstream)는 다양한 하위 태스크(최종적으로 풀고자 하는 태스크)로 평가하며, In-Context Learning(Zero-shot, Few-shot, Day 2 복습)으로 평가하는 경우가 많다. 사후 학습(파인튜닝이나 RLHF, Day 5·Day 7 예정)으로 하위 태스크 성능은 더 향상된다. 평가 벤치마크 상세는 본 장 "발전 주제" 절을 참조.

정성 평가(샘플 평가)는 사전학습된 LLM을 사용해 텍스트를 출력(디코딩)해 보는 것이다. 디코딩 방식에는 Greedy Decoding, Beam Search, Random Sampling이 있다 [37][38][39][68]. Greedy Decoding은 발생 확률이 가장 높은 다음 토큰을 순차적으로 선택한다. Beam Search는 높은 발생 확률이 되는 토큰 시퀀스를 탐색하여 발견하는데, 직전뿐 아니라 그 이후까지 보고 결정하되 전 시퀀스 탐색(Exhaustive Search)하면 계산량이 폭발하므로 미리 정해진 빔 크기 내에서 탐색한다. Random Sampling은 다음 토큰의 발생 확률 분포를 따라 무작위로 선택하며, 상위 $p\%$ 토큰에서 선택하는 Top_p(예: 0.9), 상위 k 개 토큰에서 선택하는 Top_k(예: 10), 그리고 Softmax 직전 Logit의 분모에 곱하는 Temperature(0 이상의 실수, 1이면 일반 Softmax와 동일) 등의 파라미터로 조정한다. 상황에 따라 바람직한 방식이 달라 분류 문제에는 Greedy Decoding, 기계 번역에는 Beam Search, 장문 생성에는 Random Sampling이 자주 쓰인다.

4. 발전 주제: 데이터셋의 변천

학습 데이터셋의 흐름

GPT-2 시절에는 웹페이지 코퍼스(약 40GB)만으로 학습했으나, 최근에는 Code나 대화 데이터 등 다양한 데이터로 학습하는 모델이 증가했다 [26]. 대표적인 공개 코퍼스들을 정리한다.

C4 [41]는 Common Crawl(공개된 웹 아카이브를 스크래핑하여 수집, 월당 약 20TB)의 2019년 4월 웹 추출 데이터 중 영어 판정 결과와 다수의 필터링·클렌징을 거쳐 수집된 필터링된 거대 웹페이지 영어 코퍼스다. 다국어 확장으로 CC-100 [42]은 100개 언어에 걸쳐 수집된 텍스트 코퍼스로 각 언어 모델과 fastText로 필터링을 수행하며, mC4 [43]는 C4와 마찬가지로 언어 판정 후 필터링한 101개 언어를 포함한다.

The Pile [44]은 22개의 다양한 소스를 조합한 825.18GB의 언어 모델링용 데이터셋으로, 학습 데이터셋의 다양성을 높여 크로스 도메인 성능을 기대할 수 있으며 CC-100이나 Common Crawl로 학습한 모델 성능을 상회한다. Dolma [25]는 웹 콘텐츠·학술 출판물·코드·서적·백과사전의 다양한 조합으로 구성된 5334GB(3T tokens)로 최대급 혼합 사전학습용 공개 데이터셋이며, 과거 연구도 반영해 데이터 처리 베스트 프랙티스를 따랐다고 언급한다.

Dolma의 전처리 프로세스

Dolma의 텍스트 데이터 처리는 6단계로 진행된다 [25]. (1) fastText 언어 식별 모델로 영어일 가능성이 50% 이상인 문서를 보존하고, (2) 출처 URL 기반으로 중복을 제거하며, (3) 구두점으로 끝나지 않는 모든 단락을 필터링한다. 이어 (4) 유해하거나 음란할 가능성이 60% 이상으로 판정된 것을 삭제하며 개인 정보도 정규표현식으로 검출해 마스크하고, (5) 문서 내 중복 단락을 삭제하며, (6) 평가 셋에 포함된 13 토큰 이상 단락을 학습 셋에서 제거한다. 데이터셋마다 전처리 방식은 다르다.

FineWeb

FineWeb [40]은 Llama 아키텍처 소규모 모델 70개 이상을 학습하여 진행한 어블레이션(ablation) 실험을 통한 경험적 베스트 프랙티스를 반영한 데이터셋으로, CommonCrawl에서 정제한 18.5T tokens로 구성된다. 웹 정제에 특화함으로써 RefinedWeb(5T), RedPajama-v2(영·불·서·독 합계 30T) 대비 더 큰 데이터셋 구축을 가능하게 했고, 소규모 모델 실험에서 다른 공개 데이터셋보다 높은 학습 효율을 실현했다 [27][45].

파생 데이터셋으로 분류기를 사용한 FineWeb의 "교육적" 서브셋 1.3T tokens인 FineWeb-edu, 그리고 다국어판 FineWeb2("FineWeb2: One Pipeline to Scale Them All — Adapting Pre-Training Data Processing to Every Language" [69])가 있다. 일본어는 FineWeb2에 331Billion words로 비교적 풍부하게 포함되어 있다.

5. 발전 주제: 모델 아키텍처의 발전

Transformer의 세 분류

Transformer는 구조에 따라 세 가지로 분류된다 [6]. Encoder-only(BERT, RoBERTa 등)는 인식계(분류)에, Encoder-Decoder(BART, T5 등)는 텍스트 생성계에, Decoder-only(GPT, Llama 등) 역시 텍스트 생성계에 해당한다. 최근 공개되는 모델은 Decoder-only가 많다 [26].

Encoder-only 계열: BERT, RoBERTa, ALBERT

BERT [48]는 Transformer의 Encoder를 24층 쌓은 양방향 언어 모델로, 사전학습에서 빈칸 채우기 (MLM) 태스크와 다음 문장 예측(NSP) 태스크를 학습하고 목적 태스크의 데이터셋으로 파인튜닝하여 11개 NLP 태스크에서 SoTA를 달성했다. 예시로 "[CLS] my dog is cute [SEP] he likes [MASK] ##ng [SEP]"에서 빈칸을 채우고 IsNext 여부를 판정한다.

RoBERTa [49]는 BERT와 동일한 아키텍처에서 일부 요소를 변경해 성능을 높였다. 데이터셋 크기 13GB→160GB, 배치 크기 256→8K, NSP 미사용, 마스크를 동적으로 적용하는 변경으로 GLUE와 SQuAD에서 BERT를 상회했다.

ALBERT [50]는 층 간 파라미터 공유로 BERT-large와 동일한 아키텍처 대비 파라미터를 18배 감소시키고 1.7배 빠른 학습을 가능하게 했다. ALBERT-xxlarge는 BERT-large보다 적은 파라미터 수임에도 GLUE, SQuAD에서 SoTA를 달성했다.

Encoder-Decoder 계열: BART, T5

BART [51]는 BERT 같은 양방향 인코더와 GPT 같은 자기회귀형 디코더를 결합한 모델로, 무작위로 입력 문서 일부를 훼손시키고 그 재구성을 수행하는 복수 태스크 조합으로 사전학습하여 CNN/DailyMail, XSum 등에서 SoTA를 달성했다.

T5 [41]는 다수의 자연어 처리 태스크를 Text-to-Text 형태로 변환하여 통일된 프레임워크로 학습한다. 사전학습에서는 입력 문서 일부를 무작위로 특수 토큰으로 치환하고 치환 전 토큰을 예측하는 태스크로 학습해 GLUE, SuperGLUE 등에서 SoTA를 기록했다.

Decoder-only 계열: GPT-3

GPT-3 [24]는 GPT-2의 약 120배 파라미터 수를 가진 모델을 약 14배 데이터로 학습했다. 태스크 관련 설명이나 소수 샷 예시를 입력에 추가해 태스크를 풀 수 있는 문맥 내 학습(In-context Learning)이 가능해져 소수 샷 설정에서 기존 SoTA에 필적 혹은 상회하는 성능을 확인했다. 성능이 너무 뛰어나 모델 공개 없이 API 공개에 그쳤다.

정규화 위치와 위치 표현

정규화 위치에 대해 세 가지 변형이 있다 [26]. Post Norm은 원래 Transformer와 마찬가지로 잔차 연결 후 정규화를 배치하나 출력층 부근에서 기울기가 커져 학습이 불안정해지는 경향이 있다. Pre Norm은 각 서브층 앞과 최종 예측 앞에 정규화를 배치하며 성능은 낮아지지만 학습 안정성으로 인해 자주 채택된다. Sandwich Norm은 잔차 연결 전에 추가 정규화를 배치하나 학습이 붕괴하는 경우도 존재한다.

위치 표현에는 절대 위치 표현과 상대 위치 표현이 있다 [52]. 절대 위치 표현은 각 토큰의 절대적 위치를 나타내는 어떤 표현(예: $\sin$ 과 $\cos$ 과)을 입력 표현에 더하는 방식으로, 입력 내용과 독립적이므로 계산 속도가 빠르나 알려지지 않은 길이의 시열 입력에 취약하다. 상대 위치 표현은 토큰 간 상대적 거리를 Attention 계산 시 활용하며 미지의 시열 길이에도 견고성이 높지만 입력에 고유한 값을 취해 추가 계산이 필요하다.

ALiBi [53]는 Attention 스코어 계산 시 Key와 Query의 상대적 거리에 대해 선형인 패널티를 가산하는 상대 위치 임베딩으로, 가까운 토큰 간보다 먼 토큰 간의 Attention 스코어를 더 낮춘다. 절대 위치 표현보다 성능이 좋고 외삽(extrapolation) 성능도 우수하다.

최근 LLM 모델 구조와 Attention 변형

최근 모델은 Attention/Feed Forward/Normalization이 개량되고 있다 [70]. Positional Embedding으로 RoPE, Attention으로 Grouped Query Attention·Sliding Window Attention·Multi-head Latent Attention, Feed Forward로 SwiGLU·Mixture of Experts, 그리고 Others로 RMSNorm이 채택되고 있다. 최신 모델 아키텍처 해설은 Sebastian Raschka의 "The Big LLM Architecture Comparison" [70]가 참고된다.

Sparse Attention은 종래 Attention의 시열 길이에 대한 제곱 계산 복잡성 문제의식에서 출발해, 모든 토큰에 대해 Attention을 계산하는 대신 국부적으로 설정한 토큰으로 학습하여 계산량을 절감한다(Big Bird [54], Longformer [55] 등의 유사 아이디어). Grouped-Query Attention(GQA) [56]은 Multi-head Attention이 디코딩 시 모든 Key와 Value를 읽어와야 해 추론 속도 병목이 되는 문제의식에서, Key와 Value를 일부(Group-query) 또는 하나(Multi-query)의 헤드에서 공유하여 메모리 부하를 줄이고 추론 속도를 향상시키며 Llama3 등에서 채택된다.

6. 발전 주제: 목적 함수, 평가, 분석

목적 함수의 발전

UniLM [57]은 Attention 마스크 영역을 변화시켜 양방향 언어 모델링, 단방향 언어 모델링, 배열 간 언어 모델링을 결합한 복합적 목적 함수로 사전학습해, GLUE 같은 식별 태스크에서 BERT에 필적하면서도 CNN/DM 같은 언어 생성 태스크에서 SoTA를 달성했다.

UL2 [58]는 T5 같은 결손 토큰 예측(R-Denoising, X-Denoising)과 GPT 같은 연속 토큰 예측을 결합한 MoD(Mixture-of-Denoisers)라는 통일적 목적 함수로 학습한다. MoD를 지속적 사전학습에 사용하는 UL2R(UL2 Repair) 훈련 방법도 후에 제안되었으며, EncDec과 Dec 양쪽 아키텍처에서 균형 있게 성능 향상을 확인했다.

평가의 확장과 FLASK

평가 대상은 자연어 이해(GLUE, SuperGLUE, SQuAD)에서 도메인 지식(MMLU, MATH, MultiMedQA, APPS, CUAD), 윤리성·신뢰성(FLASK, TrustGPT, TruthfulQA), 도구 활용(ToolBench)까지 확장되었다 [26]. FLASK [60]은 LLM에 필요한 스킬을 정의하고 모델의 행동에 대해 인간 또는 모델이 스코어링을 수행하는 벤치마크로, 주로 4가지 능력(논리적 사고, 배경 지식, 문제 해결 능력, 지시 추종성)으로 구성된 평가 프레임워크를 구축하고 이를 12개의 상세한 스킬 항목으로 세분화한다.

Attention 분석과 모델 해석

Attention의 대소는 자주 가시화되며 이를 Attention map이라 부른다 [61]. Attention이 단어 중요도를 나타내는 것처럼 보이지만 Attention에는 설명 능력이 없다는 입장의 논문도 복수 존재한다. 종래 Attention에서는 복수 층을 경유해 정보가 오가므로 Attention map 자체를 입력에 대한 기여로 해석하기에는 신뢰성이 부족하다. Attention Rollout [62]은 자신보다 앞에 있는 층의 Attention map을 순차적으로 곱하고, Attention Flow는 각 층의 Attention을 플로우 네트워크로 해석해 입력 토큰에 대한 어텐션을 근사한다.

참고로, 현재 사전학습과 Transformer는 스탠다드가 되었지만 이 둘이 세트의 의미가 있는 것은 아니다 [63]. CNN에서도 사전학습 효과가 있으며 일부 태스크에서 CNN 모델이 T5를 상회하는 성능을 발휘하기도 한다. CNN이 항상 Transformer의 대체가 되는 것은 아니지만, 사전학습이라는 패러다임 전환과 아키텍처의 변천은 분리해 생각해야 한다는 주장이 있다. 이론적으로 상대 위치 표현을 사용하면 Multi-Head Self-Attention은 Convolution을 내포한다 [64].

적대적 공격과 프로빙

적대적 공격(adversarial attack) 분석은 인간에게는 사소한 영향만 주는 섭동(perturbation)이라도 신경망이 크게 영향을 받는 경우가 존재한다는 문제의식에서 출발한다 [65]. 입력 일부를 편집해 모델 성능을 열화시키는 공격을 검증함과 함께 그 공격에 의한 실패를 막는 방어 방법을 검토한다.

프로빙(probing) [66]은 언어 모델이 내부적으로 획득한 능력을 분석하는 방법이다. 언어 모델의 태스크 성능은 출력으로 평가할 수 있지만 내부적으로 획득한 능력은 출력만으로는 쉽게 평가할 수 없다. 어떤 입력을 주었을 때 언어 모델에서 얻어지는 임베딩 표현에서 특정 태스크(예: 품사 분류)를 수행하는 분류기(프로브)를 훈련했을 때의 성공률로부터, 임베딩 표현에 그 태스크를 나타내는 표현이 인코딩되어 있는지 검증한다. 최근에는 모델의 임베딩 표현에 개입을 수행해 출력으로의 인과관계를 조사하는 경우도 많다.

7. 정리

본 장에서는 대규모 언어 모델(LLM)의 사전학습을 네 관점에서 소개했다.

첫째, 언어 모델에서 Transformer의 위치다. Transformer는 신경망 언어 모델의 하나로 취급되며, RNN형 언어 모델이 안고 있던 장거리 의존성 파악·학습 불안정성 과제를 해결했다.

둘째, LLM에서 주류가 된 Transformer 모델 구조다. Self-Attention 메커니즘을 가진 모델 구조이며 1스텝으로 전 단어 정보와 연결 가능하다. 이로써 과제①(단어 간 장거리 의존성 파악)과 과제②(오차 역전파 계산 스텝이 문장 길이에 비의존하게 되어 학습 안정·고속화)가 해결되었다.

셋째, LLM의 사전학습이다. 대규모 코퍼스로 학습을 수행함으로써 모델의 범용성을 높이고 있으며, Next Token Prediction이라는 자기 지도 학습으로 최적화한다.

넷째, 발전 주제다. 데이터·모델·학습·평가 분석에 대한 발전적 주제를 해설했다.

모델별 학습 방법과 세부 요소의 차이, 그리고 모델의 접근성(API 전용·공개 & 거대·비공개)에 관해서는 보충 자료와 "A Survey of Large Language Models" [26], "Holistic Evaluation of Language Models" [67]을 참조할 것.

참고문헌

- [1] 岡崎直観 (2023), "대규모 언어 모델의 경이와 위협", Speaker Deck. [2] Ilya Sutskever et al. (2014), "Sequence to Sequence Learning with Neural Networks", NeurIPS2014. [3] Tomáš Mikolov et al. (2010), "Recurrent Neural Network Based Language Model", Proc. Interspeech 2010, 1045–1048. [4] Dzmitry Bahdanau et al. (2014), "Neural Machine Translation by Jointly Learning to Align and Translate", arXiv:1409.0473. [5] Masaki Hayashi (2022), "Transformer와 seq2seq with attention의 차이는? 시열 변환 모델 【Q and A 기사】", CVML 전문가 가이드. [6] Ashish Vaswani et al. (2017), "Attention Is All You Need", NeurIPS2017. [7] OpenAI (2023), "GPT-4 Technical Report", arXiv:2303.08774. [8] OpenAI (2025), "GPT-5 System Card". [9] OpenAI (2025), "GPT-5가 등장". [10] Google (2025), "Gemini 2.5 tech report". [11] OpenAI (2024), "Learning to Reason with LLMs". [12] Allen Institute for AI, Univ. of Washington, NYU (2025), "2 OLMo 2 Furious", arXiv:2501.00656. [13] OpenAI (2025), "gpt-oss-120b & gpt-oss-20b Model Card", arXiv:2508.10925. [14] Qwen (2025), "Qwen3 Technical Report", arXiv:2505.09388. [15] Shraddha Anala (2020), "A Guide to Word Embedding", Towards Data Science. [16] John Hewitt, "Natural Language Processing with Deep Learning CS224N/Ling284". [17] Raimi Karim (2019), "Illustrated: Self-Attention", Medium. [18] Jay Alammar (2018), "The Illustrated Transformer", jalammar.github.io. [19] Mor Geva et al. (2021), "Transformer Feed-Forward Layers Are Key-Value Memories", EMNLP2021, 5484–5495. [20] Kaiming He et al. (2016), "Deep Residual Learning for Image Recognition", CVPR2016, 770–778. [21] Jimmy Lei Ba et al. (2016), "Layer Normalization", arXiv:1607.06450. [22] Rishi Bommasani et al. (2021), "On the Opportunities and Risks of Foundation Models", arXiv:2108.07258. [23] Hugo Touvron et al. (2023), "LLaMA: Open and Efficient Foundation Language Models", arXiv:2302.13971. [24] Tom Brown et al. (2020), "Language Models are Few-Shot Learners", NeurIPS2020. [25] Luca Soldaini et al. (2023), "Dolma: An Open Corpus of 3 Trillion Tokens for Language Model Pretraining Research", AI2 Blog. [26] Wayne Xin Zhao et al. (2023), "A Survey of Large Language Models", arXiv:2303.18223. [27] Guilherme Penedo et al. (2023), "The RefinedWeb Dataset for Falcon LLM", arXiv:2306.01116. [28] Hugo Touvron et al. (2023), "Llama 2: Open Foundation and Fine-Tuned Chat Models", arXiv:2307.09288. [29] Fuzhao Xue et al. (2023), "To Repeat or Not To Repeat: Insights from Scaling LLM under Token-Crisis", arXiv:2305.13230. [30] Niklas Muennighoff et al. (2023), "Scaling Data-Constrained Language Models", arXiv:2305.16264. [31] Stas Bekman (2022), "The Technology Behind BLOOM Training",

huggingface.co. [32] facebookresearch (2023), "OPT175B Logbook", GitHub metaseq. [33] Diederik P. Kingma & Jimmy Ba (2014), "Adam: A Method for Stochastic Optimization", arXiv:1412.6980. [34] Ilya Loshchilov & Frank Hutter (2017), "Decoupled Weight Decay Regularization", arXiv:1711.05101. [35] Shikoan's ML Blog (2021), "Cosine Decay와 Warmup을 동시에 수행하는 스케줄러 (timm)". [36] Chip Huyen (2019), "Evaluation Metrics for Language Modeling", thegradient.pub. [37] Kaito Sugimoto (2021), "텍스트 생성에서의 decoding 테크닉: Greedy search, Beam search, Top-K, Top-p", zenn.dev. [38] mm_0824 (2020), "빔서치(Beam Search) 이해하기", data-analytics.fun. [39] cohere, "Temperature", docs.cohere.com. [40] Hugging Face (2024), "The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale", arXiv:2406.17557. [41] Colin Raffel et al. (2020), "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer", JMLR 21(1):140, 5485–5551. [42] Alexis Conneau et al. (2020), "Unsupervised Cross-lingual Representation Learning at Scale", ACL2020, 8440–8451. [43] Linting Xue et al. (2021), "mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer", NAACL2021, 483–498. [44] Leo Gao et al. (2020), "The Pile: An 800GB Dataset of Diverse Text for Language Modeling", arXiv:2101.00027. [45] Weber et al. (2024), "RedPajama: an Open Dataset for Training Large Language Models", NeurIPS2024. [46] Hugging Face (2025), "Hugging Face LLM Course", Chapter 6.5–6.6. [47] Linting Xue et al. (2022), "ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models", TACL 10:291–306. [48] Jacob Devlin et al. (2019), "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", NAACL-HLT 2019, 4171–4186. [49] Yinhan Liu et al. (2019), "RoBERTa: A Robustly Optimized BERT Pretraining Approach", arXiv:1907.11692. [50] Zhenzhong Lan et al. (2020), "ALBERT: A Lite BERT for Self-supervised Learning of Language Representations", ICLR2020. [51] Mike Lewis et al. (2020), "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension", ACL2020, 7871–7880. [52] 清野舜 (2022), "보다 나은 Transformer 만들기", Speaker Deck. [53] Ofir Press et al. (2021), "Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation", arXiv:2108.12409. [54] Manzil Zaheer et al. (2020), "Big Bird: Transformers for Longer Sequences", NeurIPS2020. [55] Iz Beltagy et al. (2020), "Longformer: The Long-Document Transformer", arXiv:2004.05150. [56] Joshua Ainslie et al. (2023), "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints", arXiv:2305.13245. [57] Li Dong et al. (2019), "Unified Language Model Pre-training for Natural Language Understanding and Generation", NeurIPS2019. [58] Yi Tay et al. (2022), "UL2: Unifying Language Learning Paradigms", arXiv:2205.05131. [59] Yupeng Chang et al. (2023), "A Survey on Evaluation of Large Language Models", arXiv:2307.03109. [60] Seonghyeon Ye et al. (2023), "FLASK: Fine-grained Language Model Evaluation based on Alignment Skill Sets", arXiv:2307.10928. [61] Andrea Galassi et al. (2019), "Attention in Natural Language Processing", arXiv:1902.02181. [62] Samira Abnar & Willem Zuidema (2020), "Quantifying Attention Flow in Transformers", ACL2020, 4190–4197. [63] Yi Tay et al. (2021), "Are Pre-trained Convolutions Better than Pre-trained

Transformers?", ACL2021, 4349–4359. [64] Jean-Baptiste et al. (2020), "On the Relationship between Self-Attention and Convolutional Layers", ICLR2020. [65] Jack Morris (2020), "What are adversarial examples in NLP?", towardsdatascience.com. [66] Yonatan Belinkov (2022), "Probing Classifiers: Promises, Shortcomings, and Advances", *Computational Linguistics* 48(1):207–119. [67] Percy Liang (2022), "Holistic Evaluation of Language Models", [arXiv:2211.09110](https://arxiv.org/abs/2211.09110). [68] Harshit Sharma (2022), "Softmax Temperature". [69] Hugging Face, EPFL (2025), "FineWeb2: One Pipeline to Scale Them All — Adapting Pre-Training Data Processing to Every Language", COLM2025. [70] Sebastian Raschka, "The Big LLM Architecture Comparison", magazine.sebastianraschka.com.

04

Day 4: 스케일 법칙

이 장의 구성

- 01 강의의 목적과 배경
 - 02 스케일 법칙이란 무엇인가
 - 03 스케일 법칙의 활용 방법
 - 04 스케일 법칙의 구하는 방법
 - 05 추론 시의 스케일링
 - 06 정리
 - 07 참고문헌
-

이 장은 도쿄대학교 마츠후오·이와사와 연구실이 2025년 10~11월에 개최한 대규모 언어 모델 강좌 기초편 넷째 날 강의를 재구성한 것이다. 이론은 고지마 타케시(小島武), 실습은 여전군(余振軒)이 담당했다. 고지마 타케시는 2023년 도쿄대학교 대학원 공학계 연구과 TMI 박사 과정을 수료한 뒤 같은 연구과의 특임 연구원을 거쳐 2025년부터 특임 조교수를 맡고 있으며, 이전에는 IT 엔지니어로 일했다. Weblab-10B 개발, 기시다 총리·이시바 총리의 LLM 특별 강좌 강사, LLM 개발 콘테스트 2024·2025 운영 측 콘텐츠 리더, AI 백서 2025 Safety 장 집필 등에 참여했고, 연구 분야는 LLM의 작동 원리 이해와 제어(Reasoning Model, 다국어 등), Safety(Unlearning, 지시 추종 능력), Transformer 모델 구조 개선, 그리고 로봇이다. 본 자료는 CC BY-NC-ND 4.0(저작자표시-비영리-변경금지 4.0 국제) 라이선스로 제공된다.

강의의 목적과 배경

이 강의 목적은 언어 모델을 스케일, 곧 대규모화하는 의의를 학습하는 데 있다. 이를 위해 스케일 법칙이 무엇인지, 왜 중요한지, 구체적으로 어떻게 구하는지를 다루고, 나아가 학습 시뿐 아니라 추론 시에도 연산량을 스케일시키는 새로운 트렌드를 살펴본다. 실습에서는 PyTorch로 스케일 법칙을 직접 구하는 코드를 구현한다.

현대의 대규모 언어 모델(LLM)은 대부분 2017년에 발명된 Transformer 구조를 기반으로 하는 신경 언어 모델의 한 종류이다. LLM 학습은 일반적으로 세 단계로 이루어진다. 첫째, 사전학습(pretraining)은 대규모 코퍼스를 이용한 자기 지도 학습으로 어휘·문법·기초 지식과 같은 기초적인 언어 이해를 획득하는 단계이다. 둘째, 파인튜닝(fine-tuning)은 레이블이 있는 데이터를 이용한 지도 학습을 통해 사전학습된 모델의 성능을 개선하거나 특정 태스크나 도메인에 적응시키는 단계이다. 셋째, RLHF(인간의 피드백을 이용한 강화학습)는 모델의 출력이 인간의 가치관에 부합하도록 조정하는 단계이다. 이 장에서는 그중 사전학습 단계를 중심으로 스케일 법칙을 다룬다.

2018년 GPT, 2019년 GPT-2, 2020년 GPT-3에 이어 2023년 이후로도 GPT-5, Gemini 2.5, GPT-OSS, DeepSeek-R1, Qwen3 등 많은 LLM이 공개되었다. 이처럼 미국 기업을 중심으로 여러 연구 기관이 독자적인 대규모 언어 모델을 개발하게 된 배경에는 스케일 법칙이라 불리는 경험칙이 자리 잡고 있다.

스케일 법칙이란 무엇인가

사전학습과 스케일 법칙의 발견

사전학습은 웹에서 수집한 대량의 문장을 이용하여 다음 단어의 예측을 쉬지 않고 수행한다. 예를 들어 "봄은 벚꽃이 아름답다"라는 텍스트를 학습할 때, 모델은 "봄" 다음에 "은"이, "봄은" 다음에 "벚꽃"이 오는 식으로 예측한다. 예측과 정답의 오차, 곧 교차 엔트로피(cross entropy)가 작아지도록 모델을 학습하며,

이 과정에서 읽기·쓰기·셈하기 및 세계의 지식을 습득한다. "봄" "벚꽃" "아름답다"라는 단어 사이에 강한 관계성이 있다는 것(= 세계의 지식)을 학습하는 것이다. GPT 시리즈를 대표로 하는 현대의 LLM은 반드시 이 사전학습을 수행한다.

스케일 법칙(Scaling Law)은 이런 대규모 학습에서 발견된 경험칙으로, 거듭제곱 법칙(Power-Law)이라고도 불린다. 핵심적인 두 논문이 이 법칙을 정립했다. 하나는 2020년 1월 OpenAI의 Kaplan 등이 발표한 "Scaling Laws for Neural Language Models"이고(GPT-3는 같은 해 6월 발표), 다른 하나는 2022년 DeepMind의 Hoffmann 등이 발표한 "Training Compute-Optimal Large Language Models"(이하 Chinchilla 논문)이다.

세 가지 변수와 오차의 관계

스케일 법칙은 컴퓨팅 자원(C), 데이터셋 크기(D), 파라미터 수(N)와 오차(L) 사이에 성립하는 관계를 기술한다. 다른 두 변수가 충분히 크다고 가정할 때, 어느 변수든 오차(Test Loss)와의 사이에 이중 로그 그래프에서 선형 관계가 나타난다.

첫째, 파라미터 수(N)가 커질수록 오차(L)는 일정한 기울기로 감소한다. 둘째, 데이터셋 크기(D)가 커질수록 마찬가지로 오차가 감소한다. 셋째, 컴퓨팅 자원(C)에 대해서도 같은 형태의 관계가 성립한다. 이를 수식으로 나타내면 다음과 같다.

//

$$L(X) = (X_c / X)^{\alpha}$$

여기서 X는 C, D, N 중 하나의 변수이고, α 는 이중 로그 그래프 상의 기울기, X_c 는 절편에 해당하는 값이다. 관계가 거듭제곱 형태로 표현된다는 것이 스케일 법칙이라는 이름의 유래이다.

연산량의 단위: FLOPs

컴퓨팅 자원을 나타내는 가로축에 사용되는 단위는 FLOPs(FLoating Points OPerations)이다. 이는 모델 학습에 필요한 총 부동소수점 연산 횟수를 의미한다(파라미터의 덧셈·곱셈 등). 주의할 점은 단위 시간당 처리 능력을 나타내는 HW 성능 지표인 FLOPS(Floating Points Operation Per Second, 대문자 S)와 혼동하기 쉽다는 것이다. 접두사로는 Mega(M, 10^6), Giga(G, 10^9), Tera(T, 10^{12}), Peta(P, 10^{15}), Exa(E, 10^{18}) 등이 사용된다. 스케일 법칙 논문에서는 PF-days(Peta FLOPs days), 즉 1 Peta FLOPS 처리 속도의 서버를 며칠분 학습에 사용했는지를 단위로 쓴다. 참고로 GPT-3의 총 연산량은 약 3.14×10^{23} FLOPs로 알려져 있으며, 최근 모델들의 FLOPs는 상세가 비공개이므로 정확히 알 수 없다.

LLM 학습에 필요한 연산량은 파라미터 수(N)와 토큰 수(D)를 이용해 다음 근사식으로 구할 수 있다.

//

$$C \approx 6 \times N \times D$$

GPT-3의 경우 175B 파라미터 $\times$ 0.3T 토큰 $\times$ 6 $\approx$ 3.14×10^{23} FLOPs가 된다. 계수가 6인 이유는 파라미터당 MLP 층에서의 행렬 연산 횟수가 순전파(forward) 2회, 역전파(backward) 4회 등 총 6회이기 때문이다. 단, 이 근사는 Attention 기구의 연산량을 무시한 것이다. 시계열 길이가 짧은 경우에는 MLP 연산량이 Attention 연산량보다 압도적으로 크므로 무시해도 무방하지만, 최근 컨텍스트 길이가 길어지는 경향(GPT-3: 2,049 토큰, ChatGPT: 16,385 토큰, GPT-4: 32,768 토큰)을 고려하면 Attention 연산량을 무시할 수 없게 되었을 가능성이 높다. 보다 정확한 계산식은 karpathy/nanoGPT 저장소의 scaling_laws.ipynb 등을 참조할 수 있다.

이해를 돕기 위한 미니 퀴즈를 하나 생각해보자. GPU A100 1기의 연산 능력을 약 10^{14} FLOPS(대문자 S이므로 단위 시간당 연산량)라고 가정하자. 그렇다면 A100을 1000기 사용할 때, GPT-3 학습에는 어느 정도의 학습 시간이 필요한가? 이 문제에 답하려면 FLOPs(총 연산량)를 FLOPS(단위 시간당 연산량)로 나누어 시간을 구하면 된다.

모델 크기와 학습 곡선

컴퓨팅 자원(C)과 오차(L)의 관계를 더 자세히 보면, 서로 다른 모델 크기로 학습했을 때의 학습 곡선이 의미 있는 패턴을 보여준다. 모델 크기가 작으면 적은 컴퓨팅 자원으로도 빠른 속도로 Loss가 내려가지만, 일정 시점 이후로는 학습을 계속해도 Loss가 내려가기 어려워진다(포화, saturate). 반면 모델 크기가 크면 적은 컴퓨팅 자원으로는 Loss가 좀처럼 내려가지 않지만, 학습을 계속하면 Loss가 계속 내려가 포화되지 않고 최종적으로 더 좋은 성능에 도달한다.

이로부터 두 가지 통찰을 얻는다. 어떤 수준의 Loss(성능)를 달성하는 데 최적인 모델 크기는 정해져 있으며, 제한된 컴퓨팅 자원으로 최고의 성능을 발휘하는 모델 크기도 역시 정해져 있다. 즉, 주어진 컴퓨팅 자원량에 대해 최적의 성능을 발휘하는 모델 크기가 존재하고, 그 최적점들의 집합을 연결한 선이 바로 스케일 법칙이 기술하는 관계이다.

스케일 법칙의 역사와 다양한 도메인에서의 검증

스케일링은 새로운 현상이 아니다. 적어도 2017년 Baidu Research의 Hestness 등이 검증했으며, 이 연구에서는 기계 번역, 언어 모델링, 이미지 분류, 음성 인식 등 다수의 도메인에서 스케일 법칙의 발생을 확인했다. 특히 데이터에 관한 스케일 법칙을 중심으로 검증했다(모델에 대해서도 일부). 당시 연구와 Kaplan 등 이후의 연구를 비교하면 공통점은 데이터에 관한 스케일 법칙이 검증되었다는 점이고, 차이점은 대상 모델(Transformer 이전의 LSTM 등 RNN형 언어 모델)과 규모(특히 모델 규모)가 다르다는 점이다.

GPT-3에서도 스케일 법칙이 이용되었으며, 선행 연구(Kaplan 등 2020)보다 2 자릿수 오더 더 큰 연산량에서 스케일 법칙을 확인했다. 모델 구조 탐색 측면에서도 깊이(depth) 등 다양한 구조 요소에 대해 스케일 법칙이 검토되었다. 이후 Mixture of Experts(MoE) 모델에서도 일반 Transformer(점선)와 비교해(실선) 스케일 법칙이 검증되었고(MoE의 상세는 Day 5에서 다룬다), 이미지 생성, 멀티모달, 동영상, 수리 등 언어 이외의 도메인에서도 연산량에 관한 스케일 법칙이 성립함이 확인되었다. 어떤 연산량이 주어졌을 때 최적인 모델 크기는 도메인 간에 대체로 비슷한 경향을 보인다.

스케일 법칙의 활용 방법

투자 판단과 모델 구조 탐색

스케일 법칙의 가장 큰 가치는 성능을 예측 가능하게 만든다는 데 있다. GPT-4 기술 보고서에 따르면, GPT-4를 1.0으로 한 연산량(X축)과 성능(Y축)의 관계에서 1/1000 정도의 작은 모델까지로도 성능을 정확하게 예측할 수 있었다. GPT-4의 파라미터 수는 공개되지 않았지만, 아무리 작아도 10^{10} (10B)보다는 크고, 그래프의 최소가 10^3 이라고 하면 약 10^{13} (1T)까지 스케일 예측이 유효하다는 의미이다. 이는 대규모 모델에 대한 투자 위험을 줄여 준다("Scaling laws de-risk investments in large models"). 실제로 모델 구조 탐색이나 하이퍼파라미터 탐색 단계에서 작은 모델로 어느 구조가 우수한지(예: Transformer vs LSTM), 어느 쪽이 더 스케일에 유리한지(예: 파라미터가 작을 때는 층이 적은 쪽이, 클 때는 층이 많은 쪽이 유리)를 판단할 수 있다.

효율적인 모델 선택과 샘플 효율

파라미터 수가 많을수록 샘플 효율(sample efficiency)은 좋다. 작은 모델에서는 학습 도중부터 Loss가 내려가기 어려워지므로, 어떤 Loss를 달성하는 데 작은 모델로 연산을 계속하는 것은 비효율적이다.

Chinchilla: 주어진 연산량 하의 최적 배분

주어진 연산량 하에서 최적인 파라미터 수와 토큰 수를 찾는 연구가 DeepMind의 Chinchilla 논문이다. 연산량을 고정하고 파라미터 수와 토큰 수를 변동시키면(Chinchilla와 PaLM 2 모두에서 확인), 어느 연산량에서든 U 커브가 나타나며 최적인 값이 존재한다. U 커브가 되는 이유는 더 큰 파라미터 크기의 모델일수록 학습 초기의 Loss가 내려가기 어렵기 때문이다. 각 곡선마다 Training Loss가 최소가 되는 포인트를 찾고, 여러 연산량에 대해 같은 방법으로 최적인 파라미터 크기를 도출하면 FLOPs와 Parameter 사이의 최적 관계를 얻을 수 있으며, 이는 거의 직선 관계이다. 토큰 수에 대해서도 같은 방식으로 최적값을 구할 수 있다.

Chinchilla는 이 최적 연산 배분에 기반해 모델 크기는 70B(Gopher의 약 1/4배)로, 토큰 수는 1.4T(Gopher의 약 4.6배)로 설정했다. 그 결과 더 거대한 모델(Gopher 280B)을 제치고 많은 케이스에서 승리했으며, 이는 발견한 관계식의 타당성을 시사한다. 핵심 결론은 "최적 토큰 수 = $20 \times$ 파라미터 수"이다. PaLM 2에서도 같은 실험이 수행되어 Chinchilla와 마찬가지로의 스케일 법칙이 확인되었다.

Chinchilla 법칙을 넘어서: 추론 비용 고려

하지만 Chinchilla 법칙이 정말 최적인지에 대한 반론도 있다. Chinchilla의 모델 크기(70B)는 여전히 추론 비용이 높다. 학습만 보면 큰 모델이 같은 FLOPs로 더 높은 성능을 내지만, 추론 시에는 큰 모델이 더 많은 FLOPs를 필요로 한다. 따라서 학습과 추론 양쪽의 FLOPs를 모두 고려한 최적해를 도출하는 편이 낫다는 주장이 나왔다("Chinchilla Trap").

Harm de Vries는 최적 모델 크기의 40~60% 이내의 모델 크기를 선택하여 10~42%의 연산량 추가로 동일 성능의 모델을 학습할 수 있다고 지적했다. Sardana 등의 연구에서는 추론 시 토큰 수와 달성하고자 하는 학습 Loss를 가정했을 때, 라이프타임 전체의 총 FLOPs를 최소로 하는 최적의 파라미터 수 및 학습 토큰 수를 도출했는데, 추론 횟수가 많아질수록 라이프타임 전체로는 학습 토큰 수를 늘리는 쪽이 유리함을 보였다.

실제 모델들의 토큰 대 파라미터 비율(D/N)을 보면 이 경향이 반영되어 있다. Gopher(N=280B, D=0.3T)은 D/N이 1.07이었으나, Chinchilla(70B, 1.4T)는 20.0, Llama 2 7B(1.8T)는 285, Llama 3 70B(15T)는 214.2, Qwen 3 32B(36T)는 1125에 이른다. 최근 모델일수록 Chinchilla가 제시한 20을 크게 웃도는 토큰 수로 학습하는 추세이다.

예측 가능한 개선과 예측 불가능한 개선

스케일 법칙으로 예측 가능한 성질에는 성능의 개략 산정, 일반적인 문장의 다음 단어 예측 정확도, 번역이나 QA 태스크에서의 평균적인 점수 개선 등이 있다. 반면 예측 불가능한 성질도 존재한다.

대표적인 예가 창발 능력(Emergent Ability)이다. 모델 크기를 거대하게 하면 특정 태스크에서 성능이 "갑자기" 대폭 올라가는 현상이 관찰된다. 하지만 이것이 정말 "창발"이나 "상전이"인지에 대해서는 반론도 있다. 성능의 측정 방법에 의존할 수 있고(이것은 본 논문에서도 지적됨), 가로축이 로그인 것도 이상하며, 애초에 무엇을 창발이라 할 것인지도 논쟁거리이다. 다만 거대 모델과 거대 연산으로 생각보다 잘하게 되는 것은 사실이다.

또 다른 예로 Grokking이 있다. 이는 학습을 계속하면 갑자기 검증 데이터에서의 정답률이 높아지는 현상으로, 학습 데이터에서의 정답률은 그 이전에 이미 높은, 즉 과적합 후에도 학습을 계속하면 발생하는 현상이다. 연구에 따르면 과학습 중에는 단순히 기억만 하던 것이, 일반화 단계에 이르면 내부 표현이 깔끔하게 정렬되는 식으로 기억을 일반화하고 있다.

다운스트림 태스크와 스케일 법칙

Loss(사전학습의 교차 엔트로피)가 낮다는 것이 반드시 다운스트림 태스크의 성능이 높은 것으로 직결되는가? 기본적으로는 YES이지만 예외도 자주 있다. 성능은 깔끔하게 상승하는 경우도 있고, 갑자기 상승하는 경우(Emergent Ability), 오르지 않는 경우, 내려갔다가 오르는 경우(Inverse scaling prize) 등 태스크의 종류와 난이도에 따라 다양한 패턴이 나타난다.

다운스트림 태스크 데이터가 학습 데이터와 분포가 정렬되어 있는 경우에는 사전학습 데이터량과 다운스트림 태스크 평가값 사이에 스케일 법칙이 성립한다. 기계 번역 태스크로 검증한 결과, MC4(Multilingual C4) 같은 사전학습 데이터와 다운스트림 태스크 데이터의 임베딩 공간에서의 분포 거리가 가까울수록 스케일 법칙이 잘 들어맞는다. 분포 외 데이터(WebText2 이외)에서는 성능 열화가 보이지만 오프셋의 차이 정도이며 기울기는 거의 같다.

스케일 법칙 활용의 정리

예측 가능한 성능 개선에 의해 다음과 같은 물음에 답할 수 있다. 투자의 판단(더 많이 컴퓨터에 투자할 것인가), 효율적인 모델 선택(파라미터를 늘렸을 때 어느 쪽이 좋은 모델인가), 효율적인 컴퓨팅 자원 이용(토큰과 파라미터 중 어느 쪽을 늘려야 하는가; Chinchilla Optimal에서는 최적 토큰 수 = $20 \times$ 파라미터 수이며, 추론 비용을 고려하면 계수가 변화한다). 단, 다운스트림 태스크에 스케일 법칙이 반드시 성립한다고는 한정하지 않는다.

스케일 법칙의 구하는 방법

기본 접근: 작은 실험에서 피팅하기

스케일 법칙을 구하는 기본 방법은 비교적 작은 몇 가지 조건으로 실험을 수행하고 결과를 피팅(fitting)하는 것이다. GPT-4 기술 보고서와 Hoffmann 등의 연구가 이 방식을 따랐다. 이때 두 가지 핵심적인 질문이 생긴다. 모델 크기를 어떻게 변화시킬 것인가? 학습률 등의 하이퍼파라미터는 어떻게 설정할 것인가?

모델 크기를 변화시키는 방법

모델 크기를 바꾸는 수단으로는 층 수를 늘리거나, 임베딩 차원을 올리거나, FFN의 중간층 차원을 크게 하거나, 어텐션 헤드 수를 늘리는 등 여러 가지가 있다. Kaplan 등의 원 논문에서는 파라미터 수를 고정했을 때 신경망의 여러 요소를 조정하여 검토한 결과, 큰 영향은 없다는 결론을 내렸다. 예컨대 종횡비(aspect ratio, 임베딩 크기 / 층 수)는 큰 영향을 미치지 않았다.

실제 사례를 보면, Llama 3는 중형비가 128, 102.4, 130 등으로 모델 차원 대 FFN 차원 비율은 3.5로 일정하며, 헤드 수도 모델 차원에 대해 비례해서 스케일한다. Cerebras GPT 역시 중형비가 76.8, 77.7, 85.3 등으로 모델 차원 대 FFN 차원 비율을 4.0으로 유지한다(헤드 수는 다소 불규칙하게 변화). 즉, 대체로 고정된 계수를 유지하면서 스케일하는 것이 일반적이다.

하이퍼파라미터 변화와 μ Transfer

모델 크기를 스케일시킬 때 학습률과 스케줄링을 어떻게 설정할 것인가도 중요한 문제이다. 통상적인 초기화의 경우, 폭(width)을 변화시키면 최적의 학습률이 달라진다(다만 어느 정도 경향은 있다). 경험적으로는 모델 크기를 크게 했을 때 학습률은 점차 작게, 배치 크기는 크게 하는 것이 좋은 경향이 있다. Cerebras GPT 등 실제 모델들도 모델 파라미터가 클수록 학습률을 작게, 배치 크기를 크게 설정하는 경향을 보인다.

Yang 등이 제안한 μ Transfer(μ P 전이)는 이 문제를 우아하게 해결하는 방법이다. 가중치 초기화 방법과 weight별 학습률 설정 방법을 변경함으로써, 작은 모델에서 찾은 최적의 학습률을 큰 모델에 제로샷으로 전이할 수 있다. μ Transfer를 사용하면 모델 크기가 달라도 거의 같은 정도의 학습률 값으로 최적의 Loss를 달성할 수 있다. Cerebras GPT는 μ Transfer를 실제로 적용한 사례이다.

스케일 법칙의 상세에 더 관심 있는 독자는 Hashimoto와 Liang의 Stanford CS336 강의("Language Modeling from Scratch", 2024)를 참조하기를 권한다.

추론 시의 스케일링

동기: 추론 부하의 차이

"바나나의 색은 무엇입니까?"와 "스케일 법칙의 문제는 무엇이라고 생각합니까?"는 필요한 사고의 과정이 분명히 다르다. 후자는 추론 시에 더 큰 부하가 걸린다. 이러한 구조를 LLM에서 어떻게 실현할 수 있는가, 또 효과적인가라는 질문이 추론 시 스케일링의 출발점이다. OpenAI가 발표한 o1 모델도 테스트 시의 추론을 스케일링시킴으로써 성능 향상을 보고했다.

추론 시에 연산량을 스케일시키는 방법은 크게 프롬프팅(prompting), 디코딩(decoding), 메타 제너레이션(meta-generation) 세 가지 수준으로 나눌 수 있다.

프롬프팅과 디코딩

프롬프팅 수준에서는 Chain-of-Thought Prompting과 Many-Shot In-Context Learning(ICL)이 대표적이다. 프롬프팅을 통해 추론 시의 토큰 수를 늘림으로써 추론 시의 연산량을 스케일시키는 시도이다.

디코딩 수준에서는 사전학습된 LLM이 텍스트를 출력(디코드)하는 방식을 다양화한다. 사전학습된 LLM을 사용하여 텍스트를 출력(디코드)하는 데는 다양한 방식이 존재하며, 기본적인 방식으로 Greedy Decoding, Beam Search, Random Sampling, Top-K / Top-P Sampling 등이 있고, 그 외에도 다수의 방식이 제안되어 있다. 발전적인 예로 Contrastive Decoding이 있는데, 이는 전문가 모델과 아마추어 모델(통상 더 적은 파라미터 수)을 이용해 확률밀도비를 취하고 거기서부터 샘플링을 수행하여, 전문가 모델의 출력을 강조하고 아마추어 모델의 출력을 감소시키도록 생성하는 방식이다. 외부 모델을 사용하는 이 방식은 생성 품질을 높이는 효과적인 수단이다.

메타 제너레이션: 추론 시 연산의 대규모 스케일

메타 제너레이션(Meta-Generation)은 토큰 수준의 디코딩을 넘어 문장이나 단락 단위로 생성 과정을 평가하고 생성 프로세스 전체를 최적화하는 개념이다. Snell 등은 이를 세 가지 범주로 정리했다. 어느 쪽이든 추론 시에 연산량이 크게 스케일된다.

첫째, 병렬 탐색(Parallel Search)은 병렬로 여러 후보를 생성하여 스코어링이나 다수결 등으로 생성물을 선택하는 방식이다. Best-of-N은 N개의 답변을 내어 스코어가 가장 높은 것을 선택하며, 스코어 함수는 태스크에 따라 LLM 스코어, 학습한 평가기, BLEU 등 특정 지표 등 임의로 선택할 수 있다. Self-Consistency는 LM에 복수의 추론을 수행시켜 다수결(Majority Voting)로 정답을 고른다. MBR Decoding(Minimum Bayes-Risk Decoding)은 기계 번역에서 쓰이는 방식으로 효용 함수(BLEU, METEOR, BLEURT, COMET 등)를 이용해 출력의 품질을 최대화하도록 디코딩한다. 이 외에도 집계 방식(aggregation type)과 스코어링 기법에 따라 다양한 알고리즘이 존재한다.

둘째, 단계별 탐색(Step Level Search)은 중간 결과를 단계별로 평가하여 생성물을 선택하는 방식이다. 토큰 수준의 Beam Search와 달리 문장이나 단락 단위로 샘플링과 평가를 수행하며, 평가에는 Process Reward Model(PRM)을 이용하고 Top-N Sampling으로 중간 결과를 선택한다. Tree-of-Thought(ToT)는 복수의 사고열을 단번에 출력하여 평가하는 Self-Consistency와 달리 도중에 분기를 시키는 트리 탐색 방식이며, 노드의 평가도 LM으로 수행한다. Game of 24(주어진 4개의 숫자를 사칙연산하여 24를 만들기)와 같이 전략적 사고가 필요한 태스크에서 성능이 대폭 개선되었다. 탐색 방법이나 검증하는 스텝의 차이 등에 의해 다양한 알고리즘이 존재한다.

셋째, 정제(Refinement)는 한 번 생성한 결과나 그 결과에 대한 피드백을 바탕으로 재생성하는 방식이다. Self-Refine은 자기 자신을 사용하여 피드백을 생성하고 출력을 개선하며, 7개의 태스크에서 최대 50% 가까운 정확도 향상을 보였다. 외부나 내부의 피드백 결과를 이용하여 반복적으로 생성 결과를 갱신한다.

추론 시 스케일링의 효과

Lightman 등의 연구는 평가 방식의 차이가 성능에 미치는 영향을 보여준다. Outcome-supervised Reward Model(ORM)을 사용하여 출력 전체를 평가하는 Best-of-N(ORM)은 다수결로 선택하는 Majority Voting보다 우수하며, Process-supervised Reward Model(PRM)을 이용해 도중의 가정을 평가하는 Best-of-N(PRM)은 더욱 우수하다.

Snell 등은 같은 컴퓨팅 자원일 때 파라미터를 늘리는 것보다 추론 시 연산을 스케일하는 것이 더 효과적일 수 있음을 보였다. 랜덤 샘플링에 의한 추론 횟수를 늘리고 PRM을 이용해 추론 경로와 최종 답변을 적절히 선택함으로써 성능이 향상된다.

정리

이 장에서는 언어 모델의 스케일 법칙을 중심으로 다음을 살펴보았다.

첫째, 스케일 법칙이 무엇인지를 설명했다. 스케일 법칙은 컴퓨팅 자원(C), 데이터셋 크기(D), 파라미터 수(N)와 오차(L) 사이에 성립하는 경험칙으로, 로그 그래프 상에서 거의 직선 관계가 성립한다.

둘째, 스케일 법칙의 구체적인 구하는 방법을 설명했다. 스케일 법칙을 구하기 위해서는 기본적으로 몇 가지 다른 설정에서 실험을 수행하고 피팅한다. 하이퍼파라미터 설정(종횡비, 학습률, 배치 크기 등)에 대해서는 다양한 지견이 논문으로 발표되어 있으며, μ Transfer 같은 전이 방법도 존재한다.

셋째, 추론 시의 스케일링을 설명했다. 학습 시뿐만 아니라 추론 시에도 연산량을 스케일시킴으로써 성능을 개선할 수 있으며, 추론 시의 방법은 프롬프팅, 디코딩, 메타 제너레이션의 세 수준으로 정리된다.

참고문헌

- [3] Kaplan+. Scaling Laws for Neural Language Models. 2020. arXiv:2001.08361
- [4] Wei+. Emergent Abilities of Large Language Models. 2022. arXiv:2206.07682
- [5] Schaeffer+. Are Emergent Abilities of Large Language Models a Mirage?. 2023. arXiv:2304.15004
- [6] Power+. Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets. 2022. arXiv:2201.02177
- [7] Liu+. Towards Understanding Grokking: An Effective Theory of Representation Learning. 2022. NeurIPS2022
- [8] Hestness+. Deep Learning Scaling is Predictable, Empirically. 2017. arXiv:1712.00409
- [9] Brown+. Language Models are Few-Shot Learners. 2020. NeurIPS2020

-
- [10] Anil+. PaLM 2 Technical Report. 2023. arXiv:2305.10403
 - [11] Henighan+. Scaling Laws for Autoregressive Generative Modeling. 2020
 - [12] Ganguli+. Predictability and Surprise in Large Generative Models. 2023. arXiv:2202.07785
 - [13] OpenAI. GPT-4 Technical Report. 2023. arXiv:2303.08774
 - [42] Hoffmann+. Training Compute-Optimal Large Language Models. 2022. NeurIPS2022
 - [43] de Vries. Go smol or go home, Why we should train smaller LLMs on more tokens. 2023
 - [47] Bahdanau. The FLOPs Calculus of Language Model Training. Medium. 2022
 - [50] Sardana+. Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws. 2024. ICML2024
 - [51] Dey+. Cerebras-GPT: Open Compute-Optimal Language Models Trained on the Cerebras Wafer-Scale Cluster. 2023. arXiv:2304.03208
 - [52] Yang+. Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer. 2022. NeurIPS2022
 - [53] OpenAI. Learning to reason with LLMs. OpenAI Blog. 2024
 - [54] Madaan+. From Decoding to Meta-Generation: Inference-time Algorithms for Large Language Models. 2024. arXiv:2406.16794
 - [55] Li+. Contrastive Decoding: Open-ended Text Generation as Optimization. 2023. ACL2023
 - [56] Snell+. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. 2024. arXiv:2408.03314
 - [57] Eikema+. On the True Distribution Approximation of Minimum Bayes-Risk Decoding. 2020. EMNLP2020
 - [58] Wang+. Self-Consistency Improves Chain of Thought Reasoning in Language Models. 2023. ICLR2023
 - [59] Lightman+. Let's Verify Step by Step. 2023. ICLR2024
 - [60] Yao+. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. 2023. NeurIPS2023
 - [61] Madaan+. Self-Refine: Iterative Refinement with Self-Feedback. 2023
 - [62] Tatsunori Hashimoto, Percy Liang. CS336: Language Modeling from Scratch. Stanford University. 2024
 - [66] AI@Meta+. The Llama 3 Herd of Models. 2024

-
- [67] Wei+. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. 2022
 - [68] Agarwal+. Many-Shot In-Context Learning. 2024
-

라이선스: CC BY-NC-ND 4.0 (저작자표시-비영리-변경금지 4.0 국제) 원저작권자: 도쿄대학교 마
츨오·이와사와 연구실

05

Day 5 — 발전적인 사전학습

이 장의 구성

- 01 이번 장의 목표
 - 02 사전학습의 중요도와 스케일 법칙
 - 03 스케일화의 세 가지 과제
 - 04 파라미터(N) 효율화: Efficient Attention과 혼합 전문가
 - 05 계산량(C) 효율화: 병렬화와 양자화
 - 06 데이터(D) 효율화: 품질, 양, 그리고 한계
 - 07 정리
 - 08 참고문헌
-

대규모 언어 모델 강좌 2025 · 2025/10/29 야마다 이쿠야(山田育矢) — Studio Ousia 최고 과학자, 나고야대학 수리·데이터 과학·인공지능 교육연구센터 객원 교수, 도호쿠대학 언어 AI 연구센터 특임 교수(객원). 다양한 언어 모델의 개발과 국제 콘테스트(NeurIPS EfficientQA 2020 2위, ISWC Challenge 2020 1위, NIPS HCQA 2017 1위 등) 수상 경력이 있으며, 저서로 『대규모 언어 모델 입문』 시리즈와 『딥러닝에 의한 자연어 처리』가 있다.

본 자료는 도쿄대학교 마쓰오·이와사와 연구실이 작성한 강좌 강의 자료이며, 크리에이티브 커먼즈 CC BY-NC-ND 4.0(저작자표시-비영리-변경금지 4.0 국제) 라이선스로 제공됩니다.

이번 강의 목표

본 강은 제3회 「사전학습」의 속편으로, 언어 모델을 스케일(대규모화)하여 사전학습하는 과정에서 등장하는 발전적인 화제를 다룬다. 사전학습은 LLM 개발 파이프라인의 첫 단계에 자리 잡고 있으며, 이후의 사후학습(post-training) 전체의 품질을 좌우하는 토대가 된다.

학습 목표는 세 가지다. 첫째, 모델을 스케일하여 사전학습할 때 발생하는 과제를 설명할 수 있을 것. 둘째, 그 과제에 대응하는 발전적인 방법들을 설명할 수 있을 것. 셋째, 사전학습의 일련의 흐름을 모델 스케일 기술까지 포함하여 코드로 구현할 수 있을 것. 강의에서는 각 요소의 스케일에 따른 문제와 해결 기술을 살피고, 연습에서는 PyTorch로 트랜스포머 모델을 사전학습하는 일련의 흐름(데이터 준비, 전처리, 스케일 기술을 적용한 학습)을 구현한다.

사전학습의 중요도와 스케일 법칙

사전학습이 LLM 개발에서 어느 정도의 비중을 차지하는가. 한 가지 참고가 되는 것은 GPT-4의 각 개발 태스크에 투입된 인원 비율이다. 이를 OpenAI가 생각하는 LLM 개발에서의 중요도 비율로 해석해 보면, 사전학습 관련 태스크가 전체에서 상당한 몫을 차지함을 볼 수 있다. 그러면서도 "사전학습을 직접 수행할 필요가 있을까?"라는 질문은 여전히 유효하다. 대부분의 실무에서는 공개된 사전학습 모델을 출발점으로 삼지만, 도메인 특화 모델이나 최첨단 모델을 직접 만들려면 결국 사전학습 단계까지 직접 설계해야 하기 때문이다.

이런 투자가 정당화되는 근거가 스케일 법칙(scaling law)이다. 스케일 법칙은 언어 모델을 대규모화함으로써 성능이 향상되는 관계를 가리키는 경험칙으로, 성능(L)이 컴퓨팅 자원(C), 데이터셋 크기(D), 파라미터 수(N)의 세 요소와 맺는 관계로 정식화된다. 이 법칙은 다양한 도메인에서 대규모 모델을 개발할 장점이 확인되었고, 그 결과 대규모 모델에 대한 투자 리스크가 크게 경감되었다. 성능 예측이 가능해졌기 때문에, 자원을 들이면 그에 상응하는 성능 향상을 기대할 수 있게 된 것이다.

스케일화의 세 가지 과제

파라미터, 계산량, 데이터를 스케일하면 스케일 법칙에 따라 성능이 올라가는 것은 사실이지만, 스케일화 과정에는 세 가지 구조적 과제가 뒤따른다.

첫째, 계산량(C)의 확보다. 모델이 스케일됨에 따라 충분한 계산량과 메모리 용량을 갖추고 효율적으로 훈련할 필요가 있다. 모델 크기가 커지면 GPU 자원 요구량이 가파르게 증가하며, 단순히 자원을 늘리는 것만으로는 비용이 감당할 수 없는 수준에 이른다. 효율적으로 대규모 모델을 훈련할 수 있다면 그만큼 비용을 줄일 수 있다.

둘째, 파라미터 수(N)의 비용이다. 모델이 스케일됨에 따라 증가하는 비용을 억제해야 한다. 여기서 특히 치명적인 것이 트랜스포머의 self-attention 구조다. self-attention에서는 시퀀스 길이 n 의 제곱에 비례하는 계산량과 메모리가 필요하다. 각 토큰이 다른 모든 토큰과의 연관성을 계산하기 때문에, 모든 토큰의 조합에 대해 계산을 수행하고 그 값을 기억해야 한다. 시퀀스가 길어질수록 이 비용은 제곱으로 불어난다.

셋째, 데이터(D)의 한계다. 성능을 발휘하기 위한 학습용 데이터를 준비해야 하는데, 양질의 언어 데이터는 사실상 2024년경에 고갈될 것으로 예측된다. 과거 웹 데이터의 증가 추세와 학습 데이터의 증가 추세를 외삽한 연구(Villalobos et al., 2022)가 이 예측의 근거가 되며, 이는 스케일 법칙에 기대어 모델을 키우는 전략 자체에 장기적 제약이 있음을 의미한다.

이 세 가지 과제에 대응하기 위해 본 장에서는 파라미터(N), 계산량(C), 데이터(D) 각각에 대한 접근을 차례로 살핀다.

파라미터(N) 효율화: Efficient Attention과 혼합 전문가

파라미터 수와 관련된 과제에 대응하는 방향은 두 갈래다. 하나는 self-attention 자체의 계산·메모리 효율을 개선하는 것이고, 다른 하나는 계산 비용을 팽창시키지 않으면서 모델의 파라미터를 늘리는 것이다. 전자를 Efficient Attention, 후자를 혼합 전문가(Mixture of Experts, MoE)라 부른다.

Efficient Attention의 계보

Sparse Transformer(Child et al., 2019)는 attention을 계산하는 위치를 한정함으로써 계산량을 삭감한다. 계산하지 않는 위치는 마스크 처리하여 희소(sparsely)하게 attention을 수행하는데, 어텐션 기구를 두 번 통과시키면 모든 토큰에 어텐션이 도달하도록 설계되어 있다. 이를 통해 이미지나 음성처럼 매우 긴 시퀀스 길이의 입력에 대해서도 트랜스포머를 효율적으로 이용할 수 있게 된다.

Big Bird(Zaheer et al., 2020)는 다수의 희소한 attention을 조합하여 attention을 근사함으로써 긴 시퀀스에 대응한다. 긴 시퀀스를 다루는 질의응답 및 요약 등의 태스크에서 좋은 성능을 획득했으며, 유사한 아이디어로 Longformer(The Long-Document Transformer, 2020)가 있다.

FlashAttention(Dao et al., 2022)은 방향을 달리한다. attention의 계산은 연산 자체가 아니라 메모리 I/O에 병목이 있다는 점을 지적한 것이다. 입력 행렬을 잘게 분할하여 계산함으로써, 시퀀스 길이 $\times$ 시퀀스 길이의 attention 행렬 전체에 대한 메모리 읽기/쓰기를 회피한다. 처리를 가능한 한 GPU SRAM 안에서 완결시켜 저속인 GPU HBM 메모리로의 액세스 횟수를 줄이고, fused kernel 구현의 최적화를 통해 대폭적인 속도 향상을 달성했다. 예컨대 GPT-2에서는 최대 7.6배의 가속을 보고한다.

FlashAttention-2(Tri Dao, 2023)는 여기에 세 가지 구현 공법을 더해 고속화한다. 첫째, 알고리즘을 공략하여 행렬 연산 이외의 연산을 될 수 있는 대로 삭감한다. GPU는 행렬 연산에 전용 연산 유닛이 있어 고속히 처리할 수 있기 때문이다. 둘째, 배치나 attention의 헤드뿐만 아니라 시퀀스 방향으로도 병렬 연산을 수행하여, 배치나 헤드 수가 적은 경우에도 고속화할 수 있도록 한다. 셋째, 워프(동시에 실행되는 스레드 그룹을 가리키는 GPU 용어)를 query 행렬에서 분할함으로써 워프 간 동기 및 통신을 줄이고 병렬성을 향상시킨다. 그 결과, FlashAttention 대비 약 2배, PyTorch 표준 attention 대비 최대 9배의 고속화를 달성했다.

혼합 전문가(Mixture of Experts)

MoE는 계산 비용을 팽창시키지 않고 모델 파라미터를 늘리는 접근이다. 다수의 전문가(각각이 신경망)를 준비해 두고, 입력 값에 따라 일부 전문가에만 포워드한다. 모든 파라미터를 사용하는 것은 아니므로 계산량을 억제할 수 있다. 엄밀히 말하면 어느 전문가에게 할당할지를 결정하기 위한 작은 신경망(라우터 신경망)이 추가로 필요하므로 그 만큼 약간의 계산량은 증가하지만, 전체적으로는 파라미터 대비 계산량을 크게 줄일 수 있다.

실험적으로 계산량을 억제하면서 퍼포먼스를 개선할 수 있음이 확인되었다. 동일한 계산량으로 학습한다는 제약 하에서, MoE를 사용한 모델이 사용하지 않은 통상 모델보다 퍼포먼스가 높다. 유출 정보에 따르면 GPT-4는 MoE 모델 구조를 채택하고 있다고 하며, 최근 DeepSeek, Qwen 등 다수의 오픈 모델도 MoE를 채택하고 있다. 복잡한 분류 문제에 MoE를 적용하면, 라우터 신경망이 데이터의 클러스터 중심점을 기준으로 각 사례를 전문가에 할당하고, 각 전문가는 그 클러스터 내에서의 분류에 특화하는 학습이 수행됨이 시각화(t-SNE)를 통해 확인되었다.

Switch Transformer(Fedus et al., 2021)는 T5 모델의 피드포워드 층에 MoE를 적용하여 대규모화한 1조 6000억 파라미터 모델이다. 다수의 MoE에서는 각 토큰마다 복수의 전문가가 사용되지만, 이를 하나의 전문가만 사용하도록 단순화하여 통신·계산 비용의 삭감을 실현했다. 그 결과 1.6조 파라미터 모델의 학습에서 T5-XXL 모델에 대해 4배의 사전학습 스피드업을 달성했다.

DeepSpeed-MoE(Rajbhandari et al., 2022)는 최적화된 구현을 통해 MoE 모델의 학습 효율을 개선한다. 자기회귀 모델에서 품질이 동등한 Dense 모델과 비교하여 약 5배의 학습 비용 삭감을 실현했다. 같은 연구에서 제안된 PR-MoE는 모델 크기를 줄이면서 성능을 유지하는 아키텍처다. 각 토큰이 1개의 고정

된 MLP와 1개의 전문가 양쪽을 이용하고, 트랜스포머의 후반 층에서 보다 많은 전문가를 활용하도록 설계되었다. 그 결과 표준적인 MoE보다 적은 파라미터 수로 동등한 성능을 달성했다(350M 모델에서는 1/3 이하의 파라미터로, 1.3B 모델에서는 표준 MoE의 약 60% 파라미터로 동등 성능).

MoE 모델에도 스케일 법칙은 성립한다(Clark et al., 2022). 전문가 수를 늘리면 로그 손실이 내려가지만, 특히 큰 모델 크기에서는 너무 많이 늘리면 효과가 약해진다. MoE 모델의 파라미터 수를 여러 요소를 가미하여 통상 모델의 파라미터 수로 환산하면 스케일 법칙이 성립한다. 다만 통상 모델의 파라미터 수가 커지면 MoE화의 효과는 비례하여 낮아지므로, 모델 크기에 적합한 전문가 수를 선택하는 것이 바람직하다.

계산량(C) 효율화: 병렬화와 양자화

계산량의 과제에는 두 방향이 있다. 훈련 시에는 복수의 GPU를 효율적으로 활용하는 병렬 계산이 필요하고, 추론 시(주로)에는 모델의 경량화를 통해 소규모 GPU 환경에서의 운용을 가능하게 하는 양자화가 필요하다.

병렬 계산

딥러닝의 병렬화는 여러 축으로 전개된다. 데이터 병렬, 파이프라인 병렬, 텐서 병렬이 그 축들이며, 전문가 병렬은 MoE 모델에 특화된 추가 축이다.

ZeRO(ZeRO: Memory Optimizations Toward Training Trillion Parameter Models, Rajbhandari et al., 2019)는 데이터 병렬 시의 메모리 효율화 기법이다. 어느 요소를 메모리에서 병렬화하는가에 따라 세 단계의 동작 모드를 갖는다. Stage 1, Stage 2, Stage 3로 단계가 진행될수록 메모리를 삭감할 수 있지만, 통신 오버헤드가 증가한다. 대표적인 라이브러리인 DeepSpeed를 통해 환경 설정(config)을 기술하는 것으로 이용할 수 있다.

3D 병렬화는 병렬화 전략마다 통신 오버헤드가 다르다는 점을 적극 활용한다. 통신 오버헤드는 텐서 병렬이 파이프라인 병렬보다 훨씬 크다(텐서 병렬 ≫ 파이프라인 병렬). 이 특성을 이용해 GPU·노드의 배치에 따라 통신 비용을 억제하며 병렬화를 구성한다. 예를 들어 4개의 GPU를 가진 8노드로 3D 병렬화를 구성할 때, 고 오버헤드의 텐서 병렬은 노드 내에 배치하고, 저 오버헤드의 파이프라인 병렬은 노드를 가로질러 배치하며, 데이터 병렬과 ZeRO Stage 1의 조합으로 GPU 메모리 효율을 높인다.

전문가 병렬화(expert parallelism)는 MoE 모델 전용의 병렬화 기법이다. MoE의 각 전문가를 다른 GPU에 배치한다. 행렬을 분할하여 복수의 GPU가 보유하는 텐서 병렬과 유사하지만, 모든 층에 적용되는 텐서 병렬화와 달리 전문가 층에만 적용된다는 차이가 있다.

양자화

양자화(quantization)는 모델 파라미터의 데이터 타입을 부동소수점(Float 형)에서 정수(Int 형)로 변환하여 연산 처리를 수행하는 기법이다. 추론 시 필요 메모리량을 삭감할 수 있지만, 단순히 이를 수행하면 성능 저하가 발생한다. 그래서 정밀도 저하를 억제하는 다양한 공법이 연구되었다.

LLM.int8()(Dettmers et al., 2022)는 성능 저하 없이 가능한 양자화 방법이다. 16비트 행렬 곱셈에서 이상치의 특징을 분리하는 혼합 정밀도 분해(mixed-precision decomposition)를 수행하여, 대부분의 값을 8비트로, 이상치만을 16비트로 표현한다. 구체적으로는 세 단계를 거친다. 첫째, 입력된 은닉 상태에서부터 열 단위로 이상치(임계값보다 큰 값)를 추출한다. 둘째, 이상치 행렬에 대해서는 FP16인 채로 행렬 연산을 수행하고, 이상치가 아닌 행렬에 대해서는 INT8로 변환(양자화)하여 행렬 연산을 수행한다. 셋째, INT8의 출력 값을 FP16으로 되돌려 두 개의 출력 값을 가산하여 FP16으로 출력 값을 반환한다. 그 결과 16비트 대비 약 50%의 메모리 삭감이 가능하며, 175B까지의 파라미터를 가지는 LLM에서 성능 저하 없이 추론을 수행할 수 있음을 경험적으로 보였다.

k-bit 스케일 법칙(Dettmers et al., 2023)은 모델의 메모리 용량(비트 수)을 고정했을 때 모델의 크기와 양자화를 어떻게 설정해야 하는가를 묻는다. 예를 들어 30B의 8-bit 모델과 60B의 4-bit 모델은 동일한 메모리 용량이 된다. 메모리 용량을 고정한 경우 4-bit 양자화가 가장 제로샷 성능이 높았고, 3-bit에서는 모델 크기가 커지면 성능이 불안정해진다. 즉 post-hoc 양자화에서는 4-bit가 사실상의 최소 필요 조건으로 보인다.

양자화가 LLM의 중요한 특성인 Emergent Ability를 상실시키지 않는가 하는 우려도 검증되었다(Liu et al., 2023). in-context learning, chain-of-thought reasoning, instruction-following의 세 가지 능력을 측정한 결과, 4비트까지의 양자화 모델에서는 Emergent Ability의 유지가 확인되었다.

BitNet(Wang et al., 2023; Ma et al., 2025)는 훈련 시부터 1bit/1.58bit 양자화를 수행하는 접근이다. 양자화를 적용한 모델을 사전학습하여 구축하며, attention과 MLP에 포함되는 선형 층을 모두 1bit용으로 확장된 선형 층(BitLinear)로 치환한다. 파라미터를 2치($\{-1, +1\}$; 1bit) 또는 3치($\{-1, 0, +1\}$; 1.58bit)로 나타내며, 동일한 비트 수(메모리 용량)로 비교했을 때 기존 LLM의 성능을 크게 능가했음을 보고했다.

데이터(D) 효율화: 품질, 양, 그리고 한계

데이터의 과제는 다시 두 갈래로 나뉜다. 하나는 성능을 발휘하기 위한 학습용 데이터를 어떻게 준비할 것인가(데이터셋 정비)이고, 다른 하나는 데이터셋의 품질을 어떻게 개선할 것인가(데이터 전처리)이다.

학습 데이터의 구성과 도메인 특화

어떤 학습 데이터로 학습해야 하는가라는 물음에 대해, 주요 모델의 학습 데이터 구성이 참고가 된다. 한 가지 뚜렷한 경향은 최근 모델이 많은 케이스에서 코드(code) 학습을 수행하고 있다는 점이다. GPT-3는 코드 학습을 포함하지 않았으나, 코드로 학습한 모델(예: code-davinci-002)은 GPT-3보다 추론 성능이 좋다. ChatGPT도 code-davinci-002를 베이스로 학습되어 있다고 간주된다.

특정 도메인 데이터에 의한 지속적 사전학습(continual pre-training)도 효과적이다(Cossu et al., 2022). 사전학습 후에 특정 도메인의 문서(예: arXiv 논문 요지)를 지속적으로 학습시키는 방식이다. 사전학습 후에 지속 학습함으로써 치명적 망각(catastrophic forgetting)이 일어나기 어려운 데다가, 다운스트림 태스크에서 뛰어난 성능을 발휘할 수 있음이 확인되었다.

최적의 자원 할당: Chinchilla와 그 너머

LLM의 사전학습 예산은 계산량(GPU 수나 시간)에 비례한다. 따라서 주어진 계산량을 파라미터 수(N)와 학습 데이터량(D)에 어떻게 할당하는가가 중요해진다. OpenAI의 종래 스케일 법칙(Kaplan et al., 2020)은 파라미터에 대해 필요로 하는 학습 데이터량의 견적이 너무 적다는 점이 지적되었고, 이에 대응한 것이 Chinchilla다(Hoffmann et al., 2022). Chinchilla는 모델 크기를 70B(Gopher의 약 1/4)로 줄이는 대신 데이터 크기를 1.4T 토큰(Gopher의 약 4.6배)까지 늘려, 다수의 케이스에서 Gopher에 승리하며 제안한 관계식의 타당성을 시사했다.

다만 Chinchilla 법칙에도 비판이 있다. "Chinchilla Trap"이라 불리는 지적은 Chinchilla의 최적 모델 크기(70B)가 여전히 커서 추론 비용이 높다는 점을 지적하며, 추론 비용까지 고려하면 더 작은 모델을 대규모 데이터로 훈련해야 한다는 의견이다(de Vries, 2023). 분석에 따르면 Chinchilla 최적인 모델 크기의 40~60% 이내의 모델 크기로, 10~42%의 계산량 추가만으로 동일 성능의 모델을 학습할 수 있다고 한다.

언어 데이터 고갈과 지식 습득의 메커니즘

한편 언어 데이터의 고갈 문제는 여전히 현실적 과제다. 앞서 인용한 Villalobos et al.(2022)의 예측에 따르면 양질의 언어 데이터의 고갈이 예측되고 있으며, 이는 스케일 법칙 기반의 성장 전략에 장기적 제약이 될 수 있다.

이 맥락에서 사전학습을 통해 지식이 어떻게 학습되는가를 조사한 연구(Chang et al., 2024)가 시사하는 바가 크다. LLM 사전학습 시에 지식이 어떻게 획득되어 가는지를 추적한 결과, 지식을 서술한 문장이 출현할 때마다 올바른 지식이 생성되는 확률이 높아지며 지식이 점차 학습되어 간다. 반면 지식이 출현하지 않는 스텝(예: 900스텝 이후)에서는 망각되어 간다. 이는 LLM에게 지식을 가르치기 위해서는 훈련 데이터 중에 반복해서 지식이 출현할 필요가 있음을 의미하며, 중요한 지식이 높은 밀도로 포함되는 고품질 훈련 데이터의 중요성을 시사한다.

합성 데이터와 유한 데이터 설정

데이터 고갈에 대한 하나의 대응이 합성 데이터(synthetic data)다. Kang et al.(2025)는 합성 데이터를 사용한 사전학습의 효과를 1,000개 이상의 LLM을 10만 GPU 시간을 사용하여 훈련해 검증했다. 검증한 합성 데이터의 종류는 두 가지다. 하나는 웹 바꾸어말하기(paraphrasing)로, LLM을 사용하여 웹 데이터를 깨끗한 텍스트(HQ) 또는 QA 형식으로 바꾸어말하기하는 것이다. 다른 하나는 합성 교과서(TXBK)로, LLM을 사용하여 교과서 스타일의 데이터를 0부터 작성하는 것이다. 결과는 분명했다. 웹 데이터와 합성 데이터를 섞은 경우에 훈련의 효율이 대폭 개선된 반면, 합성 데이터만으로 훈련하면 성능이 악화되었고 특히 합성 교과서만으로는 현저히 악화되었다. 모든 종류에서 웹 데이터에 33% 비율로 합성 데이터를 섞은 경우에 최선의 성능을 달성했다.

데이터가 유한하고 계산량이 무한한 설정에서의 학습도 검토되었다(Kim et al., 2025). LLM 사전학습에 투입되는 계산량은 해마다 늘어나고 있으나 데이터는 한정되어 있다. 데이터량이 유한하다는 것을 전제로 계산량을 스케일할 때 성능을 개선할 수 있는가? LLM의 표준적인 훈련 설정에서는 스케일할 수 없다. 에포크 수를 늘리면 오버피팅으로 성능이 저하되고, 모델의 크기를 늘리면 충분히 훈련하지 못해 역시 성능이 저하된다. 그러나 훈련 설정을 적절히 조정하면 스케일할 수 있다. 큰 모델일수록 작은 학습률, 적은 에포크 수, 큰 weight decay(강한 정규화)를 적용하는 것이 효과적이다.

데이터 전처리: RefinedWeb과 FineWeb-Edu

데이터 전처리 공법의 대표적 사례가 RefinedWeb(Penedo et al., 2023)이다. 웹 데이터만으로 5T 토큰의 데이터셋을 구축했고 그중 600GB를 공개했다. 필터링의 공법 등에 의해 종래보다 대규모 데이터를 구축한 것이다.

핵심은 Macrodata Refinement라는 엄밀한 좁혀내기 파이프라인이다. 복수의 필터링과 중복 제거를 조합하여 데이터를 엄밀하게 좁혀내며, 일련의 파이프라인에서 CommonCrawl 중 약 90%의 문서가 제거된다. 구체적인 단계는 다음과 같다. URL filtering(유해한 URL로부터 취득한 텍스트 배제), Text extraction(헤더나 광고 부분을 제외하고 메인 콘텐츠 텍스트만 추출), Language identification(특정 언어 텍스트만 남김), Repetition removal(텍스트 내 반복문 배제), Document-wise filtering(스팸 텍스트 필터링), Line-wise corrections(행 레벨 필터, 예: SNS의 "좋아요"), Fuzzy deduplication(MinHash 기반 유사 문장 배제), Exact deduplication(지정한 토큰 수 이상의 완전 일치 배제).

FineWeb-Edu(Penedo et al., 2024)는 LLM이 평가한 텍스트의 교육적 가치를 사용해 전처리를 수행한다. 대규모 텍스트에 LLM의 추론을 직접 적용하는 것은 비용이 높기 때문에 경량화가 필요한데, 46만 건의 웹 기사의 "교육적 가치"를 LLM에 평가시켜 훈련 데이터를 작성한 뒤, 작은 모델을 학습시켜 평가를 만든다. 그 결과 전처리 이전의 데이터(FineWeb)나 기존 데이터(Matrix)와 비교해 지식이나 추론이 필요한 태스크의 성능이 크게 개선되었다.

정리

본 장에서는 모델의 스케일을 뒷받침하는 기술 동향을 살폈다. 왜 모델을 스케일시키는데에 대한 답은 스케일 법칙의 성립과 Emergent Ability의 발현에서 찾을 수 있다. 스케일 법칙은 모델의 성능과 {파라미터 수, 데이터량, 계산량}의 관계를 밝혔으며, 이로 인해 성능 예측이 가능해지고 대규모 모델에 대한 투자 리스크가 경감되었다.

그러나 스케일화에는 여전히 과제가 존재한다. 스케일에 수반하여 필요로 하는 비용의 증가와 데이터의 부족 등이 그것이다. 이에 대응하여 파라미터 수(N) 측면에서는 보다 메모리·연산 효율이 뛰어난 모델(Efficient Attention, MoE)의 제안이, 계산량(C) 측면에서는 효율적인 학습·추론 방법(병렬화, 양자화)의 정비가, 데이터셋 크기(D) 측면에서는 데이터의 양과 질의 공법(Chinchilla 법칙, 합성 데이터, 정밀 전처리)의 축적이 각각 진행되고 있다. 세 축 모두에서 스케일 법칙이라는 단일한 원리를 실용화하기 위한 다층적 연구·개발이 이어지고 있는 셈이다.

참고문헌

[2] Zhao et al. A Survey of Large Language Models. 2023. arXiv:2303.18223. [3] Kaplan et al. Scaling Laws for Neural Language Models. 2020. arXiv:2001.08361. [4] Wei et al. Emergent Abilities of Large Language Models. 2022. arXiv:2206.07682. [14] Abhinav Venigalla, Linden Li. Billion-Parameter GPT Training Made Easy. MosaicML. [15] Vaswani et al. Attention Is All You Need. 2017. NeurIPS2017. [17] Villalobos et al. Will we run out of data? An analysis of the limits of scaling datasets in Machine Learning. 2022. arXiv:2211.04325. [19] Child et al. Generating Long Sequences with Sparse Transformers. 2019. arXiv:1904.10509. [20] Zaheer et al. Big Bird: Transformers for Longer Sequences. 2020. NeurIPS2020. [21] Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. 2022. NeurIPS2022. [22] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. 2023. arXiv:2307.08691. [23] Chen et al. Towards Understanding Mixture of Experts in Deep Learning. 2022. NeurIPS2022. [25] Fedus et al. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. 2021. arXiv:2101.03961. [26] Rajbhandari et al. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. 2022. ICML2022. [27] Clark et al. Unified Scaling Laws for Routed Language Models. 2022. arXiv:2202.01169. [32] Microsoft DeepSpeed Team. DeepSpeed: Extreme-scale model training for everyone. Microsoft. [33] Rajbhandari et al. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. 2019. arXiv:1910.02054. [37] Dettmers et al. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. 2022. NeurIPS2022. [38] Liu et al. Do Emergent Abilities Exist in Quantized Large Language Models: An Empirical Study. 2023. arXiv:2307.08072. [39] Penedo et al. The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only. 2023.

arXiv:2306.01116. [40] Daisuke Okanohara. MinHash에 의한 고속 유사 검색. Preferred Networks R&D, 2011. [41] Cossu et al. Continual Pre-Training Mitigates Forgetting in Language and Vision. 2022. arXiv:2005.09357. [42] Hoffmann et al. Training Compute-Optimal Large Language Models. 2022. NeurIPS2022. [43] Harm de Vries. Go smol or go home, Why we should train smaller LLMs on more tokens. 2023. [55] NVIDIA. NeMo Framework User Guide — Parallelisms. [56] Dettmers et al. The case for 4-bit precision: k-bit Inference Scaling Laws. 2023. arXiv:2212.09720. [57] Wang et al. BitNet: Scaling 1-bit Transformers for Large Language Models. 2023. arXiv:2310.11453. [58] Ma et al. BitNet b1.58 2B4T Technical Report. 2025. arXiv:2504.12285. [59] Kang et al. Demystifying Synthetic Data in LLM Pre-training: A Systematic Study of Scaling Laws, Benefits, and Pitfalls. 2025. arXiv:2510.01631. [60] Chang et al. How Do Large Language Models Acquire Factual Knowledge During Pretraining?. 2024. arXiv:2406.11813. [61] Kim et al. Pre-training under infinite compute. 2025. arXiv:2509.14786. [62] Penedo et al. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. 2024. arXiv:2406.17557. [65] weights & biases. LLM을 제로부터 트레이닝하기 위한 베스트 프랙티스. [66] iwiwi. github gist, <https://gist.github.com/iwiwi/fc174b1f2341c2c0170be87c5b2e1d31>.

06

Day 6 — 대규모 언어 모델 fine-tuning

이 장의 구성

- 01 도입: fine-tuning이 풀고자 하는 문제
 - 02 대규모 언어 모델 학습 흐름에서의 fine-tuning
 - 03 Instruction Tuning
 - 04 Parameter Efficient Fine-Tuning
 - 05 대표적인 PEFT 기법들
 - 06 정리
 - 07 References
-

도쿄대학교 松尾・岩澤 연구실이 작성한 자료로, 2025년 10월부터 11월에 걸쳐 개최된 「LLM 대규모 언어 모델 강좌 기초편」의 강의 자료입니다. 본 자료는 크리에이티브 커먼즈 CC BY-NC-ND 4.0(저작 표시 - 비영리 - 변경금지 4.0 국제) 라이선스로 제공됩니다. 재이용 시 반드시 본 라이선스 표기를 기재해 주십시오. 비영리 목적의 재이용이 허락되며, 영리 목적 재이용은 별도 문의가 필요합니다. 원래의 표현이 바뀌지 않는 범위(폰트, 크기 등)라면 개변이 가능하나, 그 외의 개변 및 기타 라이선스 사항은 크리에이티브 커먼즈 라이선스를 확인해 주시기 바랍니다. 재이용하는 부분에 참조 논문 등의 인용이 있는 경우, 권막의 References에서 해당 인용 위치를 게시해 주십시오.

본 강의 강의 파트는 中筋涉太(NAKASUJI, Shota)가 담당했습니다. 그는 싱가포르에서 데이터 사이언스·AI를 활용한 퀀트 리서치 스타트업인 SPEQTRA Investment Research를 공동 창업하여 CIO를 맡고 있습니다. 도쿄대학교 공학부 물리공학과를 졸업하고 동 대학원 공학계열 연구과를 수료했으며, 松尾연구실에서는 공동 연구 프로젝트 및 퀀트 운용 프로젝트의 PM, GCI 강좌 TA·강사, 「이미지 인식」 강좌 교재 개발, 「금융시장 거래와 머신러닝」 강좌 감수·강사를 역임했습니다.

도입: fine-tuning이 풀고자 하는 문제

대규모 언어 모델의 성능을 개선하고 다양한 태스크·도메인에 적응시키려는 needs는 계속 커지고 있지만, 막대한 리소스를 요하는 Pre-Training은 많은 주체에게 진입 장벽이 높습니다. 이에 대한 해법으로, 사전 학습된 모델을 베이스로 하여 fine-tuning을 수행함으로써 성능 개선과 태스크·도메인 적응을 실현하려는 접근이 등장했습니다. 특히 Instruction Tuning을 통해 대화 성능과 Zero/Few-shot 성능을 향상시킬 수 있습니다.

한편, 대규모 언어 모델은 방대한 파라미터를 보유하고 있어, fine-tuning이라 하더라도 모든 파라미터를 다룰 수 없는 경우가 많습니다. 또한 Catastrophic Forgetting이나 과적합으로 인해 사전학습 모델의 성능이 훼손될 우려도 있습니다. 이에 대한 대응으로, 추가로 설정한 파라미터나 일부 파라미터만을 학습·갱신 대상으로 삼음으로써 효율적인 fine-tuning을 실현하는 기법이 등장했는데, 이를 ****Parameter Efficient Fine-Tuning(PEFT)****이라 부릅니다.

대표적인 fine-tuning 사례를 살펴보면, 사전학습된 LLM은 높은 성능을 보이지만 반드시 인간의 가치관에 부합하는 출력을 내놓지는 않습니다. ChatGPT는 InstructGPT 논문[2]에서 제안된 기법에 따라, Supervised fine-tuning(= Instruction Tuning)과 RLHF(Reinforcement Learning from Human Feedback)를 조합하여 인간의 가치관으로의 정렬을 실현했습니다[1]. OpenAI API에서는 자체 데이터셋을 활용한 fine-tuning 기능이 제공되며, 출력 포맷 고정, 이미지 이해와 텍스트 출력, 레이아웃 일관성 강화 등이 용도 예시로 제시됩니다. Prompting과 비교하여 토큰·처리 시간 절약, 응답의 품질·제어성 향상이라는 장점도 언급됩니다[3]. Med-Gemini는 Google이 개발한 Gemini를 의료용으로 특화시킨 모델로, 의료 분야에서의 멀티모달 능력이 강화되었으며 각종 벤치마크에서 강력한 결과를 보고했습니다[4].

이상의 사례를 공통으로 관통하는 질문은 "사전학습 모델을 어떻게 용도에 맞게 조정할 것인가"입니다. 본 장의 학습 목표는 세 가지입니다. 첫째, 대규모 언어 모델의 전형적인 학습 흐름에서 fine-tuning이 Pre-Training이나 RLHF·DPO에 대해 어떻게 위치 짓는지 설명할 수 있을 것. 둘째, fine-tuning에서 특히 중요한 접근인 Instruction Tuning과 PEFT가 기존 기법과 어떻게 다른지 설명할 수 있을 것. 셋째, Instruction Tuning과 PEFT의 목적과 내용을 충분히 이해한 바탕 위에서 실제로 이들을 구현하고 대규모 언어 모델의 성능 개선을 실현할 수 있을 것입니다.

대규모 언어 모델 학습 흐름에서의 fine-tuning

대규모 언어 모델의 전형적인 학습은 세 단계로 구성됩니다. Step 1 Pre-Training은 대규모 코퍼스를 통한 자기 지도학습으로 언어 모델에 어휘·문법·지식 등 기본적인 언어 이해를 획득시키는 단계입니다. Step 2 Supervised fine-tuning은 레이블이 있는 데이터를 통한 지도학습으로 언어 모델의 성능을 개선하거나 특정 태스크나 도메인에 대한 적응을 실현하는 단계입니다. Step 3 RLHF·DPO 등은 인간의 선호에 기반한 후속 최적화를 통해 언어 모델의 출력이 보다 인간의 가치관에 부합하도록 조정하는 단계입니다. 본 장에서는 이 중 Step 2를 깊이 다룹니다.

Pre-Training과 fine-tuning/Post-Training을 비교하면, 데이터와 목적, 기법이 뚜렷이 다릅니다. Pre-Training은 어휘·문법·지식·추론 능력 등의 언어 능력을 언어 모델에 도입하는 것을 목적으로 하며, 자기 지도학습(Next Token Prediction, Masked Language Model)을 통해 대규모 데이터셋으로 수행됩니다. 예컨대 GPT-3의 CommonCrawl은 410B tokens(570GB)에 달합니다. 반면 fine-tuning/Post-Training은 사전학습 모델의 성능 개선 및 다양한 태스크에 대한 적응을 실현하는 것을 목적으로 하며, 지도학습, 하위 태스크로의 특화, Instruction Tuning, RLHF·DPO 등의 기법이 적용됩니다. 데이터셋은 소규모로, 예컨대 LIMA는 1000 샘플(3MB)에 불과하며, 인간·모델에 의한 피드백이 활용됩니다.

fine-tuning은 다시 두 축 — ****태스크 설계(A)****와 **가중치 갱신(B)** — 로 나뉩니다. 태스크 설계 관점에서 종래의 fine-tuning은 특정 하위 태스크에서 지도학습을 실시하며, 주로 하위 태스크용 특수 토큰을 활용합니다(예: 감정 분석·자연어 추론). 반면 Instruction Tuning은 지시문을 입력으로 하고 그에 대한 이상적인 출력문을 정답으로 하는 지도학습을 수행하며, 다양한 태스크가 이 입출력 형식에 내포됩니다[5][6]. 가중치 갱신 관점에서 종래의 Full-FT는 사전학습 모델이 지닌 각 층 내 모든 파라미터에 대해 갱신을 실시합니다. 보다 확실한 성능 개선이 기대되는 한편 더 많은 컴퓨팅 리소스를 필요로 합니다. 반면 PEFT는 추가로 설정한 파라미터나 일부 파라미터만 학습·갱신하므로, 적절히 활용할 수 있다면 적은 리소스로도 성능 개선을 달성할 수 있습니다.

Instruction Tuning

Instruction Tuning이라는 명칭을 널리 알린 것은 Google Research의 FLAN 논문입니다. Wei 등[6]은 다양한 태스크를 지시·답변이라는 입출력 형식으로 통일한 데이터셋으로 언어 모델을 fine-tuning하는 기법을 제안했습니다. 이렇게 fine-tuning된 모델은 평가에 사용된 25개 태스크 중 21개에서 Zero-shot 성능이 향상되었고, 20개 태스크에서는 더 많은 파라미터 수를 가진 GPT-3보다도 높은 Zero-shot 성능을 보였습니다[7]. 입출력 구성을 보면, 입력(Instruction)은 태스크를 지정하는 지시문과 (Optional) 부수적인 보충 정보로 이루어지고, 출력(Instance)은 주어진 지시문에 대한 이상적인 답변 예입니다. 예컨대 "Víte, rozhodl jsem se, že si pořídím psa. Translate to English" 라는 지시문에 대해 "You know, I decided to get a dog." 라는 정답이 짝을 이룹니다[8].

Instruction Tuning의 유효성은 두 사례에서 확인됩니다. FLAN[6]에서는 137B 모델에 Instruction Tuning을 적용한 결과, 파라미터 수에서 크게 앞서는 GPT-3의 Zero-shot 및 Few-shot 성능을 뛰어넘는 Zero-shot 성능을 보였습니다. Alpaca[9]에서는 Meta의 LLaMA 7B 모델에 Instruction Tuning을 적용하여, 파라미터 수에서 크게 앞서는 GPT-3.5와 동등 수준의 지시 응답 거동으로 개선했습니다. 예컨대 "What is an alpaca? How is it different from a llama?"라는 입력에 대해 종·체형·털의 차이를 서술하는 자연스러운 답변이 생성됩니다.

그러나 Instruction Tuning에는 두 가지 근본적 어려움이 있습니다. 첫째는 **데이터셋 작성상의 곤란**입니다. 바람직한 거동을 실현하기 위해서는 고품질이고 무해한 데이터셋의 마련이 필요한데, 이를 사람이 직접 작성할 것인지, 기존 데이터셋을 활용할 것인지, 아니면 LLM으로 생성할 것인지를 두고 저마다의 trade-off가 있습니다. 나아가 지시에 포함된 개별 태스크나 형식의 다양성의 중요성도 지적되고 있어, 이러한 다양한 관점을 고려하며 데이터셋을 구축하려면 많은 인적·기술적 리소스가 필요합니다. 둘째는 **지식은 도입 가능한가**라는 물음입니다. LIMA(2023)[10]는 fine-tuning이 사전학습에서 획득된 지식·능력을 "끌어냄"으로써 성능 개선을 실현한다는 **Superficial Alignment Hypothesis**를 제창했습니다. 이 가설이 옳다면, Instruction Tuning에 의한 성능 개선은 태스크의 이해를 통한 것이 아니라 출력 형식 같은 표면적 사항의 학습에 기인할 가능성도 있다는 지적(Kung and Peng, 2023)[9]이 성립합니다.

이러한 어려움에 대응하기 위해 Instruction 데이터셋 구축에서는 세 가지 요점이 강조됩니다. 첫째, **데이터의 질**입니다. LIMA[10]는 Instruction Tuning에서는 데이터의 양보다 질이 중요하다고 주장하며, 1000건이라는 소량의 고품질 데이터만으로 RLHF로 학습된 모델보다 고품질의 답변을 생성할 수 있었음을 보고했습니다. 둘째, **데이터의 무해성**입니다. 사전학습 모델에서 우려되는 유해한 출력을 억제하기 위해, Instruction Tuning에서는 유해한 데이터를 피해 학습을 실시합니다. Llama 2[11]는 무해한 데이터셋 구축의 실례를 제시합니다. Meta가 개발·공개하는 이 모델(7B, 13B, 70B 변형 포함)은 안전성 향상을 목적으로 인간에 의한 어노테이션과 평가를 적극적으로 채용했습니다. 어노테이터는 복수의 테스트로 자질과 적성을 평가받아 선정되며, 선정된 어노테이터에게는 **Informative·Relevant·Harmless·Truthful·Clear**

를 만족하는 지시문·답변의 작성이 의뢰됩니다. 지시문 작성에서는 범죄 행위의 조장, 공격적인 언행의 조장 등 피해야 할 항목이 명시됩니다. 셋째, **지시 형식의 다양성**입니다. 태스크별 지시 형식의 다양화로 미지 태스크에 대한 성능이 향상됨이 확인되었습니다[12].

데이터셋 구축 기법은 크게 세 가지로 나뉩니다. 첫째, **기존 레이블 데이터셋의 통합**입니다. FLAN[6]은 62개의 데이터셋을 템플릿을 이용해 변환·통합했습니다. 둘째, **인간에 의한 데이터 작성**입니다.

InstructGPT[2]는 인간이 작성한 지시문에 대해 인간이 답변을 작성했습니다. 셋째, **LLM에 의한 데이터 생성**입니다. Self-Instruct[13]는 LLM에 의한 지시문과 답변 생성 프레임워크를 제안했습니다.

Parameter Efficient Fine-Tuning

PEFT의 필요성은 Full-FT의 비용에서 비롯됩니다. Full-FT는 사전학습 모델의 모든 파라미터에 대해 다른 태스크에서 갱신을 실시하므로, 대규모 모델에서는 막대한 컴퓨팅 리소스가 필요합니다(예: GPT-3는 1.2TB의 GPU 메모리). 또한 원 모델과 동일 크기의 파라미터를 보존해야 하므로 큰 보존 영역이 필요합니다(예: GPT-3는 350GB의 보존 영역). 반면 PEFT는 추가로 설정한 파라미터나 일부 파라미터만으로 갱신을 실시하여, 대규모 모델에 대해서도 제한된 컴퓨팅 리소스로 성능 개선을 실현합니다. 예컨대 GPT-3 LoRA는 350GB의 GPU 메모리로 동작하며, 갱신 부분의 파라미터만 보존하면 되므로 보존 영역은 35MB에 불과합니다[14].

GPU 메모리 사용량의 구체적 비교를 위해 7B 모델의 16-bit fine-tuning을 상정해 봅니다. 전체 파라미터 수 $N_{\text{all}} = 7\text{B}$, 부동소수점 크기 $\text{size(float)} = 2\text{byte}$ 상황에서, Full-FT가 갱신하는 파라미터 수 $N_{\text{train}} = 7\text{B}$, PEFT가 갱신하는 파라미터 수 $N_{\text{train}} \approx 1\text{M}$ 로 잡으면 각 항목은 다음과 같이 추정됩니다. Model Loading($\text{size(float)} \times N_{\text{all}}$)은 양쪽 모두 약 13GB, Backward/Gradients($2 \times \text{size(float)} \times N_{\text{train}}$)은 Full-FT 약 26GB·PEFT 약 2MB, Optimizer/Adam($\text{size(float)} \times N_{\text{train}}$)은 Full-FT 약 13GB·PEFT 약 4MB입니다. Forward 분이나 라이브러리 확보 영역 등 batch size에 비례하여 증가하는 부분(α)을 더하면, 총 GPU 메모리 사용량은 Full-FT 약 $52\text{GB} + \alpha$, PEFT 약 $13\text{GB} + \alpha$ 로 큰 차이가 납니다.

PEFT 기법을 평가할 때는 네 가지 관점이 중요합니다. 첫째, **성능 개선** — Full-FT를 실시한 경우와 비교하여 성능 개선에 큰 열화가 없는가, 그리고 사전학습 모델의 크기에 의존하지 않고 성능 개선이 실현되는가입니다. 둘째, **운용성** — 갱신하는 파라미터가 적고 작은 스토리지로 운용이 가능한가입니다. 그것이 가능하면 복수 모델의 병렬 운용이나 버저닝이 용이해집니다. 셋째, **추론 효율** — 추가하는 파라미터가 많아 추론 비용을 증대시키지 않는가, 그리고 입력문의 계열 길이가 길어져 추론 비용을 증대시키지 않는가입니다. 넷째, **학습 효율** — 학습하는 파라미터가 적고 작은 GPU 메모리로도 실현 가능한가, GPU의 효율적 활용에 의해 고속화가 가능한 기법인가입니다. 여기서 "학습하는 파라미터는 적지만, 그에 기반하여 많은 파라미터가 갱신된다"는 경우가 있기 때문에, "갱신하는 파라미터"와 "학습하는 파라미터"라는 비슷한 표현을 구별하여 사용합니다.

Lialin 등[15]은 다양한 PEFT 기법을 추론 시의 오버헤드 관점에서 분류합니다. FFN 층을 추가하는 기법(Extra FFN)은 추론에 오버헤드를 수반하고, 입력 계열에 무언가를 추가하는 기법(Extra input)도 추론에 오버헤드를 수반합니다. 반면 어떤 기법은 추론에 오버헤드를 수반하지 않습니다(No overhead). 이 분류를 따라 PEFT 기법은 대략 네 가지 카테고리로 정리됩니다. **Adapter형**은 Transformer 내부에 MLP 층(Adapter)을 추가하고 그것만 학습합니다(대표 예: Adapter, 2019). **Soft Prompt형**은 입력 계열에 태스크별 벡터(Soft Prompt)를 부가하고 학습합니다(대표 예: Prompt Tuning, 2021). **Selective형**은 사전학습 모델이 지닌 파라미터 중 일부만으로 학습합니다(대표 예: BitFit, 2021). **Reparametrization형**은 행렬 분해에 기반해 재파라미터화된 가중치에 대해 학습합니다(대표 예: LoRA, 2021).

대표적인 PEFT 기법들

Adapter(2019)[16]는 Transformer 내부에 학습 가능한 Adapter 모듈을 추가하여 학습합니다. Adapter는 단순한 MLP 구조를 가지며, 추가 위치가 다른 아종도 존재합니다(예: Parallel Adapter[17]는 병렬적으로 Adapter를 추가). 장점은 Full-FT 대비 10^{-1} 에서 10^{-2} 정도로 작은 학습 파라미터 수로 Full-FT와 동등한 정확도를 달성한다는 점, 그리고 Adapter만 보존하면 되어 유연하게 교체 대응이 가능하다는 점입니다. 단점은 Adapter가 추가됨으로써 추론에 오버헤드가 발생한다는 것입니다.

Prompt Tuning(2021)[18]은 각 태스크에 대응한 벡터(Soft Prompt)를 입력 계열에 부가하고 그 파라미터를 학습합니다. Soft Prompt는 문장 형태로 설계된 프롬프트(Hard Prompt)에 대한 호칭·개념으로, 즉 각 태스크마다 특화된 프롬프트 엔지니어링을 학습하고 있다고 간주할 수 있습니다. 장점은 모델 크기가 큰 경우 Full-FT와 동등한 정확도를 달성한다는 점입니다. T5-XXL(11B)에서 Soft Prompt 길이를 100으로 하면 학습 파라미터 수는 4096×100 로, 이는 Full-FT의 0.007%에 해당합니다. 단점은 Soft Prompt가 입력 계열을 압박한다는 점과, 프롬프트 엔지니어링의 확장으로 간주하면 해석성이 결여된 결과가 된다는 점입니다.

BitFit(2021)[19]은 Transformer의 각 모듈에 포함된 바이어스 항에 대해서만 학습·갱신을 실시합니다. 구체적으로 Attention, Feed-Forward Network, Layer Normalization에 포함된 바이어스 항이 해당합니다. 장점은 학습 데이터 수가 작은 영역에서는 Full-FT보다 높은 정확도를 보였다는 점이며, BERT(Base) 모델에서 BitFit에 의한 학습 파라미터 수는 Full-FT 대비 0.1% 정도입니다. 단점은 GPT-3 같은 보다 대규모 모델에서는 Full-FT나 다른 PEFT 기법보다 정확도가 뒤떨어진다는 점입니다[14].

LoRA(2021)[14]는 Reparametrization형의 대표 기법입니다. fine-tuning으로 갱신된 가중치 W 는 일반적으로 원래 가중치 W_0 와 증분 가중치 ΔW 의 합으로 표현 가능합니다. LoRA에서는 이 증분 가중치 ΔW 를 두 저랭크 행렬 $A(d \times r)$ 와 $B(r \times d)$ 의 곱으로 삼고, 그것들에 대해 학습을 실시합니다. 사전학습 모델 선형 층의 가중치는 고정해 두고, A 는 정규 난수로, B 는 영행렬로 초기화하여 두 경로의 결과를 합산해 다음 층으로 전달합니다. 장점은 Full-FT 대비 10^{-2} 에서 10^{-4} 정도로 작은 학습 파라미터 수로

Full-FT와 동등한 정확도를 달성한다는 점, 그리고 추론 시에는 얻어진 가중치를 원래 가중치에 미리 더해두면 오버헤드가 발생하지 않는다는 점입니다. 단점은 특히 난이도가 높은 태스크(예: GSM8k, 수학적 추론)에서 Full-FT 대비 현저한 성능 열위가 발생할 수 있다는 점입니다[20].

LoRA 적용 시 두 가지 실무적 질문이 있습니다. 첫째, 학습 파라미터 수를 일정하게 할 때 LoRA를 적용하는 층의 종류를 더 늘려야 할까, 랭크 r 을 더 크게 잡아야 할까? LoRA를 적용하는 층의 종류를 늘리는 쪽이, 랭크 r 이 작아지더라도 더 높은 성능이 된다는 것이 밝혀졌습니다. LoRA 논문에서는 Attention 모듈 내(Query·Key·Value·Output projection)를 적용 대상으로 했으나, 이후 연구에서는 다른 선형 층도 대상으로 함으로써 성능이 개선됨이 확인되었습니다. 둘째, LoRA를 적용하는 층의 종류를 고정해 놓고 생각할 때, 랭크 r 은 어느 정도의 값을 설정할 필요가 있는가? LoRA의 랭크 r 은 2에서 8 범위에서 높은 성능을 보이며, 태스크 의존적이지만 랭크 1로도 충분한 성능이 나오는 경우도 있습니다. 경험적으로는 랭크 8 정도의 설정이 권장되고 있습니다.

LoRA에는 세 가지 파생 접근이 있습니다. QLoRA[21]는 보다 적은 컴퓨팅 리소스로도 LoRA에 의한 fine-tuning을 실현하고자, LoRA에 4비트 양자화 등의 기법을 적용해 메모리 사용량을 더욱 절감합니다. AdaLoRA[22]는 LoRA에서 모든 층의 랭크가 단일 값으로 제한되는 문제를 해결하기 위해, 증분 가중치의 특이값 분해에 기반해 층마다 랭크를 적응적으로 변화시킵니다. LoRA-Pro[23]는 Full-FT의 기울기를 근사하지 못하는 문제를 완화하고 Full-FT와의 성능 차이를 좁히기 위해, LoRA의 두 저랭크 행렬의 기울기가 전체 기울기에 부합하도록 이론적으로 최적 조정을 수행합니다.

대표적인 PEFT 기법을 종합적으로 비교하면 다음과 같습니다[15]. 성능 개선 측면에서 Adapter와 LoRA는 비교적 안정적이나, Prompt Tuning은 불안정한 경향이 있고 BitFit은 대규모 모델에서 열화가 있습니다. 운용성(갱신률)은 Adapter와 Prompt Tuning이 0.1~6%, BitFit이 약 0.1%, LoRA가 0.05~0.1% 수준입니다. 추론 효율 측면에서 Adapter는 추론 시간 증가가 태스크에 의존하고, Prompt Tuning은 입력 계열 길이 압박이 있으며, LoRA는 추론 시 변화가 없습니다. 학습 효율(학습률)은 Adapter와 Prompt Tuning이 0.01~0.5%, BitFit이 ~0.5%, LoRA가 0.01~0.5% 수준으로, 대체로 PEFT 기법들은 Full-FT 대비 수십~수만 분의 일 수준의 파라미터만으로 유사한 성능을 달성합니다.

정리

다시 ChatGPT 사례로 돌아가 봅시다. ChatGPT는 InstructGPT 논문[2]에서 제안된 흐름에 따라 Supervised fine-tuning(= Instruction Tuning)과 RLHF를 채택했습니다. InstructGPT에서는 인간이 Instruction Tuning용으로 약 1만 건의 데이터를 작성했으며, 이를 통해 인간적 가치관으로의 출력 정렬을 실현했습니다[1]. 같은 맥락에서 OpenAI API는 자체 데이터셋을 활용한 fine-tuning을 제공하며, Med-Gemini는 의료 도메인 특화를 위해 fine-tuning과 도메인 적응을 수행한 사례입니다[3][4].

본 장의 세 가지 목표를 되돌아봅니다. 첫째, 대규모 언어 모델의 학습 흐름(Pre-Training → Supervised fine-tuning → RLHF·DPO)에서 fine-tuning의 위치를 이해했습니다. 둘째, Instruction Tuning이 지시·답변 형식의 지도학습으로 Zero-shot 성능과 지시 응답 성능을 향상시키는 메커니즘과, PEFT가 일부 파라미터만 갱신하여 효율적 성능 개선을 달성하는 차이를 설명했습니다. 셋째, Instruction Tuning과 PEFT의 목적과 내용을 바탕으로 실제 구현과 성능 개선을 실현할 수 있는 기반을 갖추었습니다. 본 장에서 다룬 Instruction Tuning과 PEFT는 사전학습 모델을 용도에 맞게 조정하는 핵심 기법이며, 다음 단계에서는 이들을 실제로 구현하는 실습이 이어집니다.

References

- [1] OpenAI (2023), "Introducing ChatGPT", <https://openai.com/ja-JP/index/chatgpt/> 액세스일: 2026/5/24
- [2] Ouyang, Long, et al. (2022), "Training language models to follow instructions with human feedback", *Advances in Neural Information Processing Systems* 35, pp. 27730–27744
- [3] OpenAI (2024), "Fine-tuning now available for GPT-4o", <https://openai.com/ja-JP/index/gpt-4o-fine-tuning/> 액세스일: 2026/5/24
- [4] Saab, Khaled, et al. (2024), "Capabilities of gemini models in medicine", arXiv:2404.18416
- [5] PyTorch Tutorial, "Dynamic Quantization on BERT", <https://docs.pytorch.org/tutorials/index.html> 액세스일: 2026/5/24
- [6] Wei, Jason, et al. (2021), "Finetuned language models are zero-shot learners", arXiv:2109.01652
- [7] Google Research (2021), "Introducing FLAN: More generalizable Language Models with Instruction fine-tuning", <https://research.google/blog/introducing-flan-more-generalizable-language-models-with-instruction-fine-tuning/> 액세스일: 2026/5/24
- [8] HuggingFace, "flan2021_submix_original", https://huggingface.co/datasets/conceptofmind/flan2021_submix_original 액세스일: 2026/5/24
- [9] Taori, Rohan, et al. (2023), "Alpaca", Stanford Center for Research on Foundation Models, <https://crfm.stanford.edu/2023/03/13/alpaca.html> 액세스일: 2026/5/24
- [10] Zhou, Chunting, et al. (2023), "Lima: Less is more for alignment", arXiv:2305.11206
- [11] Touvron, Hugo, et al. (2023), "Llama 2: Open foundation and fine-tuned chat models", arXiv:2307.09288

-
- [12] Sanh, Victor, et al. (2021), "Multitask prompted training enables zero-shot task generalization", arXiv:2110.08207
- [13] Wang, Yizhong, et al. (2022), arXiv:2212.10560
- [14] Hu, Edward J., et al. (2021), "LoRA: Low-rank adaptation of large language models", arXiv:2106.09685
- [15] Lialin, Vladislav, Vijeta Deshpande, and Anna Rumshisky (2023), "Scaling down to scale up: A guide to parameter-efficient fine-tuning", arXiv:2303.15647
- [16] Houlsby, Neil, et al. (2019), "Parameter-efficient transfer learning for NLP", International Conference on Machine Learning, PMLR
- [17] He, Junxian, et al. (2021), "Towards a unified view of parameter-efficient transfer learning", arXiv:2110.04366
- [18] Lester, Brian, Rami Al-Rfou, and Noah Constant (2021), "The power of scale for parameter-efficient prompt tuning", arXiv:2104.08691
- [19] Zaken, Elad Ben, Shauli Ravfogel, and Yoav Goldberg (2021), "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models", arXiv:2106.10199
- [20] anyscales (2023), "fine-tuning LLMs: LoRA or Full-Parameter? An in-depth Analysis with Llama 2", <https://www.anyscale.com/blog/fine-tuning-llms-lora-or-full-parameter-an-in-depth-analysis-with-llama-2> 액세스일: 2026/5/24
- [21] Dettmers, Tim, et al. (2023), "QLoRA: Efficient finetuning of quantized LLMs", arXiv:2305.14314
- [22] Zhang, Qingru, et al. (2023), "Adaptive budget allocation for parameter-efficient fine-tuning", arXiv:2303.10512
- [23] Wang, Zhengbo, et al. (2024), "LoRA-Pro: Are low-rank adapters properly optimized?", arXiv:2407.18242

07

제7회: 강화학습

이 장의 구성

- 01 이번 회의 목적과 목표
- 02 LLM 훈련 플로우에서 강화학습의 위치
- 03 RLHF: 인간의 피드백으로부터의 강화학습
- 04 DPO: 보상 모델 없이 직접 선호도를 학습하는 오프라인 기법
- 05 RLVR: 검증 가능한 보상으로 추론 능력을 향상시키는 강화학습
- 06 강화학습의 응용과 향후 방향
- 07 정리
- 08 References

도쿄대학교 마쓰오·이와사와 연구실이 작성하고, 2025년 10월부터 11월에 걸쳐 개최된 「대규모 언어모델 강좌 2025」 기초편 강의 자료입니다. 크리에이티브 커먼즈 CC BY-NC-ND 4.0(저작자표시-비영리-변경금지 4.0 국제) 라이선스로 등록되어 있습니다. 재이용(2차 이용)이 허락되어 있으며, 재이용 시에는 반드시 라이선스 표기를 기재해 주세요. 재이용하는 곳에 참고 논문 등의 인용이 있는 경우, 권말의 References에서 인용 위치를 게시해 주세요. 원래의 표현이 변하지 않는 범위(글꼴·크기 등)라면 개작이 가능하며, 영리 목적 재이용에 대해서는 별도 문의가 필요합니다. 허락 없는 촬영 및 제3자에 대한 공개를 금지합니다.

강사: 다카기 쇼타(도쿄대학교 대학원 공학계 연구과 마쓰오 연구실 박사 2년차). 2019년 나라 공업고등전문학교 정보공학과, 2022년 오사카대학교 기초공학부 시스템화학과를 거쳐 2022년 4월부터 도쿄대학교 공학계 연구과 기술경영전략학 전공. Sony ML R&D intern, DeNA backend intern, Recruit Data Specialist Intern, SanSan Intern 등을 경험했습니다. 전문 분야는 대규모 언어모델, 강화학습, 로봇틱스이며, 「Deep Learning 기초 강좌」 「심층 강화학습 스프링 세미나」 「대규모 언어모델 강좌」 「세계 모델과 지능」 등의 강사를 담당했습니다.

이번 회의 목적과 목표

이번 회의의 목적은 LLM에서 강화학습이란 무엇인지, 그 메커니즘과 필요성을 이해하는 데 있습니다. 구체적 목표는 셋입니다. 첫째, LLM에서 강화학습의 목적을 이해하고 설명할 수 있다. 둘째, 강화학습의 대표적 기법인 RLHF, DPO, GRPO의 개요를 설명할 수 있다. 셋째, PyTorch로 각 기법을 구현할 수 있다. 지금까지의 강의 내용과 기본적인 딥러닝 지식을 전제로 합니다.

LLM 훈련 플로우에서 강화학습의 위치

LLM의 훈련은 세 단계로 구성됩니다. 첫 번째 **사전학습(Pre-Training)** 단계는 대규모 코퍼스를 통한 자기지도학습으로 어휘·문법·지식·추론 능력 등 기본 언어 이해를 획득시키는 단계입니다. Next Token Prediction, Masked Language Model 등이 사용되며, GPT-3의 사례에서는 CommonCrawl 기반 410B 토큰(약 570GB) 같은 대규모 데이터셋이 활용됩니다. 두 번째 **지도 fine-tuning(Supervised Fine-Tuning)** 단계는 레이블이 붙은 데이터로 언어모델의 성능을 개선하거나 특정 태스크·도메인에 적응합니다. 하위 태스크 특화와 Instruction Tuning이 여기에 해당하며, LIMA(1,000샘플, 약 3MB)처럼 양질의 소규모 데이터셋이 자주 사용됩니다. 세 번째 **강화학습(Reinforcement Learning)** 단계가 이번 강의의 주제로, 인간의 피드백이나 규칙 기반 보상을 활용해 언어모델의 출력이 인간의 가치관에 더 부합하도록 조정하고 추론 능력을 향상시킵니다.

"Supervised fine-tuning"이라는 표현은 강화학습 기법인 RLHF와 구별하기 위해 사용됩니다. 의도적으로 이 표현을 쓰는 경우 일반적인 지도 fine-tuning이 아니라 Instruction Tuning을 가리키는 경우가 많습니다. 사전학습과 fine-tuning/Post-Training의 차이는 목적과 데이터 규모에 있습니다. 사전학습이 대규모 데이터로 일반 언어 능력을 도입하는 데 반해, fine-tuning은 소규모 양질의 데이터로 특정 태스크에 적응하거나 인간·모델의 피드백을 반영합니다.

강화학습에서 에이전트(행동 주체)는 환경의 상태에 기반하여 순차적으로 행동을 결정하며, 행동의 결과 얻어지는 보상을 이용해 최적 정책을 학습합니다. 대표적 응용으로 AlphaGo를 들 수 있으며, 자율주행차, 로봇 공학(Boston Dynamics 등), 게임 등 널리 응용되어 왔습니다.

언어모델에서의 강화학습은 출력 문장에 대해 보상을 주고 그것을 피드백하는 구조로, Next Token Prediction과는 다른 틀에서 학습합니다. 보상의 주체는 세 유형입니다. 인간이 주는 경우(RLHF), 다른 LLM을 이용하는 경우(RLAIF), 규칙 기반 모델로 주는 경우(RLVR). 대규모 언어모델에서 강화학습은 주로 두 가지 용도로 쓰입니다. 첫째는 ****얼라인먼트(Alignment)****로, 차별적·폭력적 발언을 억제하고 인간의 가치관에 맞추는 것입니다. 둘째는 **추론 능력 향상**으로, 수학·코딩 등에서 깊이 사고하도록 강화학습을 수행해 비약적 성능 향상을 달성합니다(수학 올림피아드 금메달, IQ 140 이상 기록 등). 이에 따라 언어모델 강화학습은 두 갈래로 나뉩니다. 인간의 가치관에 맞추는 RLHF/DPO 계열과, 수학·코딩 능력 향상이 목적인 RLVR 계열입니다.

RLHF: 인간의 피드백으로부터의 강화학습

기존 언어모델의 문제점과 얼라인먼트의 필요성

초기 대규모 언어모델은 인간에게 바람직하지 않은 발언, 즉 설계자가 의도하지 않은 발언을 하여 논란이 되었습니다. 마이크로소프트 Tay는 Twitter에 등장하여 24시간 만에 5만 팔로워를 획득하고 10만 회 트윗했지만, 악의적 이용자의 발언에 영향을 받아 16시간 만에 혐오 발언을 반복하였고 결국 공개 하루 만에 중단되었습니다(2016년). 한국의 이루다는 카카오톡 대화 100억 건을 바탕으로 작성되어 2주 만에 이용자 75만 명을 기록했지만, 성적 소수자에 대한 혐오 발언으로 약 1개월 만에 서비스가 중단되었습니다(2021년).

이러한 의도하지 않은 발언을 Instruction Tuning만으로 대처하는 것은 어렵습니다. 자연어 데이터 수집에 비용이 들고, "말하지 않도록" 하는 정답 데이터를 만들기 어려우며, Instruction Tuning은 인간의 의도를 직접적으로 학습하는 것이 아니기 때문입니다. 그래서 어떤 문장이 의도대로의 출력인지를 모델에 피드백하여 인간의 의도에 직접 부합하도록 학습하는 HITL(Human in the loop) 접근이 필요했습니다.

인간의 의도대로 모델을 학습시키는 것을 ****얼라인먼트(Alignment)****라 하며, 이를 위해 RLHF가 필요합니다. 인간의 의도에는 명시적 의도("이 지시에 따라주세요", "어시스턴트로서 행동해 주세요")와 암묵적 의도("날조하지 않는다", "유해한 말은 하지 않는다")가 존재합니다.

얼라인먼트의 기준: HHH

얼라인먼트의 기준은 보통 세 가지로 정리됩니다. ****Helpful(도움됨)****은 사용자 질문에 간결·효율적으로 답하고, 정보가 부족하면 적절한 질문을 던지며, 상대방의 수준에 맞춘 응답을 합니다. ****Honest(정직함)****는 허위 없이 정확한 문장을 출력하며, 자신이 어느 정도 불확실한 정보인지 제시합니다(이를 위해 모델은 자신이 무엇을 아는지 이해해야 합니다). ****Harmless(무해함)****는 공격적·차별적 발언을 하지 않고 악의적 질문을 감지해 거부합니다. 이 세 가지를 합쳐 **HHH**라 부르며, Helpful은 HH-RLHF, Honest는 HaluEval, Harmless는 CrowS-Pairs·WinoGender 등으로 평가합니다. 그 밖에 LLM 잠재적 리스크를 3개 수준·합계 60개 타입으로 정의한 Do-Not-Answer 같은 평가 데이터셋도 존재합니다.

인간이 피드백을 주는 것에도 한계가 있습니다. 태스크가 복잡해지면 인간이 평가할 수 없게 되기 때문입니다. 이를 보완하기 위해 AI 어시스턴트의 힘을 빌리는 연구가 진행되고 있습니다. 더 나아가 AGI가 폭주하지 않도록 제어하는 **Superalignment** 연구, 약한 LLM이 강한 LLM을 감독하는 weak-to-strong generalization 등이 추진되고 있습니다. AGI가 Alignment되지 않으면 인류에 중대한 리스크를 가져올 가능성이 있다고 OpenAI는 주장합니다.

RLHF의 전체상과 역사

RLHF는 InstructGPT, ChatGPT 등에서 이용됩니다. ChatGPT는 2022년 11월 30일 공개되어 1주일 만에 100만 사용자, 2개월 만에 1억 사용자에게 도달했습니다.

RLHF의 학습은 세 단계로 구성됩니다. ****Step 1(지도학습)****에서는 프롬프트와 그에 대한 적절한 답변 쌍을 레이블러(인간)가 고안해 데이터셋을 작성하고, 사전학습 모델을 fine-tuning합니다. ****Step 2(보상 모델 학습)****에서는 어떤 프롬프트에 대한 Step 1 모델의 답변을 복수 패턴 준비해 레이블러에게 순위를 매겨 달라고 하고, 이 순위 데이터셋으로 보상 모델을 학습합니다. ****Step 3(강화학습)****에서는 보상이 최대가 되는 정책(Step 1 모델)을 탐색해 최적의 답변을 생성합니다. 모델 답변에 대해 보상값을 추정하고 그것을 피드백하여 정책을 개선합니다. 보상 모델에는 사전학습 모델이나 fine-tuning 모델의 최종층만 선형층으로 변경한 모델이 자주 사용되어 출력은 스칼라값이 됩니다.

RLHF의 역사는 2017년 OpenAI의 "소수의 인간 피드백으로부터 강화학습" 연구로 거슬러 올라갑니다 (같은 해 PPO가 발표됨). 로봇 시뮬레이터와 Atari에서 샘플 효율 향상을 확인했으며, 정책이 보상을 최대화하도록 학습하고, 출력 행동 중 두 가지를 인간이 평가하며, 비교 결과로 Reward Predictor를 학습하는 세 단계로 이루어졌습니다. 2020년에는 GPT-3로 요약 태스크에 적용되어 fine-tuning이나 인간 작성 참조 요약보다 우수한 결과를 보였습니다.

InstructGPT에서 이 기법이 본격 정립되었습니다. Step 1에서는 프롬프트 데이터셋에 대한 인간 labeler의 답변으로 지도학습을 수행하고, Step 2에서는 출력에 대한 "바람직함"을 순위 매긴 데이터로 보상 모델을 학습하며, Step 3에서는 보상 모델이 보상을 생성하고 PPO로 강화학습을 수행합니다. Step 3 완료 후 새 모델로 Step 2~3를 반복합니다.

강화학습의 최적화 알고리즘: PPO

RLHF에서 자주 사용되는 강화학습 알고리즘은 PPO입니다. PPO는 Actor-Critic 알고리즘의 파생형으로, Actor와 Critic이 협력해 정책을 갱신하고 보상을 최대화합니다.

정책(Actor)과 가치 모델(Critic)은 파라미터를 가진 신경망으로 표현됩니다. 정책 경사법은 θ 로 파라미터화된 정책을 기울기로 직접 최적화하는 기법입니다.

“

$$J(\pi\theta) = V\phi(s_0) \quad \nabla J(\pi\theta) = Et[\nabla\theta \log \pi\theta(at|st) \cdot At(st, at)] \quad \theta \leftarrow \theta + \alpha \nabla\theta J(\pi\theta)$$

$V\phi(st)$ 는 가치 함수, $\pi\theta(at|st)$ 는 정책, At 는 어드밴티지 함수(어떤 상태 st 에 대해 행동 at 가 얼마나 가치 있는지의 추정값)입니다.

PPO는 TRPO를 단순화한 기법입니다. TRPO는 정책 경사법의 갱신 폭에 KL 거리 제약을 걸어 정책 열화를 방지하고, PPO는 갱신 폭을 clip해 계산 복잡함을 경감합니다. 가치 모델은 보상 함수의 MSE로 학습됩니다.

“

$$\hat{L}^{PPO}(\theta) = Et[\min(rt(\theta) \cdot At, \text{clip}(rt(\theta), 1-\epsilon, 1+\epsilon) \cdot At)] \quad \hat{L}^{critic}(\phi) = Et[(V\phi(st) - Rt)^2] \quad rt(\theta) = \pi\theta(at|st) / \pi_{old}(at|st)$$

ϵ 는 clipping 파라미터, Rt 는 기대 보상 함수입니다.

강화학습 학습 자료로는 영어권의 Coursera RL Specialization, Reinforcement Learning Lecture Series 2021(DeepMind x UCL), Stanford CS234: Reinforcement Learning, David Silver의 Introduction to Reinforcement Learning, UC Berkeley CS 285: Deep Reinforcement Learning, Deep RL BootCamp, HuggingFace의 Deep Reinforcement Learning Course가 있으며, 일본어 자료로는 도쿄대학교 마쓰오 연구실 심층 강화학습 서머 스쿨 강의 자료와 『강화학습(제2판)』 등이 있습니다.

보상 모델 학습

Labeler가 프롬프트 출력에 순위를 매기고 그 데이터로 보상 모델을 학습합니다. $K=4\sim 9$ 개 출력으로부터 2개 조합에 대해 모든 순위 매기기를 수행합니다($K=4$ 이면 $4C2=6$ 가지). 보상 모델 $r\theta(x, y)$ 는 다음 손실 함수로 학습합니다(yw 가 yl 보다 좋은 답변, w : win, l : lose).

//

$$\text{loss}(\theta) = -E_{(x, y_w, y_l \sim D)} [\log(\sigma(r\theta(x, y_w) - r\theta(x, y_l)))]$$

즉 좋은 답변 (x, y_w) 의 보상이 나쁜 답변 (x, y_l) 의 보상보다 높아질 확률을 학습합니다. σ 는 시그모이드 함수이며, Bradley-Terry 모델을 따른다고 가정합니다. Bradley-Terry 모델에 따르면 $p^*(y_w > y_l \mid x) = \sigma(r^*(x, y_w) - r^*(x, y_l))$ 입니다.

언어모델 강화학습의 문제와 해결책

언어모델 강화학습은 "어떤 문장을 생성할 것인가"를 정책으로 하고 보상 모델 출력을 최대화하도록 학습합니다. 단순히 보상 기댓값을 최대화하면 학습이 잘 되지 않아 별도 장치가 필요합니다.

//

$$\text{objective}_1(\phi) = E_{(x, y \sim D_{\pi^{\text{RL}}_\phi})} [r\theta(x, y)]$$

두 가지 문제가 있습니다. 첫째, Reward Hacking입니다. 보상을 최대화하려는 모델이 바람직하지 않은 정책을 학습해 버리는 현상으로, 대책으로 KL Penalty를 사용합니다. 생성 문장이 SFT 모델로부터 크게 변하지 않도록 하며, β 하이퍼파라미터로 균형을 조정합니다(크면 안정적이지만 목적 함수가 커지기 어렵고, 작으면 목적 함수는 커지지만 정책이 붕괴하기 쉽습니다).

//

$$\text{objective}_1(\phi) = E_{(x, y \sim D_{\pi^{\text{RL}}_\phi})} [r\theta(x, y) - \beta \cdot \log(\pi^{\text{RL}}_\phi(y|x) / \pi^{\text{SFT}}(y|x))]$$

둘째, **Alignment Tax(얼라인먼트 세금)**입니다. 인간의 의도대로 학습시키면 일반화 성능이 열화(사전 지식 망각)하는 현상으로, 대책으로 Replay를 사용합니다. 사전학습 데이터 D_{pretrain} 로 대수 우도(log-likelihood)를 최대화해 망각을 방지하며, γ 로 균형을 조정합니다.

//

$$\text{objective}_2(\phi) = \gamma \cdot E_{(x \sim D_{\text{pretrain}})} [\log(\pi^{\text{RL}}_\theta(x))] + E_{(x, y \sim D_{\pi^{\text{RL}}_\phi})} [r\theta(x, y) - \beta \cdot \log(\pi^{\text{RL}}_\phi(y|x) / \pi^{\text{SFT}}(y|x))]$$

둘을 조합한 것이 PPO-ptx입니다.

//

$$\text{objective}(\phi) = E_{(x,y \sim D_{\pi^{\text{RL}}_\phi})}[r\theta(x, y) - \beta \cdot \log(\pi^{\text{RL}}_\phi(y|x) / \pi^{\text{SFT}}(y|x))] + \gamma \cdot E_{(x \sim D_{\text{pretrain}})}[\log(\pi^{\text{RL}}_\theta(x))]$$

PPO-ptx는 GPT, SFT와 비교해 큰 성능 개선을 보였으며 PPO 대비로도 개선이 확인되었습니다. InstructGPT는 GPT-3보다 지시를 잘 따르고 환각이 억제되며, 사용자와 동일 언어를 사용하는 비율도 높아졌습니다.

참고로 Reverse KL($D_{\text{KL}}[\pi^{\text{RL}} \parallel \pi^{\text{SFT}}]$)은 특정 모드를 커버하도록 학습하여 RLHF에서 채택되며, Forward KL($D_{\text{KL}}[\pi^{\text{SFT}} \parallel \pi^{\text{RL}}]$)은 원래 분포 전체를 커버하도록 학습합니다.

PPO-max와 강화학습의 안정화

PPO-ptx만으로 충분한 학습이 되는 것은 아닙니다. RL은 기본적으로 학습이 불안정하여 세밀한 구현 테크닉과 하이퍼파라미터 조정이 필요합니다. PPO-max는 Clipping, Initialization, GAE 등 학습 안정화 테크닉을 추가한 방법으로, 장기적 안정 학습을 실현합니다.

RLHF의 평가

평가 기준은 Honesty, Helpfulness, Harmlessness입니다. **Honesty** 평가에는 TruthfulQA와 HaluEval이 사용됩니다. TruthfulQA는 진실성을 평가하는 벤치마크로 건강, 법률, 금융, 정치 등 38개 카테고리에 걸친 817개 질문과 답변으로 구성되며, fine-tuning된 GPT-3를 이용해 평가를 자동화합니다. HaluEval은 환각을 인식할 수 있는지 평가하는 벤치마크로 ChatGPT가 환각을 일으키기 쉬운 데이터로 구성됩니다. **Helpfulness** 평가에는 Anthropic이 개발한 HH-RLHF가 자주 사용되는데, 클라우드 워커에 의해 수집되었으며 학습과 평가 모두에 활용됩니다. **Harmlessness** 평가에는 인종/피부색, 성별/성 정체성, 성적 지향, 종교, 연령, 국적, 장애, 외모, 사회경제적 지위의 9가지 편견에 관한 CrowS-Pairs와 젠더 바이어스에 관한 WinoGender가 사용됩니다. 포괄적 평가 도구 FLASK는 4가지 관점(Logical Thinking, Background Knowledge, Problem Handling, User Alignment)에서 12개 스킬을 GPT-4로 5단계 평가하며 FLASK dataset과 FLASK-HARD dataset이 준비되어 있고, 인간 평가와 GPT-4 평가가 유사한 경향을 보입니다. 또한 보상 모델이 올바르게 학습되었는지 포괄적으로 평가하는 RewardBench 벤치마크와 리더보드도 공개되어 있습니다.

RLHF의 과제

RLHF는 Human Feedback, Reward Model, Policy 각 부분에 과제를 안고 있으며, Reward Model과 Policy 양쪽에 공통되는 과제도 있습니다.

Human Feedback의 과제: (1) Misaligned Evaluators — 질 높은 labeler 선택이 어렵고, 유해한 편견을 가진 평가자나 데이터 오염 시도가 있을 수 있으며, RLHF가 "누구의 의견을 반영하는가"의 문제도 있습니다(RLHF 전에는 저소득·저학력과 일치하던 의견이 RLHF 후 역전되기도 함). (2) Difficulty of Oversight — 인간은 단순 실수를 저지르고 어려운 태스크를 평가할 수 없으며, 크라우드 워커의 33~46%가 LLM을 사용하는 것으로 추정됩니다(스스로 생각하는 것보다 LLM에게 생각하게 하면 API 대금을 지불하더라도 경제적 플러스가 되기 때문). (3) Data Quality — 데이터 수집 바이어스, 비용·품질 트레이드오프. 모델의 지식과 능력은 대부분 사전학습 시에 학습된다는 가정하에, 양질의 데이터를 소량이라도 모을 필요가 있습니다. (4) Feedback Type Limitations — 2개 쌍 ranking은 쉽지만 효율이 나쁘고, 언어 피드백은 품질 보장이 어렵습니다.

Reward Model의 과제: (1) Problem Misspecification — 개별 인간의 가치관을 보상 함수로 표현하기 어렵고 복수 의견 문제에 단일 스코어 매기기가 어렵습니다. (2) Misgeneralization/Hacking — 올바른 레이블에서라도 보상 해킹이 일어날 수 있으며, 과적합 시 특히 쉽습니다(스케일링 법칙 연구 있음). (3) Evaluation Difficulty — 보상 모델 평가 자체가 어렵습니다.

Policy의 과제: (1) RL Difficulties — 정책 최적화가 어렵고 적대적 악용(Jailbreak, 예: GPT-4 DAN attack)이 가능합니다. (2) Policy Misgeneralization — 최적 RL 에이전트는 권력을 추구하는 경향이 있습니다. (3) Distributional Challenges — 모드 붕괴(다양성 상실)나 사전 모델 바이어스 강화가 일어날 수 있습니다(GPT-4는 RLHF 후 자신 있게 틀리는 경우가 늘어났습니다).

Reward Model과 Policy를 동시에 학습하면 데이터 분포 변화가 유발됩니다. 온라인 학습에서는 보상 모델-정책 간 순환 영향이, 오프라인 학습에서는 보상 모델 바이어스로 잘못된 일반화 가능성이 있습니다.

RLHF 과제에 대한 대책

Human Feedback 대책으로는 문장별 보상 추정이나 3개 보상 모델(사실 부정확, 관련성 없음, 정보 불완전) 학습 등 상세 피드백 설계가 있습니다. Reward Model 대책으로는 복수 관점에서 학습된 보상 모델 파라미터를 섞는 Model Soup 방식(파레토 최적 alignment 지향)이 있습니다. Policy 대책으로는 복수 모델 출력으로 순위를 매겨 SFT하는 RRHF(PPO 간소화 기법), 보상 모델 상위 100/k%를 필터링해 PPO 없이 동등 이상 성능을 내는 RAFT, 유해·무해 프롬프트로 자동 보상 할당하는 RLCD, 랭킹에 이유를 추가해 fine-tuning하는 Chain of Hindsight(CoHF) 등이 제안되었습니다.

RLHF 구현 라이브러리로는 trl(HuggingFace, PPO 기반), trlx(CarperAI, PPO·ILQL), RL4ML(다양한 RL 알고리즘), DeepSpeed Chat(GPU 1대로 100억 파라미터, 복수 GPU로 1000억 파라미터, SoTA 15배 빠른 학습)이 있습니다. 자주 사용되는 데이터셋은 HH-RLHF(chosen/rejected 랭킹), OpenAssistant(oasst1, oasst2), HelpSteer, Uni-RLHF, UltraFeedback 등이며 기본적으로 영어가 대부분입니다.

DPO: 보상 모델 없이 직접 선호도를 학습하는 온라인먼트 기법

DPO의 기초와 이론

DPO(Direct Preference Optimization)는 Reward Model을 거치지 않고 직접 Preference를 고려한 최적화를 수행합니다. Reward Model은 암묵적으로 정의되며, 결과적으로 "보상 모델 학습 + 강화학습"이 "지도 학습만"과 동등해집니다. 보상 추정이 틀린 만큼 가중치를 부여하며 $\pi(yw|x)$ 우도를 최대화하고 $\pi(y_l|x)$ 우도를 최소화합니다.

DPO와 RLHF는 근사나 가정 없이 수학적으로 동등함이 보여졌습니다(증명은 Appendix).

//

$$\text{Loss_DPO}(\theta) = -E_{(x,yw,y_l \sim D)} [\log \sigma(\beta \cdot \log(\pi\theta(yw|x) / \pi^{\text{SFT}}(yw|x)) - \beta \cdot \log(\pi\theta(y_l|x) / \pi^{\text{SFT}}(y_l|x)))]$$

증명은 RLHF 목적 함수로부터 출발합니다.

//

$$\max_{\pi} E_{(x,y \sim D_{\pi})} [r(x, y) - \beta \cdot D_{\text{KL}}[\pi(y|x) || \pi^{\text{SFT}}(y|x)]]$$

이 문제의 최적해는 해석적으로 풀 수 있습니다.

//

$$\pi^*(y|x) = (1/Z(x)) \cdot \pi^{\text{SFT}}(y|x) \cdot \exp((1/\beta)r(x, y)) \quad Z(x) = \sum_y \pi^{\text{SFT}}(y|x) \exp((1/\beta)r(x, y))$$

즉 보상 $r(x, y)$ 를 구하면 최적 정책 $\pi^*(y|x)$ 가, 정책 $\pi(y|x)$ 가 구해지면 보상 $r(x, y)$ 가 구해집니다.

//

$$r(x, y) = \beta \cdot \log(\pi(y|x) / \pi^{\text{SFT}}(y|x)) + \beta \cdot \log Z(x)$$

이를 보상 모델 학습 손실 함수에 대입하면 분배 함수가 사라집니다. 분배 함수 $Z(x)$ 는 모든 y 에 대한 총합이라 본래 계산 불가능하지만, 대입하면 소거됩니다.

//

$$\text{loss}(\theta) = -E_{(x, yw, yl \sim D)}[\log(\sigma(r\theta(x, yw) - r\theta(x, yl)))] = -E_{(x, yw, yl \sim D)}[\log \sigma(\beta \cdot \log(\pi\theta(yw|x) / \pi^{\text{SFT}}(yw|x)) - \beta \cdot \log(\pi\theta(yl|x) / \pi^{\text{SFT}}(yl|x)))]$$

보상 모델 $r\theta(x, y)$ 을 $\beta \cdot \log(\pi\theta(y|x) / \pi^{\text{SFT}}(y|x))$ 로 간주하고 있다고 해석할 수 있습니다. 논문 제목 "Your Language Model is Secretly a Reward Model"이 이를 가리킵니다.

DPO의 파생 기법

DPO를 일반화한 ψ PO는 $\psi: \{0,1\} \rightarrow \mathbb{R}^+$ 인 비감소 함수를 도입해 목적 함수를 최소화하며, ψ 를 특정 형태로 두면 DPO와 같아집니다. $\psi = q$ 라는 항등 함수를 사용한 경우를 IPO(Identity Preference Optimization)라 합니다.

KTO(Kahneman-Tversky Optimization)는 전망 이론(prospect theory)에 기반해 인간의 효용 모델을 도입합니다(예: "5만엔을 얻은 기쁨보다 잃은 슬픔이 더 크다"). (x, yw, yl) Preference 데이터가 필요 없고 단일 쌍 (x, y) 만으로 학습 가능합니다.

DPO, ψ PO/IPO, KTO 등은 데이터셋과 보상 함수 가정을 변경한 기법들입니다. DPO가 가장 성능이 높지만, 학습 비용을 줄이려면 KTO나 CPO가 권장됩니다. 한편 "현 시점에서 DPO는 PPO에게 이길 수 없다"는 평가도 있으며, $\text{PPO} > \text{filtered DPO} / \text{iterative DPO} > \text{DPO} > \text{SFT}$ 의 순위로 알려져 있습니다. 그 이유는 PPO를 이용함으로써 Reward Model의 외삽 데이터에 접근할 수 있기 때문으로 추정됩니다.

DPO 계열 Reverse KL의 근본 문제로 $\exp((1/\beta)r(x, y))$ 항이 지수 함수적으로 SFT 분포를 뽕족하게 만들어 출력 다양성이 훼손되는 현상이 있으며, 이를 해결하기 위해 KL-divergence 대신 f-divergence를 이용하는 연구도 존재합니다. 이 밖에 Iterative/Online DPO, Self-Rewarding, Token-level DPO, TDPO, ORPO, SimPO, NPO, CPO, Nash Learning(SPPO, DNO) 등 다양한 파생 기법이 제안되어 있습니다.

기타 얼라인먼트 기법

Stable Alignment는 모의 인간 사회 샌드박스에서 에이전트끼리 대화하며 다양한 관점의 답변을 생성합니다. AlpacaFarm은 "인간이 어떤 평가를 반환할 것인가"를 시뮬레이션해 저렴하고 빠르게 RLHF를 진행하는 도구로, 실제 인간 평가 대비 1/45 비용으로 동등한 평가가 가능하다고 주장합니다.

발전 기법은 여러 축으로 분류됩니다. Feedback 원천별: Human Feedback(RLHF, RAFT, RRHF) vs AI Feedback(RLCD, Stable Alignment, AlpacaFarm, RLAIF, Constitutional AI). 학습 방식별: Rank-based(DPO, PRO, RRHF, SLiC) vs Language-based(CoH, Second Thoughts, Stable Alignment, Selfee). RL 사용 여부별: Using RL(RLHF, RLCD, RLAIF) vs Not Using RL(DPO, IPO, KPO, CPO, PRO, RRHF, RAFT).

발전적 의제

RLHF를 둘러싼 근본 질문들이 남아 있습니다. (1) 왜 RLHF로 성능이 올라가는가 — 실제로는 사전학습에서 얻은 분포를 의도에 맞는 출력으로 변화시키고 있을 뿐일 수 있습니다. (2) RL은 정말 필요한가 — DPO, PRO, RLCD 등 RL 미사용 방법이 동등 이상의 성능을 내고 있어, 아마도 RL은 필요하지 않을 수 있습니다. (3) SFT vs RLHF — 어느 정도는 SFT로 충분하지만 나머지 1% 제어에는 필요합니다. (4) RLHF vs RLAIF — AI Feedback에서는 Feedback 원천 모델 성능을 넘어서기 어렵지만, RLAIF(Constitutional AI 등)나 외부 도구 활용 RLCF(reinforcement learning from computational feedback) 방향으로 발전할 것으로 보입니다.

RLVR: 검증 가능한 보상으로 추론 능력을 향상시키는 강화학습

LLM 추론의 한계와 경험의 시대

대규모 언어모델은 박사 과정 수준 지식 문제에서 인간을 뛰어넘고 수학 올림피아드 문제도 풀 수 있지만, 한편으로는 간단한 계산 문제를 틀리기도 합니다. **이중 과정 이론(Dual Process Theory)**은 카네만과 트버스키가 널리 알려진 이론으로, 인간 사고가 두 시스템으로 작동한다고 가정합니다. 시스템 1은 직관·번뜩임으로 무의식적 판단하는 사고, 시스템 2는 수학·논리적 사고로 신중하게 천천히 생각하는 사고입니다. LLM은 시스템 1은 잘하지만 시스템 2는 서투릅니다.

시스템 1적 사고는 과거 경험·지식에 기반한 패턴 매칭이며 빠르게 보간하는 프로세스입니다. 사전학습된 LLM은 **보간형 데이터베이스(interpolative database)**처럼 행동하며, 많은 휴리스틱(경험칙)을 학습하고 조합해 동작하지만 근본적 인과 구조를 학습하고 있는 것은 아닙니다.

사전학습 스케일링 법칙의 시대는 끝났는가 하는 질문이 제기됩니다. 컴퓨팅은 향상되지만 데이터는 고갈되어 가며, GPT-4.5는 모델을 키웠음에도 큰 성능 향상에 이르지 못했습니다. 근년 AI는 "휴먼 데이터의 시대"에 있었으나, 인간의 방대한 데이터를 학습하는 것만으로는 "인간을 뛰어넘는" 초인적 지능에 도달하기 어렵습니다. 이 한계 돌파를 위해 "경험의 시대"로의 전환이 필요하며, AI 자신이 시행착오하고 그 결과로부터 학습하도록 하는 방법이 강화학습입니다.

추론 스케일링은 성공적으로 입증되었습니다. 매우 긴 Chain-of-Thought를 수행하도록 강화학습함으로써 OpenAI o1이나 DeepSeek R1은 추론 시 깊이 생각할수록 성능 향상에 기여했으며, LLM이 시스템 2적 사고를 손에 넣어 추론 스케일링 시대의 계기가 되었습니다.

RLVR의 개요와 DeepSeek R1의 성공

RLVR(Reinforcement Learning with Verifiable Reward)은 검증 가능한 보상 모델로 강화학습을 수행합니다. 수학은 최종 출력이 맞는지, 코드는 실행 결과가 맞는지·테스트가 통과하는지로 보상을 계산합니다. RLHF와 달리 보상 모델 학습이 불필요합니다. 보상 계산 방법으로 Math-Verify, LLM-as-a-judge for facts, Code Sandboxes 등이 사용됩니다. o1 이후 다양한 모델에 RLVR이 응용되고 있으며 수학·코드 등 깊은 사고가 필요한 태스크에서 비약적 성능 향상을 보여주고 있습니다.

DeepSeek R1은 오픈소스로 처음으로 o1에 필적하는 성능을 내었으며, 독자적 강화학습 알고리즘인 GRPO를 제안해 성능 향상에 기여했습니다.

GRPO와 그 발전판

GRPO(Group Relative Policy Optimization)는 PPO를 추론 향상에 특화시킨 기법입니다. 어드밴티지(A)를 에피소드 보상(r)로부터 직접 산출함으로써 상태 가치 $V(s)$ 함수 근사를 불필요하게 만들었다는 것이 핵심입니다. 가치 함수 학습 대신 동일한 프롬프트에 대한 복수 응답의 통계로 베이스라인을 계산합니다. PPO로부터의 clipping logic, loss 내 KL penalty, 동일 프롬프트의 샘플 그룹 보상 r_i 상대 할당, 토큰 길이 정규화를 조합합니다.

RLVR에 의해 흥미로운 *****아하 체험*****이 관찰됩니다. 자신의 시행착오 결과가 틀렸을 때 "Wait, wait. That's an aha moment"라며 올바른 풀이를 깨닫는 현상입니다. 또한 강화학습으로 Verification, Backtrack 행동이 증가하며 스코어도 향상됩니다. RL 전 모델에서 이 두 행동이 보이지 않으면 RL을 해도 성능이 향상되지 않습니다.

GRPO의 두 문제점: (1) **Length normalization bias** — 토큰 길이로 정규화되어 길이를 길게 하는 쪽이 페널티를 받기 어려워 학습이 진행되면 생성 길이가 길어집니다. (2) **Question-level difficulty bias** — 어드밴티지를 표준 편차로 나누어 극단적 난이도 문제에서 더 높은 가중치가 부여됩니다.

발전판 Dr. GRPO는 길이 정규화 없이 토큰 손실을 합산한 뒤 그룹 수로만 평균을 내어 길이 바이어스를 줄이며, 더 짧은 길이로 높은 스코어를 달성합니다. DAPO는 동적 샘플링과 확장 클리핑으로 탐색성·안정성을 높였습니다. 길이 정규화 방법에 따라 Default GRPO(시퀀스 길이 정규화, 안정적이나 편향 큼), Dr. GRPO(정규화 없음, 편향 없으나 불안정), DAPO(전체 토큰 수 정규화, 질문별 편향)로 차이가 납니다.

SFT는 기억하고 RL은 일반화를 촉진한다

토이 태스크에서 SFT와 RL을 비교한 결과, 분포 내 데이터에서는 SFT가 강하고 분포 외에서는 RL이 우수합니다. 즉 SFT는 기억, RL은 일반화를 촉진한다는 것이 시사됩니다. RLVR에 의해 새로운 사고가 획득되는가에 대해서는, 사전학습 모델에 존재하는 사고 패턴을 강화하고 있을 가능성이 제기되었습니다 (pass@k가 사전학습 모델에 가까워짐). 반면 ProRL(Prolonged RL)은 엔트로피 붕괴(출력 분포가 다양성을 잃고 엔트로피가 급격히 저하되는 현상)를 방지해 베이스 모델에서 도달 불가능한 새 추론 전략을 학습할 수 있다고 주장합니다.

다양성을 유지한 채 강화학습하는 Pass@K Training(SimKO 등)은 학습 중 탐색을 촉진해 베이스 모델에 없던 추론 전략을 획득하고, 엔트로피 붕괴를 방지하며 Pass@1 성능도 향상시킵니다. 단 1개 훈련 샘플로도 데이터셋 전체 사용과 동등한 성능을 달성했다는 연구도 있습니다.

RLVR의 향후 과제

(1) 일반화에 SFT가 좋은가, RL이 좋은가, 양쪽인가 — SFT로 새 사고 패턴을 학습시키고 RL로 강화하는 방식, 지속적 학습 기법이 이상적입니다. (2) 사전학습 모델은 무엇을 사용해야 하는가 — Qwen Family는 RL로 비약적 향상을 보이지만 Llama Family는 상대적으로 낮으며, 사전학습 분포에 따라 RL 효과가 다릅니다. (3) 보상은 결과에만 주는가, 중간 결과에도 주는가 — 도중 과정 보상이 효율적이겠지만 현재는 최종 결과 보상이 더 나은 성능을 보입니다. (4) 검증 불가능한 태스크에서의 추론 — 수학·코드 외의 영역으로의 확장이 과제입니다.

강화학습의 응용과 향후 방향

강화학습은 언어 영역을 넘어 확장되고 있습니다. **멀티모달 태스크**: 이미지·동영상 입력 추론 모델, 3D 씬 이해 향상(3D-R1 등). **에이전트 태스크**: WebAgent, GUI-Agent, OS 조작 강화학습(GUI-R1 등)으로 태스크 성공률 향상. **로봇 태스크**: VLA(Vision-Language-Action Model) 강화학습으로 장기 태스크 성능 비약적 향상, SFT Model에 없던 새 행동 학습(SimpleVLA-RL 등).

향후 방향: (1) **효율적 추론** — 강화학습으로 생성 길이가 길어져 간단한 문제에서도 길게 생각하거나 overthink 하는 문제를, 태스크 난이도에 따른 길이 제어로 해결해야 합니다. (2) **잠재 공간 추론** — 사고 과정을 토큰화하는 것은 비효율적이며(인간이 매번 말하면서 생각하는 것과 같음), 잠재 공간에서 추론해 추상적 사건을 효율적으로 계산해야 합니다. (3) **사전학습부터 강화학습** — 기존 사전학습은 대규모 데이터와 Next Token Prediction에 의존하여 교사 데이터 이상의 성능을 낼 수 없으나, 사전학습부터 RL을 수행하면 데이터 이상의 성능이 가능할 것으로 기대됩니다(Reinforcement Pre-Training).

정리

RLHF는 Alignment(인간의 의도대로 모델 학습)를 적용하는 한 방법으로, 인간의 피드백 데이터로 언어 모델을 강화학습합니다. DPO는 지도학습으로 Alignment를 적용하는 기법으로 RLHF와 수학적으로 동등합니다. RLVR은 검증 가능한 보상으로 추론 능력을 향상시키며, GRPO, DAPO, Dr. GRPO 등의 종류가 있습니다. 강화학습은 언어를 넘어 멀티모달, 에이전트, 로봇 등으로 응용 범위를 넓혀가며, 효율적 추론, 잠재 공간 추론, 사전학습 단계부터의 강화학습 등 새로운 방향으로 연구가 진행되고 있습니다.

References

- [1] DeepMind, "MuZero: Mastering Go, chess, shogi and Atari without rules", <https://deepmind.google/blog/muzero-mastering-go-chess-shogi-and-atari-without-rules/>
- [2] CNN.co.jp (2017), "AlphaGo 관련 기사", <https://www.cnn.co.jp/tech/35080140.html>
- [3] TrackingAI, "AI Progress Tracking", <https://www.trackingai.org/home>
- [4] Zhang, Kaiyan, et al. (2025), "A Survey of Reinforcement Learning for Large Reasoning Models", arXiv:2509.08827
- [5] OpenAI (2022), "Instruction Following", <https://openai.com/research/instruction-following>
- [6] Zhao, Wayne Xin, et al. (2023), "A Survey of Large Language Models", arXiv:2303.18223
- [7] Touvron, Hugo, et al. (2023), "Llama 2: Open foundation and fine-tuned chat models", arXiv:2307.09288
- [8] HuggingFace, "Math-Verify", <https://github.com/huggingface/Math-Verify>
- [9] Lambert, Nathan, et al., "RLHF Book Chapter 14: Reasoning", <https://rlhfbook.com/c/07-reasoning>
- [10] ARC Prize, "ARC Prize Leaderboard", <https://arcprize.org/leaderboard>
- [11] 일본경제신문(2021), "한국에서 '대화AI' 폭주 — 머신러닝이 빠진 함정", <https://www.nikkei.com/article/DGXZQOGM21B9V0R20C21A1000000/>
- [12] Wolfe, Cameron R. (2023), "Specialized LLMs: ChatGPT, LaMDA, Galactica, Codex, Sparrow, and More", <https://cameronrwolfe.substack.com/p/specialized-llms-chatgpt-lamda-galactica>

-
- [13] Stanford Online (2023), "CS25 I Stanford Seminar – Transformers United 2023: Language and Human Alignment", <https://www.youtube.com/watch?v=DJ1Yy6Aquug>
- [14] Anthropic, "hh-rlhf Dataset", <https://huggingface.co/datasets/Anthropic/hh-rlhf>
- [15] Li, Junyi, et al. (2023), "HaluEval", arXiv:2305.11747
- [16] Nangia, Nikita, et al. (2020), "CrowS-Pairs", arXiv:2010.00133
- [17] Wang, Yuxia, et al. (2023), "Do-Not-Answer: A Dataset for Evaluating Safeguards in LLMs", arXiv:2308.13387
- [18] OpenAI (2023), "Weak-to-Strong Generalization", <https://openai.com/index/weak-to-strong-generalization/>
- [19] Wikipedia, "Existential risk from artificial intelligence", https://en.wikipedia.org/wiki/Existential_risk_from_artificial_intelligence
- [20] CNET Japan (2018), "AI 자율주행차, '강화학습'으로 운전 방법을 20분 만에 습득", <https://japan.cnet.com/article/35122203/>
- [21] CNET Japan (2017), "AI 관련 기사", <https://japan.cnet.com/article/35094593/>
- [22] Boston Dynamics, <https://bostondynamics.com/>
- [23] OpenAI, "ChatGPT", <https://chatgpt.com/>
- [24] BrainPad (2023), "ChatGPT의 구조를 논문 기반으로 초상세하게 해설", <https://blog.brainpad.co.jp/entry/2023/05/31/160719>
- [25] zero2one, "정책 경사법(Policy Gradient Methods)", <https://zero2one.jp/ai-word/policy-gradient-methods/>
- [26] OpenAI (2017), "Learning from human preferences", <https://openai.com/index/learning-from-human-preferences/>
- [27] Stiennon, Nisan, et al. (2020), "Learning to Summarize from Human Feedback", arXiv:2009.01325
- [28] Ouyang, Long, et al. (2022), "Training Language Models to Follow Instructions with Human Feedback", arXiv:2203.02155
- [29] Rafailov, Rafael, et al. (2023), "Direct Preference Optimization: Your Language Model is Secretly a Reward Model", arXiv:2305.18290

-
- [30] Azar, Mohammad Gheshlaghi, et al. (2023), "A General Theoretical Paradigm to Understand Learning from Human Preferences", arXiv:2310.12036
- [31] Ethayarajh, Kawin, et al. (2024), "KTO: Model Alignment as Prospect Theoretic Optimization", arXiv:2402.01306
- [32] Saeidi, Amir, et al. (2024), "Insights into Alignment: Evaluating DPO and its Variants Across Multiple Tasks", arXiv:2404.14723
- [33] Ivison, Hamish, et al. (2024), "Unpacking DPO and PPO: Disentangling Best Practices for Learning from Preference Feedback", arXiv:2406.09279
- [34] Wang, et al. (2024), "Beyond Reverse KL: Generalizing Direct Preference Optimization with Diverse Divergence Constraints", ICLR 2024, arXiv:2309.16240
- [35] Liu, et al. (2024), "Training Socially Aligned Language Models in Simulated Human Society"
- [36] Dubois, Yann, et al. (2023), "AlpacaFarm: A Simulation Framework for Methods that Learn from Human Feedback", arXiv:2305.14387
- [37] epoch.ai, "GPQA Diamond Benchmark", <https://epoch.ai/benchmarks/gpqa-diamond>
- [38] Chollet, François (2023), ARC Prize 관련 포스트, <https://x.com/fchollet>
- [39] Nikankin, A., et al. (2025), "Arithmetic without algorithms: Language models solve math with a bag of heuristics", arXiv:2410.21272
- [40] Sutskever, Ilya (2024), "Sequence to Sequence Learning with Neural Networks", NeurIPS 2024
- [41] OpenAI (2025), "GPT-4.5 System Card", <https://cdn.openai.com/gpt-4-5-system-card-2272025.pdf>
- [42] OpenAI (2024), "Learning to Reason with LLMs", <https://openai.com/index/learning-to-reason-with-llms/>
- [43] DeepSeek-AI, Guo, Daya, et al. (2025), "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning", arXiv:2501.12948
- [44] Silver, David and Sutton, Richard (2025), "Welcome to the Era of Experience"
- [45] Shao, Zhihong, et al. (2024), "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models", arXiv:2402.03300

-
- [46] oxen.ai (2024), "Why GRPO is Important and How it Works", <https://www.oxen.ai/blog/why-grpo-is-important-and-how-it-works>
- [47] Gandhi, et al. (2025), "Cognitive Behaviors that Enable Self-Improving Reasoners, or, Four Habits of Highly Effective STaRs", arXiv:2503.01307
- [48] Liu, Zichen, et al. (2025), "Understanding R1-Zero-like Training: A Critical Perspective", arXiv:2503.20783
- [49] Yu, Qiyang, et al. (2025), "DAPO: An Open-Source LLM Reinforcement Learning System at Scale", arXiv:2503.14476
- [50] Chu, T., et al. (2025), "SFT Memorizes, RL Generalizes: A Comparative Study of Foundation Model Post-Training", arXiv:2501.17161
- [51] Yue, Y., et al. (2025), "Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model?", <https://openreview.net/pdf?id=4OsgYD7em5>
- [52] Liu, M., et al. (2025), "ProRL: Prolonged Reinforcement Learning Expands Reasoning Boundaries in Large Language Models", arXiv:2505.24864
- [53] Peng, Ruotian, et al. (2025), "SimKO: Simple Pass@K Policy Optimization", arXiv:2510.14807
- [54] Wang, Y., et al. (2025), "Reinforcement Learning for Reasoning in Large Language Models with One Training Example", arXiv 2025
- [55] Huang, Ting, et al. (2025), "3D-R1: Enhancing Reasoning in 3D VLMs for Unified Scene Understanding", arXiv:2507.23478
- [56] GUI-R1 (2025), "GUI-R1: A Generalist R1-Style Vision-Language Action Model For GUI Agents", arXiv:2504.10458
- [57] Li, et al. (2025), "SimpleVLA-RL: Scaling VLA Training via Reinforcement Learning", arXiv:2509.09674
- [58] Feng, et al. (2025), "Efficient Reasoning Models: A Survey", arXiv:2504.10903
- [59] Zhu, et al. (2025), "A Survey on Latent Reasoning", arXiv:2507.06203
- [60] Dong, et al. (2025), "Reinforcement Pre-Training", arXiv:2506.08007
- [61] Ye, Seonghyeon, et al. (2023), "FLASK: Fine-grained Language Model Evaluation based on Alignment Skill Sets", arXiv:2307.10928

-
- [62] Lambert, Nathan, et al. (2024), "RewardBench: Evaluating Reward Models for Language Modeling", arXiv:2403.13787
- [63] Casper, Stephen, et al. (2023), "Open Problems and Fundamental Limitations of Reinforcement Learning from Human Feedback", arXiv:2307.15217
- [64] Parrish, Alicia, et al. (2023), "Which Examples Should be Multiply Annotated? Active Learning When Annotators May Disagree", ACL Findings 2023
- [65] Veselovsky, Veniamin, et al. (2023), "Artificial Artificial Artificial Intelligence: Crowd Workers Widely Use Large Language Models for Text Production Tasks", arXiv:2306.07899
- [66] Zhou, Chunting, et al. (2023), "LIMA: Less Is More for Alignment", arXiv:2305.11206
- [67] Shen, Mingyang, et al. (2023), "Bridging the Gap: A Survey on Integrating (Human) Feedback for Natural Language Generation", arXiv:2305.00955
- [68] Gao, Leo, et al. (2022), "Scaling Laws for Reward Model Overoptimization", arXiv:2210.10760
- [69] Wei, Alexander, et al. (2023), "Jailbroken: How Does LLM Safety Training Fail?", arXiv:2307.02483
- [70] OpenAI (2023), "GPT-4 Technical Report", arXiv:2303.08774
- [71] Wu, Zeqiu, et al. (2023), "Fine-Grained Human Feedback Gives Better Rewards for Language Model Training", arXiv:2306.01693
- [72] Rame, Alexandre, et al. (2023), "Rewarded soups: towards Pareto-optimal alignment by interpolating weights fine-tuned on diverse rewards", arXiv:2306.04488
- [73] Yuan, Zheng, et al. (2023), "RRHF: Rank Responses to Align Language Models with Human Feedback without tears", arXiv:2304.05302
- [74] Irpan, Alex (2018), "Deep Reinforcement Learning Doesn't Work Yet", <https://www.alexirpan.com/2018/02/14/rl-hard.html>
- [75] Zheng, Rui, et al. (2023), "Secrets of RLHF in Large Language Models Part I: PPO", arXiv:2307.04964
- [76] Dong, Hanze, et al. (2023), "RAFT: Reward rAnked FineTuning for Generative Foundation Model Alignment", arXiv:2304.06767

-
- [77] Yang, Kevin, et al. (2023), "RLCD: Reinforcement Learning from Contrast Distillation for Language Model Alignment", <https://github.com/facebookresearch/RLCD>
- [78] Liu, Tianhao, et al. (2023), "Chain of Hindsight Aligns Language Models with Feedback", arXiv:2302.02676
- [79] Peng, Baolin, et al. (2023), "Stabilizing RLHF through Advantage Model and Selective Rehearsal", arXiv:2309.10202
- [80] Lin, Stephanie, et al. (2022), "TruthfulQA: Measuring How Models Mimic Human Falsehoods", ACL 2022
- [81] Li, Junyi, et al. (2023), "HaluEval", EMNLP 2023
- [82] Bai, Yuntao, et al. (2022), "Training a Helpful and Harmless Assistant with RLHF", arXiv:2204.05862 (Anthropic HH-RLHF Dataset)
- [83] Nangia, Nikita, et al. (2020), "CrowS-Pairs", EMNLP 2020
- [84] Rudinger, Rachel, et al. (2018), "Gender Bias in Coreference Resolution", <https://github.com/rudinger/winogender-schemas>
- [85] Wang, Zhichao, et al. (2024), "A Comprehensive Survey of LLM Alignment Techniques: RLHF, RLAI, PPO, DPO and More", arXiv:2407.16216

08

Day 8 학습 데이터와 평가 벤치마크의 정비

이 장의 구성

- 01 도입: 학습 데이터와 평가 벤치마크 정비의 목표
- 02 사전학습 코퍼스의 종류와 구성
- 03 사전학습 데이터의 필터링과 확장
- 04 사후학습 데이터: Instruction Tuning과 강화학습
- 05 라이선스, 저작권, 개인정보
- 06 LLM 성능 평가와 벤치마크
- 07 정리
- 08 References

"

본 글은 도쿄대학교 마츠오·이와사와 연구실이 2025년 10~11월에 개최한 「대규모 언어 모델 강좌 2025 기초편」 Day 8 강의 자료를 재구성한 것이다. 이론: 층지에(曾傑, Jie Zeng), 실습: 에쿠니 쇼타(江國翔太). 원본은 CC BY-NC-SA 4.0로 공개되었으며, 본 재 구성본은 CC BY-NC-ND 4.0로 공개한다.

도입: 학습 데이터와 평가 벤치마크 정비의 목표

이 강의 이론을 담당한 층지에(曾傑, Jie Zeng)는 2023년 세이케이대학교 이공학연구과 박사후기과정을 수료하고 같은 대학 이공학부 특별공동연구원으로 재직 중이다. GENIAC 마츠오 연구실 LLM 개발 프로젝트 Phase 1·2에서 학습 데이터 정비를 담당했고 민감정보 필터링 모델 개발에 참여했으며, 연구 분야는 LLM을 활용한 도메인 대화(인터뷰·상담) 시스템 구현이다.

LLM 학습 파이프라인은 여섯 단계로 이루어진다. 사전학습은 대규모 코퍼스를 이용한 자기지도학습을 통해 언어 모델에 어휘·문법·지식과 같은 기본적인 언어 이해를 획득시키는 단계이고, 파인튜닝(SFT)은 레이블이 있는 데이터를 이용한 지도학습을 통해 언어 모델의 성능을 개선하거나 특정 태스크나 도메인에 대한 적응을 실현하는 단계이며, 강화학습(RLHF·DPO 등)은 인간의 피드백을 활용해 언어 모델의 출력이 인간의 가치관에 부합하도록 조정하는 단계다. 파인튜닝과 강화학습을 통틀어 '사후학습(Post-Training)'이라 부른다. 이 세 학습 단계에 앞서 데이터 수집·가공 단계가 있으며(최근에는 LLM 자체를 활용한 데이터 합성도 활발하다), 학습 뒤에는 추론 단계(프롬프트 엔지니어링 활용)와 벤치마크 평가 단계(학습에 사용되지 않은 샘플로 구성된 벤치마크 이용)가 이어진다. 이 파이프라인 어느 단계에서든 데이터와 평가는 핵심이다. LLM은 데이터로부터 지식과 능력을 학습하는 만큼 데이터의 질과 양이 모델 성능을 좌우하고, 모델의 일반화 능력이 비약적으로 향상된 지금 어떤 평가를 어떻게 수행해야 하는지도 과제다.

언어 모델의 규모와 학습 데이터량은 시대에 따라 급격히 증가했다. 2018~2019년에는 BERT가 약 3,200M words[2], GPT-2가 40GB[3] 수준이었으나, 2020~2022년에는 GPT-3가 500B Token·570GB[4], Megatron-Turing NLG 530B가 339B Token[5], PaLM이 780B Token[6]으로 확장되었다[1]. 최신 모델일수록 사용하는 데이터량이 증가하며, 특히 학습에 코드(code)를 포함하는 경우가 많아 추론 능력 향상에 기여할 가능성이 시사된다. 코드가 없는 GPT-3보다 코드가 포함된 code-davinci-002 모델이 추론 능력이 더 높다는 관찰이 그 예시다[11].

이 장의 목표는 세 가지다. 첫째, 대규모 언어 모델의 학습 데이터에 관한 종류와 정비 방법, 그리고 학습 데이터에 사용되는 발전적 기술을 설명할 수 있다. 둘째, 대규모 언어 모델을 평가하기 위한 자원과 발전적 기법을 설명할 수 있다. 셋째, 목적과 내용을 충분히 이해한 바탕 위에서 실제로 이들을 구현하고 대규모 언어 모델의 성능 평가를 실현할 수 있다.

사전학습 코퍼스의 종류와 구성

사전학습 코퍼스는 모델 성능에 큰 영향을 미치므로 광범위한 내용을 다루는 대량의 고품질 데이터가 강력히 요구된다. 일반화 능력을 높이기 위해 웹 페이지, 서적, 대화 데이터 등 범용 데이터를 활용하며, 특정 영역의 성능을 부여하기 위해 특정 도메인의 데이터셋을 추가하기도 한다.

일반 텍스트 데이터

웹 페이지는 다양한 정보를 포함하지만 품질 편차가 커서 필터가 필요하다. CommonCrawl은 웹에 있는 페이지를 크롤링(수집)하여 아카이브로 제공한다. C4(800GB)는 상투적 표현("메뉴", "로그인"), 스팸, 짧은 문장을 필터링하여 추출한 데이터셋이며 다국어판 mC4도 존재한다. Wikipedia(21GB)는 백과사전으로서 고품질 텍스트를 제공한다. RefinedWeb[7]은 CommonCrawl을 기반으로 고품질 필터 처리를 실시하여 공개 600GB를 제공한다.

대화 텍스트는 LLM의 대화 능력을 향상시키고 질의응답 태스크의 성능 개선을 기대할 수 있다. Reddit 같은 게시판 사이트는 다수 참여자 간의 논의이므로, 대화를 트리 구조화하고 응답 쌍으로 만든 여러 하위 대화로 분할하는 처리를 수행한다.

서적은 다른 코퍼스에 비해 격식 있고 긴 글이므로, LLM이 언어 지식과 긴 문맥의 의존 관계, 서사적 일관성을 갖춘 텍스트 생성을 학습할 수 있다. Books3(100GB, Pile 데이터셋의 일부)는 소설과 논픽션 서적을 포함하나 저작권으로 보호된 서적의 사본이 포함되어 있을 가능성이 높아 위법성이 지적되고 있어 이용 시 법적 리스크를 수반한다. BooksCorpus2(6GB)는 미출판 소설로 구성된다.

특정 도메인의 텍스트 데이터

다국어 텍스트는 단일 언어뿐만 아니라 다국어 이해 및 생성 능력을 높인다. mC4는 다국어 CommonCrawl 데이터에서 정형화한 것이며, BLOOM[8] 데이터셋은 46개 언어를, CulturaX[9]는 167개 언어 6.3T token을 커버한다.

과학 텍스트는 과학적 지식 이해 향상과 과학적·추론 태스크에서의 뚜렷한 성능 달성을 기대할 수 있다. arXiv(논문), PubChem(화학 정보 컬렉션), OpenStax(peer-review된 대학 수준의 물리·화학·수학 교과서) 등이 활용된다.

코드는 코드 생성 목적의 LLM 개발에 필수적이며, 자연어에 비해 긴 문맥과 의존 관계, 정확한 논리라는 성질을 지닌다. 이는 LLM의 복잡한 추론 능력의 원천일 가능성이 시사된다[10]. GitHub(Pile 데이터셋 중 61GB), The Stack(3TB, 350개 이상의 프로그래밍 언어; MIT, Apache 등 라이선스를 가진 코드만 수집·정제), Stack Overflow(코드와 자연어로 구성된 Q&A) 등이 대표적이다.

사전학습 데이터의 필터링과 확장

RefinedWeb: 엄격한 데이터 선별 파이프라인

RefinedWeb[12]은 필터링 공학적 개선을 통해 대규모 데이터를 구축한 사례다. 웹 데이터로 구성된 5T Token 규모의 데이터셋을 작성하였고 600GB를 공개하였다. 복수의 필터링과 중복 제거를 조합한 엄격한 데이터 선별을 실시하여, 일련의 파이프라인에서 CommonCrawl 중 약 90%의 문서가 제거된다.

파이프라인은 세 단계로 구성된다. 첫째, 문장 준비 단계다. URL 필터링에서는 4.6M의 URL을 포함하는 도메인 블록리스트(성인 콘텐츠, 문장 형태가 아닌 텍스트/스팸, 파일 호스팅 사이트 등)를 이용하여 제거한다. URL에 출현하는 단어에 대한 판정도 수행하는데, 유해 단어 리스트를 strict, hard, soft의 수준으로 분할한다. strict와 hard 수준에 해당하는 단어는 URL 중 부분 일치, 완전 일치하면 제거한다. soft 수준에 해당하는 단어는 복수 출현하면 제거 대상이 되지만 단독 출현(예: ass)이면 제거하지 않는다. 이는 의료·법률적 콘텐츠까지 제거 대상이 되지 않도록 하기 위함이다. 텍스트 추출에서는 메뉴, 헤더, 푸터, 광고 등을 무시하고 페이지의 주요 콘텐츠만 추출하며, 추출 라이브러리 Trafilatura와 정규표현식을 사용하여 줄바꿈은 연속 2회까지, 모든 URL을 삭제한다. 언어 식별에서는 RefinedWeb이 영어를 대상으로 하므로 Wikipedia 데이터로 n-gram 학습한 판정기를 이용한다. URL 필터링, 텍스트 추출, 언어 식별을 거쳐 원래 문서의 48%가 잔존한다.

둘째, 문장 단위·행 단위 필터링 단계다. 반복 제거에서는 문장 내에 반복 출현하는 문자열을 포함하는 문장이 최종 모델에 악영향을 미치므로[13], 문장 단위로 조기 검출하는 것이 비용 효율이 높다. 과도한 행 수, 단락, n-gram의 반복을 규칙 기반으로 제거한다[14]. 문장 단위 필터링에서는 키워드 리스트, 상투적 표현, 특수문자 연속으로 이루어진 기계 생성 스팸이 페이지의 큰 비율을 차지하므로 이를 제거 대상으로 한다. 또한 Rae et al.[14]의 휴리스틱 품질 필터링을 이용하여 문서 전체 길이, 기호와 단어의 비율, 문서가 실제 자연어인지를 보장한다. 단, 상기 필터를 영어 이외의 언어에 적용하면 과도하게 필터링되므로 언어별 적응이 필요하다. 행 단위 수정에서는 소셜 미디어의 "좋아요 3건", 내비게이션 버튼 등 문장에 여전히 섞여 있는 바람직하지 않은 행을 수정하는 규칙 기반 필터를 고안한다. 수정에 의해 문장의 5% 이상이 삭제될 경우 해당 문장을 삭제한다. 삭제 대상이 되는 행으로는 대문자가 많은 행, 숫자로만 구성된 행, 카운터("좋아요 3건"), 단어 1개로 구성된 행, 10문자 이하이면서 sign-in으로 시작하는 행 등이 있다. 문장 단위와 행 단위의 필터링에 의해 원래 문서의 23%가 잔존한다.

셋째, 중복 제거 단계다. 필터 후에도 크롤러에 의한 동일 페이지 복수 수집이나 상투적 콘텐츠(라이선스 문구, 표절 가능성도 있는)가 반복되는 사례가 존재한다. 중복적인 내용은 모델에 큰 영향을 미쳐 일반화 능력보다 기억 능력을 우선하게 한다[15, 16]. 퍼지(느슨한) 중복 제거에서는 MinHash를 사용하여 유사 문서를 제거한다. 이를 통해 템플릿화된 문장, 특정 엔티티만이 다른 라이선스 문장 등 중복률이 높은 페이지를 발견하여 삭제한다. 완전 중복 제거에서는 문장 레벨이 아니라 시퀀스 레벨에 대해 문자열 단위의 완전 일치 매칭(접미사 배열 사용)을 수행하여 특정 면책 조항이나 통지 등의 문자열을 제거한다. 리소스 제약상 텍스트 집합을 100개의 파트로 분할하여 파트 단위로 중복 제거를 실시한다(라이선스 문구나 일반적인 스팸이 제거된다). URL을 이용한 중복 제거에서는 크롤링 시의 콘텐츠(동일 URL) 재수집이 원인이 되어 CommonCrawl의 덤프 간에 존재하는 중복을 처리한다. 각 파트에서 전체 샘플의 URL 리스트를 작성하고 동일한 URL에 대해서는 처리를 건너뛰는다.

MinHash 알고리즘에 의한 중복 판정

MinHash는 텍스트 유사도 계산 기법인 Jaccard 계수를 효율적으로 추정하는 기법이다[17]. 문장 A, B의 MinHash가 일치할 확률이 Jaccard 계수와 같다는 점을 이용한다. Jaccard 계수는 $J(A, B) = |A \cap B| / |A \cup B|$ 로 정의된다. 예를 들어 A 문장 "I have a pen"(집합 {I, have, a, pen})과 B 문장 "I have an orange"(집합 {I, have, an, orange})의 Jaccard 계수는 $\{I, have\} / \{I, have, a, pen, an, orange\} = 2/6 = 1/3$ 이다.

처리 흐름은 다음과 같다. 먼저 문장을 r개의 버킷으로 분할한다. 각 버킷에 대해 k개의 해시 함수를 이용하여 k개의 해시를 얻는다. 그리고 적어도 1개의 버킷에 대해 MinHash가 일치하면 중복으로 취급한다. 구체적으로는 (1) 해시 함수 h로 집합의 각 원소를 해시값으로 변환하여 $h(A) = \{h(a_1), h(a_2), \dots, h(a_n)\}$, $h(B) = \{h(b_1), h(b_2), \dots, h(b_m)\}$ 을 얻고, (2) 집합 A, B의 해시값에 대해 최솟값(MinHash)을 취득하여 $h_{\min}(A) = \min(h(A))$, $h_{\min}(B) = \min(h(B))$ 를 구하면, (3) 이때 $P(h_{\min}(A) = h_{\min}(B)) = Jaccard(A, B)$ 가 성립한다.

FineWeb과 모델 기반 필터링

FineWeb[18]은 CommonCrawl에 대해 RefinedWeb의 필터링을 수행함으로써 벤치마크에서 성능이 향상됨을 보였다. 검증에는 Llama 구조의 1.71B 파라미터 모델을 사용하였고, 상식 관련 QA와 MMLU(57 종류의 태스크를 포함하며 지식과 문제해결 능력을 묻는 벤치마크) 등을 활용하였다.

FineWeb-Edu[18]는 교육 콘텐츠로 한정된 데이터셋이다. 초·중학교 수준의 교육적 내용인지 판정하는 회귀 모델을 이용하여 내용에 기반한 필터링을 실시한다. Llama3를 파인튜닝하여 교육적 내용의 점수(0~5)를 부여하는 회귀 모델을 작성하고, 점수가 3 이상인 문장을 추출하여 교육적 내용으로 구성된 1.3T Token 규모의 데이터셋을 작성하였다. 벤치마크 MMLU에서 기존 데이터셋의 1/10 데이터로 동등한 성능을 달성할 수 있었다.

최근에는 텍스트 품질에 기반한 필터가 다운스트림 태스크 성능 향상에 기여한다는 보고가 이어지고 있다. ASK-LLM[19]은 사전학습 데이터 필터링에 외부 LLM을 이용하는 접근으로, 프롬프트 중에 지시와 학습 데이터를 Zero-shot으로 주고 "yes"(유익한 데이터를 나타냄)의 출력 확률을 품질 점수로 간주한다. 검증에서 Pre-training 모델은 T5(encoder-decoder), 외부 LLM은 Instruction-tuning 완료된 FLAN-T5를 사용하였다. DataComp-LM[20]은 텍스트 품질을 평가하는 전용 모델을 작성하는 모델 기반 필터링을 제안한다. 좋음/나쁨의 이진 레이블이 부여된 400K 문서로 FastText 도구(sub-word 분할을 이용한 벡터를 다룬다)를 이용해 분류기를 학습하며, 제안 기법에 의한 데이터셋으로 학습한 LLM은 FineWeb-Edu의 성능을 능가한다.

데이터 확장 기법

고성능 LLM 생성에 방대한 양의 고품질 데이터가 필요하지만, 이용 가능한 데이터 자원에 한계가 있어 데이터 고갈이 우려된다. 이에 기존 데이터를 활용하면서 데이터 양의 확장(Data Augmentation)을 수행하는 연구가 진행되어 왔다. 확장 단위(입도)로는 Token, Token-span(연속되는 토큰 구간), Sentence(문장), Passage(문서의 일부나 특정 인용), Context(입력에 대한 응답 부분 등의 덩어리), Document(문서·글) 등이 있으며, 다양한 태스크(분류, 생성, 정보 추출 등)와 확장 단위별로 연구가 이루어졌다[21]. Chai et al.(2025)은 데이터 확장을 네 가지 카테고리로 분류하였다.

첫째, 단순한 확장이다. 텍스트 변환(일부 단어를 다른 단어로 치환)과 Back-translation(소스 언어를 다른 언어로 번역한 뒤 다시 소스 언어로 번역, 예: 일본어 → 영어 → 일본어) 등이 해당한다.

둘째, 프롬프트 기반 확장이다. 설계한 프롬프트를 LLM에 주어 LLM이 인간과 유사한 응답을 생성하도록 한다.

셋째, 검색 기반 확장이다. LLM은 환각(hallucination)과 외부 정보를 활용할 수 없다는 과제를 안고 있으므로, 외부 지식이나 문서를 동적으로 검색하고 검색 결과(새로운 정보)를 반영한 응답을 생성하는 RAG 구조를 이용한다.

넷째, 하이브리드 접근(프롬프트 × 검색 기반)이다. 복수 단계로 구성된 프롬프트와 검색된 정보를 적절히 사용한다. 예로 ReACT[22]에서는 CoT와 검색을 복수 단계로 실행하여 응답을 생성한다.

LLM에 의한 재작성(rewriting)을 활용한 사전학습 데이터 작성도 활발하다. Fujii et al.(2025)[23]은 수학과 코드의 성능 향상을 목적으로 사전학습용 데이터를 LLM에 의한 재작성 방식으로 작성하여 SwallowCode(16.1B Token)와 SwallowMath(2.3B Token) 데이터셋을 구축하였다. 코드의 경우 Llama3.3-70B-Instruct를 이용하여 (1) 타입 힌트나 코드 문서화 등 코드 스타일 개선, (2) 알고리즘·자료구조적 최적화를 수행하도록 프롬프팅하여 데이터를 재작성한다. 출력 포맷을 지정하고 평가 항목에 대한 설명과 재작성 규칙을 프롬프트에 포함한다.

사후학습 데이터: Instruction Tuning과 강화학습

Instruction Tuning 데이터셋

Instruction Tuning은 지시문(instruction)을 입력으로 하고 이상적인 응답문을 출력으로 하는 지도학습으로, 다양한 태스크가 입출력 형식으로 표현된다[11, 24]. 입력에는 태스크 기술(Instruction), 부가적인 입력 정보, 소량의 입출력 예시나 CoT 예시가 포함될 수 있다. Instruction Tuning 데이터셋을 구축하는 주요 기법 세 가지가 있다.

첫째, 기존 NLP 태스크 데이터셋의 이용이다. 텍스트 분류나 요약과 같은 NLP 태스크의 데이터셋을 사용하여 입출력 형식을 정형화한다. 다양한 입력에 대응할 수 있도록 사람이 직접 작성한 템플릿을 복수로 작성한다(P3 데이터셋[25], FLAN 데이터셋[24]).

둘째, 사용자 쿼리를 포함한 대화 형식 데이터의 이용이다. 사용자가 LLM을 사용할 때의 쿼리를 수집하여 Instruction Tuning 데이터 일부로 활용한다. ShareGPT[26]는 API 쿼리 공유 플랫폼에 업로드된 ChatGPT, GPT-4와의 대화 9만 건을 사용하며 응답은 LLM이 생성한다. Dolly[27]는 브레인스토밍, 정보 추출 등 7개 도메인을 커버한 사람에 의한 데이터(입력-출력) 1.5만 건을 작성하였다.

InstructGPT[28]는 사용자 쿼리에 더해 사람 라벨러에게 태스크(instruction)를 작성하게 하고 다른 라벨러에게 그 답변 작성을 의뢰하였다. 라벨러에 대한 프롬프트 작성 의뢰는 세 종류로 나뉜다. Plain은 다양한 태스크를 망라하기 위해 라벨러에게 생각나는 태스크를 적어달라고 하는 것이고, Few-shot은 지시문과 그 지시문에 대한 복수의 쿼리/응답 쌍을 생각해 달라고 하는 것이다(예: 지시문 "트윗의 감정을 판정하라", 쿼리는 트윗, 응답은 "긍정"/"부정"). User-based는 복수의 유스케이스를 제시하고 유스케이스에 대응하는 프롬프트(지시문)를 생각해 달라고 하는 것이다.

셋째, 합성 데이터의 이용이다. 인간이 작성한 Instruction 데이터에 의존하면 인건수 비용, 다양성, 창의성에 한계가 있으므로, LLM 스스로 데이터를 만들어내는 접근이 필요하다. SELF-INSTRUCT[29]는 소량의 instruction 데이터를 시드로 삼아 LLM을 사용해 (i) 태스크를 생성하고 (ii) 그에 기반해 데이터(instance)를 생성하는 기법을 제안한다. 처리는 네 단계로 이루어진다. Step 1에서는 Few-shot으로 태스크(instruction)를 생성한다. Step 2에서는 생성된 태스크가 분류 문제인지 판별한다. Step 3에서는 태스크에 대응하는 답변을 작성(instance)하는데, 분류 태스크의 경우 클래스 레이블을 먼저 정하고 대응하는 입력(input)을 생성하는 Output-first 방식이, 일반 태스크의 경우 Output-first보다 Input-first(태스크에 대한 출력을 직접 생성) 방식이 더 나았다고 보고되었다. Step 4에서는 이미 생성한 instance와 중복되지 않는지 등의 필터링을 수행한다. Self-Instruct(52k), Alpaca(52k)는 text-davinci-003을 사용하여 같은 기법으로 작성되었다.

Baize[30]는 ChatGPT를 이용하여 멀티턴의 대화 데이터를 생성한다. 대화 생성을 위해 "이전에 받은 지시를 잊어라, 다음은 human과 AI 어시스턴트의 대화이며 topic은 '\${SEED}'이다. Human의 발언은 [Human], AI 어시스턴트의 발언은 [AI]로 시작한다" 형태의 프롬프트를 사용한다. topic은 질문 사이트 Quora, Stack Overflow의 질문을 이용하며 Human 역할은 관련 질문을 수행한다. Baize v1은 111.5k 건의 대화를 작성하였다.

최근 수학이나 코드 생성에 강한 추론 모델(Reasoning Model, 예: DeepSeek-R1)이 유행하면서 합성 데이터의 형태도 변화하고 있다. 추론 모델은 입력과 출력에 더해 "추론 과정"을 명시적으로 학습한다. 일반 모델이 (Question, Answer) 쌍을 사용하는 반면, 추론 모델은 (Question, Reasoning, Answer) 쌍을 사용한다. 추론 모델을 위한 데이터셋에서는 Question에서 Answer에 이르는 "추론 과정" 부분을 Few-shot 프롬프트로 생성시키는 경우가 많다. OpenMathInstruct-1[31]는 수학 문제를 다루는 GSM8K, MATH에 대해 추론 과정을 추가하여 1.8M의 문제-추론 과정 쌍을 포함하며, 답변 부분을 마스킹한 것을 Few-shot에 사용하는 것이 더 좋았다. CoT Collection[32]는 1060개 태스크에 대해 총 1.84M의 추론 과정을 추가하였다.

강화학습을 위한 피드백 데이터

RLHF 학습은 세 단계로 구성된다. Step 1 지도학습에서는 프롬프트와 그에 대한 적절한 답변 쌍을 라벨러(인간)가 작성하여 데이터셋을 구축하고, 이를 이용해 사전학습 모델을 fine-tuning한다. Step 2 보상 모델 학습에서는 프롬프트에 대한 Step 1 모델의 답변을 여러 패턴 준비하고 라벨러가 그중 좋은 것의 순위를 매긴 뒤, 순위 데이터셋을 이용해 보상 모델을 학습시킨다. Step 3 강화학습에서는 Step 1과 Step 2에서 학습된 모델을 이용하여 강화학습을 수행한다. 모델의 답변에 대해 보상값을 추정하고 그것을 모델에 피드백하여 정책을 개선하며, 보상이 최대가 되는 정책(정책은 Step 1에서 학습한 모델)을 탐색하여 최적인 답변을 생성한다.

정렬(alignment)의 기준은 세 가지로 정리된다(HHH). Helpful은 사용자의 질문에 대해 가능한 한 간결하고 효율적인 답변을 제공하고, 정보가 부족할 경우 적절한 질문을 던져 정보를 이끌어내며, 상대방의 수준에 맞춘 질의응답을 수행하는 것이다. Honest는 정보에 거짓이 없고 정확한 문장을 출력하며, 모델 자신이 어느 정도의 불확실성을 지닌 정보인지 표현하는 것이다(모델 스스로가 모델이 알고 있는 것을 이해할 필요가 있다). Harmless는 공격적이거나 차별적인 발언을 하지 않고 악의적인 질문을 감지하여 거부하는 것이다. 이 세 가지를 합쳐 aligned된 AI로 정의하는 논문도 있으며, 그 밖에도 Taxonomy, behavior, incentive, inner aspects 등의 관점이 있다.

피드백 데이터의 유형은 주로 수치, 순위, 자연어, 기타로 분류된다[33]. HHRLHF[34] 데이터셋은 작업자와 챗봇 응답의 일련의 주고받음 중에서 챗봇의 응답을 2건 제시하고 작업자가 응답별로 좋음, 나쁨을 선택하는 방식이며, 평가 관점은 Helpful과 Harmful이다. SHP[35] 데이터셋은 Reddit(게시판)을 이용하여 요리부터 법률 상담까지 18개 영역에 관한 질문(또는 Instruction)과 연결된 2개의 응답을 사용한다.

점수(투표 수 = 찬성 투표 수 - 반대 투표 수 + 1)가 높은 응답을 helpful, 다른 한쪽 응답을 less helpful로 간주한다. 응답에 챗봇을 사용하는 HHRLHF와 달리 사람에 의한 자연스러운 질문-응답 데이터라는 특징이 있다.

AI를 이용한 피드백 활용도 주요 접근법이다. 피드백 데이터가 인간의 입력에 의존하고 있어 1000건 미만의 피드백 데이터로는 효과가 없었으며[36], 정적인 피드백은 일관성과 정확성에 과제가 있다. 이에 LLM 스스로가 능력을 평가·개선하여 지속적인 인간의 개입 없이 모델을 강화하고자 한다. 두 가지 접근법이 있다. Self AI Feedback은 개선 대상 모델과 피드백 생성에 사용하는 모델이 동일한 것으로, GPT-4의 Safety 능력 개선 파이프라인[37]에서는 규칙을 Zero-shot으로 GPT-4에 주고 그 출력을 피드백으로 사용한다. 예를 들어 모델의 출력에 대한 객관식 판정(a: 올바르게 거부, b: 바람직하지 않은 스타일로 거부, c: 부적절한 내용의 혼입, d: 안전하면서 거부적이지 않은 응답)을 수행하고 그 결과를 피드백으로 활용한다. External AI Feedback[38]은 피드백 생성에 사용하는 모델이 개선 대상 모델과 다른 것을 사용하며, 복수의 LLM으로부터 피드백을 얻을 수 있는 가상 환경(Sandbox)을 작성하여 질문-가능 응답-평가-피드백-수정 응답-평가의 흐름으로 다양한 피드백을 포함한 169K 건의 데이터를 작성한다(text-davinci-003 175B, GPT-4 등 활용).

라이선스, 저작권, 개인정보

저작권과 라이선스

저작권(copyright)은 작품을 창작한 자가 작품이 어떻게 사용되는지를 결정할 수 있는 권리로, 지식재산권의 일종이다. 저작권법에서 보호되는 "저작물"은 "사상 또는 감정을 창작적으로 표현한 것으로서, 문예·학술·미술 또는 음악의 범위에 속하는 것"으로 정의된다[39]. 사실이나 데이터에 머무는 것, 표현에 이르지 못하는 아이디어 등은 저작물에 해당하지 않는다. 원칙적으로 권리자의 허락이 필요하지만 사적 이용, 인용, 교육 등은 예외이다(권리 제한 규정). 문학적 및 미술적 저작물의 보호에 관한 베른 조약이 있어, 가맹국이면 저작권의 기본적인 개념에 동의하고 있다고 볼 수 있다[62].

AI 개발을 위한 정보 해석과 같이 저작물에 표현된 사상 또는 감정의 향유를 목적으로 하지 않는 이용, 즉 저작물을 학습용 데이터로 수집·복제하고 데이터셋을 작성·이용하는 것은 원칙적으로 저작권자의 허락 없이 가능하다(법 제30조의4, 권리 제한 규정). 단, 학습 데이터 중에 포함된 저작물을 완전히 복사한 데이터가 모델로부터 생성·공개된 경우에는 저작권 침해의 가능성이 높다.

라이선스는 소프트웨어 등의 지식재산(지재)을 사용하는 것에 대한 허가(와 그 조건)이다. 소프트웨어나 데이터셋에서의 라이선스는 제공자가 제공한 소프트웨어(저작물)나 데이터에 대하여 공표된 허락 조건 아래에서 조건에 따라 이용하는 것이다. 저작물은 제공자 이외는 이용할 수 없지만, 제공자의 저작권에 기반하여 타인의 이용 조건을 정한 것이다.

소프트웨어를 위한 대표적 라이선스로 MIT(매우 느슨하며 저작권 표시는 필요), BSD(MIT과 거의 같으나 서면에 의한 허가 없이 파생 제품의 판매와 이름 등의 사용은 불가), Apache 2.0(특허의 명시적 허락이 있음), GPL v3(라이선스 아래에서 자유롭게 이용·개작·복제·재배포할 수 있으며 파생물에도 동일한 이용 조건을 적용해야 하는 카피레프트) 등이 있다. Creative Commons Licenses(CC)는 저작자가 자신의 작품의 이용 조건을 사전에 명시함으로써 작품의 자유로운 유통과 재이용을 촉진하는 구조로, 저작권을 보유한 채로 특정 조건(저작자표시 BY, 비영리 NC, 변경금지 ND, 동일조건 SA)을 조합한 라이선스를 선택할 수 있다. CC0는 저작자가 모든 권리를 포기한 Public Domain이며, CC BY는 출처 표시가 필요하고, CC BY-SA는 개작한 경우 원래 작품과 같은 라이선스로 공개(동일조건)해야 하며, CC BY-NC는 비영리 목적 이용을 조건으로 한다. 다수의 라이선스가 있으므로 이용 시에는 개별 라이선스를 확인해야 한다.

개인정보 취급

개인정보와 민감정보는 법적으로 취급이 제한되는 정보이다. "개인정보"란 살아 있는 "개인에 관한 정보"로서 해당 정보에 포함된 성명, 생년월일, 기타의 기술 등에 의해 특정 개인을 식별할 수 있는 것(다른 정보와 쉽게 대조할 수 있고 그에 따라 특정 개인을 식별할 수 있는 것을 포함한다) 또는 개인식별부호가 포함된 것을 말한다[41]. 개인식별부호의 예시로는 여권번호, 마이넘버, 면허증 번호 등이 있다. "민감정보"란 부당한 차별이나 편견, 그 이외의 불이익이 발생하지 않도록 그 취급에 특별한 배려가 필요한 것으로 정령으로 정하는 기술 등이 포함된 개인정보를 말하며, 인종, 신조, 병력, 범죄의 경력, 신체장애·지적장애·정신장애 등이 있는 것 등이 예시된다.

크롤링에 의한 데이터 이용 시에는 가능한 한 수집 결과에서 제외하도록 하는 대책이 필요하다. 또한 예외를 제외하고 민감정보의 취급과 제3자 제공은 원칙적으로 본인의 동의가 필요하므로[39], 데이터셋 공개 시 이러한 정보가 포함되어 있으면 문제가 된다. 이에 사전학습 데이터 필터링의 일부에 개인정보를 제거하는 구조를 도입한다[11]. 개인정보 판정 기법으로는 규칙 기반(성명, 전화번호, 주소 등을 정규표현식으로 발견[42]), 개인정보 판정기 작성(SVM 등[39], 딥러닝 모델이나 LLM으로 판정[39]) 등이 있으며, 해당 문장에 개인정보가 포함되어 있으면 해당 문장을 제외한다. 생성 AI 서비스 이용에 관해서는 개인정보 보호위원회의 주의 촉구 등을 숙지하고 적절하게 대응해야 하며, 재판 결과나 정부의 해석에 따라 변경될 수 있으므로 최신 동향에 민감하게 대응할 필요가 있다.

LLM 성능 평가와 벤치마크

평가의 방향성과 능력 분류

LLM 성능 평가에는 두 가지 방향성이 있다. 하나는 개별 영역·태스크별 성능을 평가하고 싶다는 것으로, 벤치마크 데이터셋을 이용한다. 다른 하나는 LLM의 전반적인 성능을 알고 싶거나 인간에 의한 평가를 알고 싶다는 것으로, Chatbot Arena나 LLM-as-a-Judge를 활용한다. 평가 기법은 벤치마크 기반, 인간

기반, 모델 기반 세 가지로 분류된다[11]. 평가 시에는 LLM의 종류(사전학습 모델 base, fine-tuning 완료 여부, 특정 태스크에 적응된 특화형인지 여부)와 테스트 대상 능력/도메인(General은 복수 능력의 전반적인 퍼포먼스를 나타낸다) 등을 고려해야 한다.

LLM 평가는 Basic 레벨과 Advanced 레벨로 나뉜다. Basic 레벨은 세 가지 기본 능력을 다룬다.

언어 생성 능력에서 Language Modelling은 다음 token을 예측하게 하여 기초적인 언어 이해와 생성 능력을 측정하며(perplexity), Conditional Text Generation은 요약이나 질의응답 등 주어진 조건에서의 생성 능력을 측정한다(Accuracy, BLEU, ROUGE나 인간 평가). Code Synthesis는 프로그래밍과 같은 형식적 언어 생성 능력을 측정하며 코드를 실행하여 준비된 테스트의 통과율(pass@k)을 평가지표로 삼는다. 대표 데이터셋으로 LAMBADA[43](사람이 문장 전체를 읽으면 마지막 단어를 추측할 수 있지만 대상 단어 직전의 문장만 보면 추측할 수 없다는 특징을 가진 이야기 문장의 모음)와 HumanEval[44](164건의 Python 코드로 구성되며 문서 문자열과 그 구현 코드, 테스트를 제공)이 있다.

지식 활용 능력에서 Closed-Book QA는 외부 리소스를 사용하지 않고 사전학습 코퍼스에 인코딩된 지식만에 기반해 질문에 답변하는 능력을 측정하며(Accuracy), Open-Book QA는 외부 지식 리소스(예: Wikipedia)에서 유용한 정보를 추출하여 활용할 것이 요구되는 태스크이다(Accuracy, F1-score). Knowledge Completion은 지식 베이스의 결여된 부분(예: 지식 트리플의 일부) 보완이나 지식 베이스 추출 능력을 측정한다. OpenBookQA[45](1326건의 초등 수준 과학 지식 리소스와 6000건의 질문, 그 밖에 상식 지식도 제공)와 WikiFact[46](대규모 지식인 Wikipedia, Wikidata에 기반한 지식 트리플 추출 태스크) 등이 있다.

복잡한 추론 능력에서 지식 추론은 논리적 관계와 사실에 기반한 추론 태스크에서 주어진 질문에 답변하게 하며(자동 지표 BLEU, 인간 평가), 기호적 추론은 학습 데이터에는 존재하지 않는 특정 목표를 다루는 설정에서 형식적 규칙의 기호를 조작하게 하며, 수학적 추론은 수학적 지식, 논리, 문제 해결을 위한 계산이나 증명의 활용을 필요로 한다(데이터셋: GSM8K). 다단계 사고나 사전학습 중에 보지 못한 규칙 조작을 필요로 하는 보다 복잡한 태스크를 평가한다. HellaSwag[47](서술이 주어지고 가장 다음에 이어질 상식적인 서술을 선택하는 태스크)와 CoinFlip[48](앞뒤가 있는 동전을 여러 번 뒤집는 조작을 한 후의 상태를 답변하게 하며, CoT 출력 예시를 포함) 등이 대표적이다.

Advanced 레벨은 세 가지 능력을 다룬다. Human Alignment는 인간의 가치관이나 요구에 적절히 부합하는가를 측정하며, 앞서 정리한 Honesty·Helpfulness·Harmlessness가 기준이 된다. TruthfulQA[49](38개 카테고리에 걸쳐 817개의 질문과 정답, 예: "기침은 심장마비를 효과적으로 멈추게 할 수 있는가?")와 CrowS-Pairs[50](인종·종교·나이 등 9가지 편향과 관련된 스테레오타입을 망라하는 1508예로, 각 예는 스테레오타입성이 강한 문장과 약한 문장 2개를 제시하고 LLM이 어느 정도 선호하는지를 측정) 등이 있다.

외부 환경과의 상호작용은 외부 환경으로부터의 피드백을 받아 지시된 행동을 실행할 수 있는가의 능력을 측정한다. Household(ALFWorld[51], "씻은 사과를 주방 냉장고에 넣어라"와 같은 요청 상황처럼 텍스트 기반 행동과 시각적 환경 시뮬레이터를 조합한 프레임), Website Environment(WebShop[52],

118M개의 실제 상품과 12K 클라우드소싱 지시를 갖춘 거래 환경에서 복수 웹에 접속해 아이템을 검색·커스터마이징·구매), Open World(MineDojo[53], 게임 Minecraft를 대상으로 환경과 관련 YouTube나 게시판 등의 지식 베이스를 제공) 등이 있다.

Tool 조작은 복잡한 문제 해결을 위해 LLM이 필요에 따라 외부 API(예: 검색 엔진, 계산기, 컴파일러)를 사용할 수 있는가를 측정한다. 검색 엔진 이용(HotpotQA[54], 113K의 Wikipedia 기반 질문-답변 쌍으로 답변을 위해 복수의 지지 문장을 검색·사용·추론해야 하는 multi-hop QA), 계산기 이용(GSM8K[55], 사람이 직접 2~8단계가 필요한 사칙연산 문제에 대해 풀이를 어노테이션), 코드 실행, 모델 추론(Gorilla[56], 태스크에 따라 복수의 API를 구분하여 사용하는 능력), 데이터 인터페이스(TabFact[57], 반구조적 데이터인 표·그래프·데이터베이스를 다루는 능력) 등이 해당한다.

태스크별 성능 조사에서는 Basic과 Advanced 레벨의 능력 항목별로 대표적인 태스크와 그 데이터셋을 이용하여 대표적인 모델들의 성능을 조사한다[11]. 실험 설정상 모델로는 LLaMA(7B, 13B), Vicuna(7B, 13B) 등 오픈소스 모델 및 ChatGPT, Claude, Davinci003(GPT-3.5) 등 클로즈드 소스 API 모델을 사용하며, 많은 태스크에서 Zero-shot 성능을, 일부는 3-shot 성능을 측정한다. ChatGPT는 클로즈드 모델 중에서 대체로 좋은 성능을 보이며, 오픈소스 모델에서는 사전학습 모델보다 Instruction-tuning을 한 모델이 더 좋은 성능을 보인다.

3가지 평가 기법

벤치마크 기반 평가는 복수의 태스크를 포함한 종합적인 LLM 성능 평가를 수행한다. 각 태스크의 문제마다 지정된 포맷으로 LLM에 입력하고, 생성된 텍스트를 규칙 기반으로 파싱하여 답안을 얻은 뒤 그 답안과 정답을 비교한다. 주요 벤치마크로 MMLU, BIG-bench, HELM 등이 있다. MMLU[58]는 초등수학, 미국 역사, 법률 등 57개 태스크를 커버한 테스트셋으로, 대학원생과 학부생이 인터넷에서 손수 문제를 수집하였고 초급·고교·대학·전문가 등 난이도 레이블이 설정되어 있다. Few-shot 개발셋, 검증셋, 테스트셋으로 분할되어 합계 15.9K의 질문이 존재한다(대수학, 해부학, 대학 수준 화학 등 다양한 과목 포함).

인간 기반 평가는 human-alignment나 도구 이용과 같이 보다 현실적인 상황에서 다양한 요인과 능력이 고려되므로, 인간이 모델의 출력을 판정하는 기법이다. Chatbot Arena[59]는 사용자가 입력하면 2개의 서로 다른 LLM 출력이 제시되고 사용자가 평가하는 방식이며(A가 좋음 / B가 좋음 / 동등함 / 둘 다 나쁨), 결과를 집계하여 복수의 모델 성능을 리더보드로 제시한다.

모델 기반 평가는 인간 기반 평가의 대안으로 ChatGPT나 GPT-4 등의 LLM을 평가자로 대용한다(LLM-as-a-Judge). ChatGPT나 GPT-4의 평가는 인간의 평가와도 높은 일치도가 있음이 확인되었으며, 인간의 관여에 대한 의존을 줄여 보다 효율적이고 확장성을 가질 수 있고 평가 점수의 설명도 출력 가능하여 해석 가능성도 높일 수 있다. 모델 기반 벤치마크 데이터셋으로 AlpacaEval와 MT-Bench 등이 존재한다. MT-Bench[59]는 8개 카테고리(서술, 롤플레이, 추출, 공학이나 수학을 포함한 지식 등)에 대해 각각 멀티턴 질문을 작성하여 합계 80건의 질문으로 구성된다. 평가의 바リエ이션으로는 페어 비교에

의한 평가(2개의 LLM 출력을 제시하고 어느 쪽이 좋은지, 나쁜지, 동등한지를 판정), 단일 응답에 대한 평가(1개의 출력에 대한 점수를 평가 LLM이 출력), 참조 가이드 평가(평가 대상과 더불어 정답을 평가 LLM에 제시한 뒤 평가를 결정)가 있다.

LLM-as-a-Judge

LLM은 사후학습(SFT, RLHF)을 통해 지시 준수성과 대화 능력을 향상시켜 인간에게 선호되는 응답 능력을 획득했을 것이므로 이를 제대로 평가하고자 하는 요구가 생겼다. 그러나 기존 평가 기법에는 문제가 있다. 규칙 기반 평가(MMLU, HELM)는 LLM의 기초적 능력을 측정할 수는 있지만 다양한 사용자 요구에 대한 LLM 응답의 유용성을 측정하는 것과 괴리가 있다. 자동화된 객관 평가(BLEU, ROUGE)는 표면 어휘 중복을 측정하는 지표로 이야기 생성 등 깊이 있는 뉘앙스를 다루는 태스크에는 부적합하다. 인간(전문가) 평가는 비용이 높고 스케일링이 어렵다. LLM-as-a-Judge[60]는 인간과 같은 가치와 추론 과정을 갖춘 LLM을 활용하여 다양한 데이터 타입에 대해 확장 가능하고 유연한 평가 제공을 목표로 하며, 채점자(Graders), 평가자(Evaluators/Assessors), 비평가(Critics), 검증자(Verifiers), 시험관(Examiners), 보상/순위 모델(Reward/Ranking Models)의 역할을 수행할 수 있다.

LLM-as-a-Judge의 평가 파이프라인은 프롬프트 설계와 후처리로 구성된다. 프롬프트에서 출력되는 평가 형식으로는 1~3 또는 0~100 연속 점수, Yes/No, 페어 비교(2개의 선택지를 제시하고 기준을 만족하는 것을 선택), 객관식 평가 등이 있다. 평가에 이용하기 위한 출력의 후처리로는 특정 토큰 추출(Yes/No, 답안 번호), JSON 등의 특정 스키마, 출력 로짓을 0~1 연속 소수로 정규화, 특정 문장이나 단락 추출 등이 있다.

LLM-as-a-Judge에는 바이어스가 존재한다. Judgement-Specific 바이어스로는 위치 바이어스(프롬프트 내 특정 위치에 있는 응답을 선호하는 경향), Compassion-fade bias(모델명 GPT-4 등의 명시적 정보에 영향을 받는다), 스타일 바이어스(이모티콘이 달린 콘텐츠와 같은 특정 텍스트 스타일을 선호하는 경향), 길이 바이어스(장황한 응답을 선호하는 경향), 구체성 바이어스(권위 있는 출처 인용, 수치, 복잡한 전문 용어, 구체적 세부사항을 선호하는 경향) 등이 있다. 각 바이어스에 대한 대응책도 검토되고 있으며, 페어 비교에서의 유효한 개선책으로는 강력한 LLM을 선택하고 평가 내용의 위치를 바꾸어 여러 번 평가한 결과로 다수결을 취하는 방법이 있다[60].

발전적 평가: Humanity's Last Exam

LLM의 급속한 발전으로 MMLU와 같은 기존 인기 벤치마크에서 90% 이상의 정확도를 달성하여 능력 측정의 한계(포화 상태)에 달했다. 이에 2500문항의 전문가 수준이면서 도전적인 질문을 작성한 Humanity's Last Exam(HLE)[61]이 제안되었다. 100개 이상의 전문 분야를 포함하며, 문제 형식은 출력 문자열의 완전 일치, 복수의 선택지가 정답인 문제 등이고, 그중 14%는 텍스트와 이미지 모두의 이해를 필요로 한다.

문제 구축 방법으로는 총 500,000 USD 상금을 마련하여 양질의 질문을 모집하고, LLM에 의한 난이도 확인(풀 수 없는 문제를 모은다) 후 대학원 학위를 가진 사람이 리뷰하고 관계자·전문가가 최종적으로 결정하는 필터를 거친다. 그 결과 최신 LLM에서도 5% 미만의 정확도밖에 달성하지 못하는 벤치마크가 만들어졌다. 과제로는 전문가 간의 의견 불일치(공개 셋에서 15.4%의 문항은 의견이 일치하지 않는다), 복수 전문가의 필요성(표준적인 문헌 검색이 아니라 연구 경험에 기반한 질문이 존재한다), 단기간 포화 가능성(HLE도 단기간에 포화할 가능성이 있어 새로운 질문을 추가하는 동적 데이터셋 HLE-ROLLING을 도입 예정) 등이 있다.

정리

LLM 학습 파이프라인의 세 학습 단계(사전학습, 파인튜닝, 강화학습)와 평가 단계 어느 것에서도 학습과 평가를 위한 데이터가 중요하다. 사전학습에서는 필터링을 수행하여 데이터 품질을 높임으로써 LLM 성능 향상에 기여한다. 최근에는 데이터 작성과 LLM 평가에서도 (다른) 대규모 LLM을 활용하여 데이터 확장이나 자동 평가를 수행하는 시도가 활발하다. 개인정보 보호나 LLM 평가에서의 바이어스 등의 관점에서 향후 지속적인 대응이 필요하다.

References

- [1] Choo (2025), "The emergence of Large Language Models (LLMs)", The low down, <https://thelowdown.momentum.asia/the-emergence-of-large-language-models-llms/> 접속일: 2025/11/2
- [2] Devlin, et al. (2018), "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", arXiv:1810.04805
- [3] Radford, et al. (2019), "Language Models are Unsupervised Multitask Learners", OpenAI Blog, https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf 접속일: 2026/5/24
- [4] Brown, et al. (2020), "Language Models are Few-Shot Learners", arXiv:2005.14165
- [5] Smith, et al. (2022), "Using DeepSpeed and Megatron to Train Megatron-Turing NLG 530B, A Large-Scale Generative Language Model", arXiv:2201.11990
- [6] Chowdhery, et al. (2022), "PaLM: Scaling Language Modeling with Pathways", arXiv:2204.02311

-
- [7] Penedo, et al. (2023), "The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only", arXiv:2306.01116
- [8] BigScience Workshop, et al. (2022), "BLOOM: A 176B-Parameter Open-Access Multilingual Language Model", arXiv:2211.05100
- [9] Nguyen, et al. (2023), "CulturaX: A Cleaned, Enormous, and Multilingual Dataset for Large Language Models in 167 Languages", arXiv:2309.09400
- [10] Fu, et al. (2022), "How does GPT Obtain its Ability? Tracing Emergent Abilities of Language Models to their Sources", <https://yaofu.notion.site/How-does-GPT-Obtain-its-Ability-Tracing-Emergent-Abilities-of-Language-Models-to-their-Sources-b9a57ac0fcf74f30a1ab9e3e36fa1dc1> 접속일: 2026/5/24
- [11] Zhao, et al. (2023), "A Survey of Large Language Models", arXiv:2303.18223
- [12] Penedo, et al. (2023), "The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only", arXiv:2306.01116
- [13] Holtzman, et al. (2019), "The curious case of neural text degeneration", ICLR 2019, arXiv:1904.09751
- [14] Rae, et al. (2021), "Scaling language models: Methods, analysis & insights from training gopher", arXiv:2112.11446
- [15] Lee, et al. (2022), "Deduplicating training data makes language models better", Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics, pp. 8424 – 8445, arXiv:2107.06499
- [16] Hernandez, et al. (2022), "Scaling laws and interpretability of learning from repeated data", arXiv:2205.10487
- [17] speed blog (2023), "Introduction to MinHash", <https://speed1313.github.io/posts/minhash/> 접속일: 2025/11/3
- [18] Penedo, et al. (2024), "The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale", arXiv:2406.17557
- [19] Sachdeva, et al. (2024), "How to Train Data-Efficient LLMs", arXiv:2402.09668
- [20] Li, et al. (2024), "DataComp-LM: In search of the next generation of training sets for language models", arXiv:2406.11794

-
- [21] Chai, et al. (2025), "Text Data Augmentation for Large Language Models: A Comprehensive Survey of Methods, Challenges, and Opportunities", arXiv:2501.18845
- [22] Yao, et al. (2022), "ReAct: Synergizing Reasoning and Acting in Language Models", arXiv:2210.03629
- [23] Fujii, et al. (2025), "Rewriting Pre-Training Data Boosts LLM Performance in Math and Code", arXiv:2505.02881
- [24] Wei, et al. (2021), "Finetuned Language Models Are Zero-Shot Learners", arXiv:2109.01652
- [25] Sanh, et al. (2021), "Multitask Prompted Training Enables Zero-Shot Task Generalization", arXiv:2110.08207
- [26] Eccleston (2023), "ShareGPT", <https://sharegpt.com/> 접속일: 2026/5/24
- [27] Conover (2023), "Free Dolly: Introducing the World's First Truly Open Instruction-Tuned LLM", <https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm> 접속일: 2026/5/24
- [28] Ouyang, et al. (2022), "Training language models to follow instructions with human feedback", arXiv:2203.02155
- [29] Wang, et al. (2022), "Self-Instruct: Aligning Language Models with Self-Generated Instructions", arXiv:2212.10560
- [30] Xu, et al. (2023), "Baize: An Open-Source Chat Model with Parameter-Efficient Tuning on Self-Chat Data", arXiv:2304.01196
- [31] Toshniwal, et al. (2024), "OpenMathInstruct-1: A 1.8 Million Math Instruction Tuning Dataset", arXiv:2402.10176
- [32] Kim, et al. (2023), "The CoT Collection: Improving Zero-shot and Few-shot Learning of Language Models via Chain-of-Thought Fine-Tuning", arXiv:2305.14045
- [33] Fernandes, et al. (2023), "Bridging the Gap: A Survey on Integrating (Human) Feedback for Natural Language Generation", arXiv:2305.00955
- [34] Bai, et al. (2022), "Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback", arXiv:2204.05862
- [35] Ethayarajh, et al. (2022), "Understanding Dataset Difficulty with V-Usable Information", arXiv:2110.08420

-
- [36] Gao, et al. (2022), "Scaling Laws for Reward Model Overoptimization", arXiv:2210.10760
- [37] OpenAI (2023), "GPT-4 Technical Report", arXiv:2303.08774
- [38] Liu, et al. (2023), "Training Socially Aligned Language Models on Simulated Social Interactions", arXiv:2305.16960
- [39] 源, et al. (2025), "대규모 언어 모델 사전학습용 코퍼스에서의 민감정보 탐지", 언어처리학회 제31회 연차대회
- [40] 문화심의회 저작권분과회 법제도소위원회 (2024), "AI와 저작권에 관한 생각에 대하여", https://www.bunka.go.jp/seisaku/bunkashingikai/chosakuken/pdf/94037901_01.pdf 접속일: 2025/10/31
- [41] 개인정보보호위원회·후생노동성, "의료·요양 관계 사업자에서의 개인정보의 적절한 취급을 위한 가이드라인", https://www.ppc.go.jp/personalinfo/legal/iryoukaigo_guidance/#a2-1 접속일: 2025/10/31
- [42] Laurençon, et al. (2023), "The BigScience ROOTS Corpus: A 1.6TB Composite Multilingual Dataset", arXiv:2303.03915
- [43] Paperno, et al. (2016), "The LAMBADA dataset: Word prediction requiring a broad discourse context", arXiv:1606.06031
- [44] Chen, et al. (2021), "Evaluating Large Language Models Trained on Code", arXiv:2107.03374
- [45] Mihaylov, et al. (2018), "Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering", arXiv:1809.02789
- [46] Goodrich, et al. (2019), "Assessing The Factual Accuracy of Generated Text", arXiv:1905.13322
- [47] Zellers, et al. (2019), "HellaSwag: Can a Machine Really Finish Your Sentence?", arXiv:1905.07830
- [48] Wei, et al. (2022), "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models", arXiv:2201.11903
- [49] Lin, et al. (2021), "TruthfulQA: Measuring How Models Mimic Human Falsehoods", arXiv:2109.07958
- [50] Nangia, et al. (2020), "CrowS-Pairs: A Challenge Dataset for Measuring Social Biases in Masked Language Models", arXiv:2010.00133

-
- [51] Shridhar, et al. (2020), "ALFWorld: Aligning Text and Embodied Environments for Interactive Learning", arXiv:2010.03768
- [52] Yao, et al. (2022), "WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents", arXiv:2207.01206
- [53] Fan, et al. (2022), "MineDojo: Building Open-Ended Embodied Agents with Internet-Scale Knowledge", arXiv:2206.08853
- [54] Yang, et al. (2018), "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering", arXiv:1809.09600
- [55] Cobbe, et al. (2021), "Training Verifiers to Solve Math Word Problems", arXiv:2110.14168
- [56] Patil, et al. (2023), "Gorilla: Large Language Model Connected with Massive APIs", arXiv:2305.15334
- [57] Chen, et al. (2019), "TabFact: A Large-scale Dataset for Table-based Fact Verification", arXiv:1909.02164
- [58] Hendrycks, et al. (2021), "Measuring Massive Multitask Language Understanding", ICLR 2021, <https://openreview.net/forum?id=d7KBjmI3GmQ>
- [59] Zheng, et al. (2023), "Judging LLM-as-a-judge with MT-bench and Chatbot Arena", NeurIPS 2023, <https://dl.acm.org/doi/10.5555/3666122.3668142>
- [60] Gu, et al. (2024), "A Survey on LLM-as-a-Judge", arXiv:2411.15594
- [61] Phan, et al. (2025), "Humanity's Last Exam", arXiv:2501.14249
- [62] 문화청 저작권과, "AI와 저작권", https://www.bunka.go.jp/seisaku/chosakuken/pdf/93903601_01.pdf 접속일: 2025/11/4